

基于滚动预测与多阶段优化的微网协同调度模型

摘要

本文针对微网与外部电网的电力调控问题，综合考虑购电和储能两部分，建立了由混合整数线性规划、时间序列滚动预测、分位数风险修正和动态电价预测等算法组成的购电与储能调度模型，为微网在不同信息条件和电价机制下制定经济、可靠的购电策略。

针对问题一，我们以购电量、充放电量、储电量和充放电状态为决策变量，以全天购电费用最小为目标，建立混合整数线性规划（MILP）模型。通过使用 HiGHS 求解器，我们求得最优计划：全天购电量约为 59482.70 kWh，购电费约为 35126.95 元，全天充电量和放电量分别约为 20740.67 kWh 和 16799.94 kWh，储能损耗约为 3940.73 kWh。

针对问题二，在负载和光伏均存在不确定性的条件下，我们建立了历史数据驱动的日前购电模型。负载预测采用 31 天滚动岭回归，光伏预测则采用扩展窗口岭回归。在计算计划量时，我们为避免仅按点预测安排购电，采用风险裕度对净负荷进行风险修正，再通过每日储能 MILP 确定计划购电量、充放电量 and 富余量。最终全年计划购电量约为 22046434.84 kWh，紧急购电量约为 126347.96 kWh，总费用约为 14224462.52 元。结果表明，风险裕度分位数修正能在购电成本与缺电风险之间取得较好的平衡。

针对问题三，我们建立了多阶段滚动优化模型，其中负载继续采用问题二的滚动岭回归模型预测，光伏则只采用附件 3 的整点预报数据。为提高 10 分钟粒度预报精度，我们通过线性插值得到每 10 分钟的光伏发电量预报值。我们通过风险裕度搜索，得到误差分位数 (用于量化风险裕度大小) 均为 0.60。全年总费用约为 13557776.15 元，紧急购电量降至约 47836.98 kWh。后续的信息价值分析表明，现有四个预报时刻已覆盖主要可削减成本的信息，无需增加其它预报时点。

针对问题四，我们建立了基于动态电价预测的购电优化模型，利用过去 7 天日均电价构造逐时段滚动预测电价模型，分别重新求解问题 2 和 3。计划购电和调整费用按预测电价计算，紧急购电费用按实际电价计算。电价预测模型的逐时段 R^2 为 0.8709，这说明了我们的模型能够较好的预测电价。经过计算，我们得出问题 4-2 总费用约为 14223905.42 元，问题 4-3 总费用约为 13536142.45 元，比 4-2 节省约 687762.97 元，说明多阶段调整确实能够有效降低实时电价环境下的紧急购电风险。

关键词：微网调度；混合整数线性规划；滚动时间序列预测；分位数风险修正；多阶段调整；动态电价

1 问题重述

微网是一种将光伏发电、小区负载和储能设备等分布式资源进行本地协调的小型发用电系统^[7]。如何根据电价、小区负载、光伏发电功率和储能设备当前储电量，合理安排各时段的计划购电量与储能充放电量，是微网与外部电网协同调控的关键。

由题意，储能设备最大容量为 12000 kWh，最大充放电功率为 5000 kW，2025 年 1 月 1 日 0:00 的初始储电量为 6000 kWh。为保证设备安全运行，后续过程中，储能设备的储电量必须始终保持在 1200-10800 kWh 之间，充放电效率均为 90%。

根据附件提供的数据，本题需要依次解决以下四个问题：

- (1) 在每天的电价和小区负载相同的前提下，每天 0:00 制定当天的计划购电策略，即当天每个 10 分钟时段的购电量，并给出全天购电量和购电费，以及六个四小时区间储能设备的充放电量和起止时刻储电量。
- (2) 每天的电价相同，但小区负载和光伏发电功率随时间变化，紧急情况下需要高价购电，利用实际小区负载与光伏发电功率历史数据，在每天 0:00 预测当天负载和光伏发电量，结合实际电价，制定当天的计划购电策略，并给出指定日期的购电量、充放电量、储电量及紧急购电结果。
- (3) 系统根据 0 点之前的历史数据和每个时间节点的将来光伏发电功率预报调整购电策略。若计划购电量与调整后的购电量不同，则要求支付相应的调整费用。需要建立包含计划购电费用、紧急购电费用和调整费用的优化模型，并分析是否需要引入其他时刻的预报。
- (4) 在外网电价随实际交易时刻波动的情况下，我们需要利用动态电价数据预测当天电价，重新建立微网购电模型，并依据已知历史数据分别重新求解问题 2 和问题 3。

2 问题分析

本文要求建立模型来研究微网与外部电网之间的电力调控策略。核心任务是在储能设备运行约束下，根据电价、小区负载、光伏发电功率和当前储电量，合理确定计划购电量及储能充放电量，使微网既能满足小区负载需求，又能尽可能降低购电费用^[7]。本文按照“确定性优化—时间序列预测—场景优化—多阶段调整”的思路展开^[7]。图 1 清晰地展示了整个系统电力资源的流向与分布，总体上由外网和光伏发电来提供能量，供给小区负载并把多余的电能存储在储能设备中^[7]。

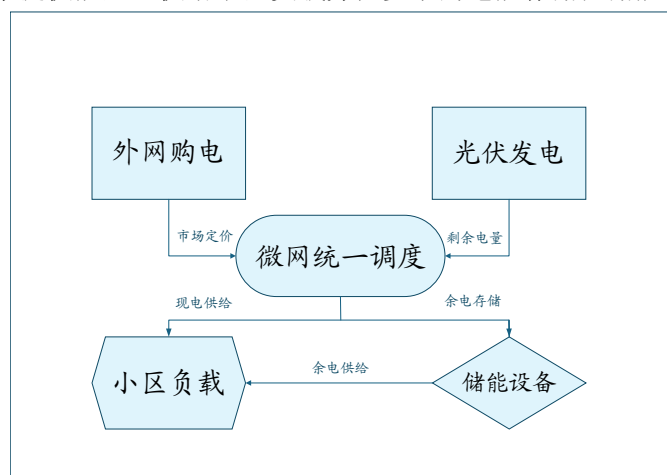


图 1: 系统电量调配路径示意图

2.1 问题一的分析

问题一是电价、小区负载和光伏发电预测功率均已知的确定性日前调度问题。其中充放电时间窗口为 10 分钟，相对较短，在同一时段内充放电明显会提升问题复杂度。为简化问题，我们采用离散决策，通过设置决策变量 $z_{d,t}$ 来决定某一时段是否充放电。我们将问题一的求解过程划分为以下几个步骤：

- (1) 将一天划分为 144 个 10 分钟时段，把已知的各时段负载功率和光伏发电功率换算为对应的负载电量和光伏发电量。
- (2) 设置购电量、充电量、放电量、时段末储电量和充放电状态五个决策变量，以全天购电费用最小为目标，建立混合整数线性规划模型 (MILP)。
- (3) 依次引入逐时供需平衡、储电量递推、储电量上下限、最大充放电功率、充放电互斥以及日初日末储电量相等约束，保证调度计划满足负载需求和储能设备安全运行的条件。
- (4) 利用 HiGHS 求解器求得 MILP 最优解，并通过独立计算供需残差、储电量递推残差和功率约束残差检验所得计划的可执行性。

2.2 问题二的分析

问题二中每天相同时段的电价不变，但小区负载和光伏发电功率逐日、逐时段变化。我们通过历史数据建立小区负载和光伏发电量的时间序列预测模型。预测数据用于指导优化 0:00 的计划购电策略，在实际运行中根据情况紧急购电，进而计算最后的总费用。具体分为以下几个步骤：

- (1) 利用 2025 年 1 月数据进行模型参数选择，并计算 2025 年 2 月 1 日 0:00 的初始储电量。针对小区负载，使用前 1、2、3、7 日的滞后值、前 3 日和 7 日的同时刻均值、最近两个相同星期数日期的均值，同时考虑星期周期、年内周期和日内周期特征，建立 31 天滚动岭回归预测模型。
- (2) 针对光伏发电，引入前 1、2、3、7、14 日的滞后值，前 3 日、7 日和 14 日的同时刻均值，同时考虑由太阳赤纬描述的年内季节特征和日内周期特征，建立扩展窗口岭回归预测模型；随后将岭回归结果与最近 7 日同时刻均值进行加权融合，并对结果进行非负截断。
- (3) 根据负载和光伏预测计算净负荷 (同一时段小区负载与光伏发电量之差)，并利用最近 60 天同一时段的滚动预测误差确定具体的风险裕度系数和相应的裕度值。将该裕度值与预测净负荷相加，得到计划净负荷。由于实际缺口需要按 5 倍电价紧急购电，我们需要在计划购电成本和缺电风险之间建立量化平衡，避免仅按预测均值安排购电。
- (4) 以计划购电费最小为目标，综合考虑多个约束条件，在当天 0:00 求解储能调度 MILP 模型。在微网实际运行阶段，使用当天的真实负载和光伏逐时段发电功率计算紧急购电量。
- (5) 按照上述流程，滚动求解 2025 年 2 月 1 日至 2025 年 12 月 31 日的计划购电量、实际充放电量、紧急购电时段和总费用。

2.3 问题三的分析

问题三在第二问的基础上增加了日内光伏预报更新和购电调整机制。每天有四个预报发布的时刻：0:00、6:00、12:00 和 18:00。通过预报数据，在相应时刻调整供电量。为刻画信息更新、预报订正和费用结算过程，我们将问题三分为以下几个步骤：

- (1) 首先对预报数据进行历史回归，明确光伏发电量的变化规律。计算目标日之前若干天光伏发电功率的实际小时均值，建立一元线性回归，插值求解光伏发电量每 10 分钟的预测值。
- (2) 本文采用**分段裕度系数**控制当天计划购电的保守程度。对每天的前 36 个时段和后 108 个时段，我们使用最近 60 天预测数据的误差分别进行修正，同时遍历裕度系数，得到夜间和白天的最优分位数作为最终裕度系数。
- (3) 使用计划购电与储能 MILP 模型，求解全天计划购电量、充放电量、富余量和储电量。在 6:00、12:00 和 18:00 根据最新光伏预报、已发生实际数据和当前储电量，调整尚未执行的当天后续时段。在实际运行时优先消耗储能设备中的富余电量，剩余差值按 5 倍电价紧急购电。

2.4 问题四的分析

问题四进一步考虑外网电价随交易时刻的波动，分别重新计算问题 2 和问题 3。此时电价不再是已知的固定曲线，我们需要依据历史电价和日内规律进行预测和调整。实际运行中的紧急购电费用按照实际电价结算。将问题四分为以下步骤：

- (1) 分析并总结实际电价的变化趋势和规律，设计 Rolling 滚动均值预测模型，在 0:00 预测当天各个时段的电价。
- (2) 重新计算问题 2 时，使用预测电价来建立储能 MILP 模型；计算问题 3 时，使用预测电价来建立多阶段调整-储能优化模型，求解总费用、计划购电量、充放电量 and 紧急购电量。实际运行中，紧急购电费使用实际电价计算。

3 模型假设

1. 假设附件中的电价、小区负载和光伏发电功率数据真实可靠。各时段的功率数据视为该 10 分钟区间内的平均功率。
2. 题目所给时间标签表示对应时段的结束时刻，例如附件中的 0:10 表示 0:00–0:10 时段，0:00+1 表示当天 24:00。
3. 假设外网能够完全满足微网的计划购电和紧急购电需求，不考虑外网故障、限电或供电中断。
4. 模型不考虑储能设备的自放电以及电池老化、循环寿命和折旧费用。
5. 同一 10 分钟时段内储能设备最多只能充电、放电或闲置，不能同时进行充电和放电；每个时段内的平均充放电功率不超过设备功率上限。
6. 问题一中光伏预测电量全部用于供应小区负载或为储能设备充电，不向外网售电，也不允许浪费光伏设备的发电量；问题二至问题四中未被利用的富余电量不能获得退款或售电收益。
7. 紧急购电只在储能调节后仍然存在供电缺口时发生，且不考虑额外手续费、容量费用或违约以外的其他惩罚费用。
8. 模型不考虑通信延迟、设备故障、人工操作误差以及题目未给出的其他随机因素。

4 符号说明

表 1: 重要符号表

符号	定义说明	单位
d, t	日期索引和日内时段索引, $t = 1, \dots, 144$	—
τ	日内预报发布与决策时刻, $\tau \in \{0, 6, 12, 18\}$	时
Δt	每个时段的长度, $\Delta t = 1/6$	h
$p_t, \hat{p}_{d,t}$	固定时段电价及第 d 天的预测动态电价	元/kWh
$p_{d,t}^{\text{act}}$	第 d 天第 t 时段的实际动态电价	元/kWh
$L_{d,t}, V_{d,t}$	实际小区负载功率和实际光伏功率	kW
$\hat{L}_{d,t}, \hat{V}_{d,t}$	负载预测功率和光伏预测功率	kW
$N_{d,t}$	实际净负荷电量, $N_{d,t} = (L_{d,t} - V_{d,t})\Delta t$	kWh
$\hat{N}_{d,t}$	预测净负荷电量	kWh
$N_{d,t}^{\text{plan}}$	经风险修正后的计划净负荷电量	kWh
$\varepsilon_{d,t}$	实际净负荷与预测净负荷之差	kWh
$q_{d,t}$	预测误差的历史分位数风险裕度	kWh
α_1, α_2	问题三夜间和白天采用的风险分位数	—
$g_{d,t}^{\text{plan}}$	0:00 确定的计划购电量	kWh
$g_{d,t}^{\text{adj}}$	根据新预报调整后的最终购电量	kWh
$e_{d,t}$	实际运行中的紧急购电量	kWh
$c_{d,t}, d_{d,t}$	电池充电量和放电量, 均按外部电量口径	kWh
$u_{d,t}$	未利用的富余电量	kWh
$S_{d,t}$	时段结束时电池的內部储电量	kWh
$z_{d,t}$	充放电状态二元变量	0–1
η_c	电池充电效率	—
η_d	电池放电效率	—
$S_{\min}, S_{\max}$	电池储电量上下限	kWh
S_{target}	日末储电量目标值	kWh
$C_{\max}, D_{\max}$	单个时段最大外部充电量和放电量	kWh
λ_L, λ_V	负载和光伏岭回归的正则化参数	—
α_V	光伏预测中最近 7 日均值的融合权重	—
δ_d	第 d 天的太阳赤纬	度
VOI	增加预报时刻所带来的信息价值	元
F, C_d	问题一全天购电费用及第 d 天总费用	元

5 模型建立与求解

5.1 问题一：确定性条件下的计划购电与储能优化模型

5.1.1 建模思路与时间离散化

电价、小区负载和光伏预测均为已知量，目标是在满足小区负载需求和储能设备安全约束的前提下，确定全天各时段的购电量以及储能充放电量，使全天购电费用最小。由于储能设备的当前动作会影响后续时段的可用容量，且同一时段不能同时充电和放电，该问题同时包含连续变量和 0-1 变量，因此本文将建模为混合整数线性规划问题^[7]。

我们将一天等分为 $T = 144$ 个时段，来对齐附件表格，每个时段 $\Delta t = 10min$ 。第 t 个时段记为 $[(t-1)\Delta t, t\Delta t]$ 。设第 t 时段的小区负载平均功率为 L_t ，光伏预测平均功率为 $\hat{P}_t$ ，则小区负载电量和光伏预测电量分别为

$$\ell_t = L_t \Delta t, \quad v_t = \hat{P}_t \Delta t \quad (1)$$

5.1.2 决策变量与目标函数

设第 t 时段从外网购入的计划电量为 g_t ，微网送入电池的外部电量为 c_t ，电池向微网交付的外部电量为 d_t ，时段结束时电池的內部储电量为 S_t 。我们引入 0-1 变量 z_t 表示充放电状态：

$$z_t = \begin{cases} 1, & \text{第 } t \text{ 时段充电,} \\ 0, & \text{第 } t \text{ 时段放电或闲置} \end{cases} \quad (2)$$

由于购电量 g_t 的单位为 kWh，目标函数可直接写为各时段购电量与购电价格乘积之和：

$$\min F = \sum_{t=1}^T p_t g_t \quad (3)$$

5.1.3 供需平衡与储能约束 (模型约束条件)

在模型假设下，每个时段获得的电量等于负载消耗与电池充电量之和，因此逐时供需平衡约束为

$$g_t + v_t + d_t = \ell_t + c_t, \quad t = 1, 2, \dots, T \quad (4)$$

整理可得

$$g_t = \ell_t - v_t + c_t - d_t \quad (5)$$

由该式可以看出，电池充电会增加购电需求，电池放电会减少购电需求，而光伏发电则会直接抵扣部分负载电量。

采用电池内部电量计算储电量，充放电效率分别为 η_c 和 η_d ，所得储电量递推关系为

$$S_t = S_{t-1} + \eta_c c_t - \frac{d_t}{\eta_d}, \quad t = 1, 2, \dots, T \quad (6)$$

其中 $\eta_c = \eta_d = 0.9$ 。电池运行过程中必须满足储电量上下限^[7]：

$$S_{\min} \leq S_t \leq S_{\max}, \quad S_{\min} = 1200, \quad S_{\max} = 10800 \quad (7)$$

同时，题目要求日初和日末储电量相同：

$$S_0 = S_T = 6000 \quad (8)$$

设电池的最大充电功率和最大放电功率分别为 $P_c^{\max} = P_d^{\max} = 5000$ kW，则每个时段的最大外部充放电量为

$$C_{\max} = P_c^{\max} \Delta t = \frac{2500}{3} \text{ kWh}, \quad D_{\max} = P_d^{\max} \Delta t = \frac{2500}{3} \text{ kWh}. \quad (9)$$

利用二元变量 z_t 建立充放电功率约束和互斥约束：

$$0 \leq c_t \leq C_{\max} z_t, \quad (10)$$

$$0 \leq d_t \leq D_{\max}(1 - z_t), \quad (11)$$

$$g_t \geq 0, \quad z_t \in \{0, 1\}$$

当 $z_t = 1$ 时， $d_t = 0$ ，电池只能充电；当 $z_t = 0$ 时， $c_t = 0$ ，电池只能放电或闲置，从而避免同一时段出现同时充放电的现象。

综合以上目标函数和约束条件，我们将问题一建立的完整模型表述如下

$$\begin{aligned} \min \quad & F = \sum_{t=1}^T p_t g_t \\ \text{s.t.} \quad & g_t + v_t + d_t = \ell_t + c_t, \quad t = 1, \dots, T, \\ & S_t = S_{t-1} + \eta_c c_t - \frac{d_t}{\eta_d}, \quad t = 1, \dots, T, \\ & S_{\min} \leq S_t \leq S_{\max}, \quad t = 0, \dots, T, \\ & 0 \leq c_t \leq C_{\max} z_t, \quad t = 1, \dots, T, \\ & 0 \leq d_t \leq D_{\max}(1 - z_t), \quad t = 1, \dots, T, \\ & g_t \geq 0, \quad z_t \in \{0, 1\}, \quad t = 1, \dots, T, \\ & S_0 = S_T = 6000 \end{aligned} \quad (12)$$

模型中的目标函数和所有约束均为线性形式，其中 z_t 为整数变量，其余变量为连续变量，因此该模型属于混合整数线性规划模型^[7]。

5.1.4 基于 HiGHS 求解器的模型求解

我们将已知的电价、小区负载和光伏预测功率代入模型后，使用 SciPy 提供的 `optimize.milp` 接口调用 HiGHS 求解器直接进行求解。

求解器在约 0.033 秒内收敛，最优性相对间隙为 0，即模型在当前数值容差下达到最优。需要强调的是，求解器报告成功只表示数值算法在当前容差内找到了满足约束的最优解，并不能代替物理可行性证明。因此，本文在得到求解器返回的 g, c, d, S, z 后，不直接以求解器状态作为最终判据，而是重新按照物理关系计算残差。

首先计算逐时供需平衡的最大残差：

$$R_{\text{balance}} = \max_t |g_t + v_t + d_t - \ell_t - c_t| \quad (13)$$

其次计算储能状态递推的最大残差：

$$R_{\text{storage}} = \max_t \left| \Delta S_t - \eta_c c_t + \frac{d_t}{\eta_d} \right|, \quad \Delta S_t = S_t - S_{t-1} \quad (14)$$

此外，我们还需要独立检查储电量上下限、日初日末储电量、购电量和充放电电量非负、充放电功率上限、状态变量整数性、同时充放电电量以及全天储能循环关系。设允许容差

$$\varepsilon = 10^{-5} \text{ kWh} \quad (15)$$

只有当上述全部残差均不超过 ε 时，才判定该解通过物理可行性校验；若任一残差超过容差，则拒绝该解，而非仅根据 HiGHS 状态接受求解结果。

为了进一步检验全天电能总体守恒情况，定义全天购电量、负载电量、光伏电量和充电量：

$$G = \sum_{t=1}^T g_t, \quad E_L = \sum_{t=1}^T \ell_t, \quad E_{PV} = \sum_{t=1}^T v_t, \quad C = \sum_{t=1}^T c_t \quad (16)$$

由于日初和日末储电量相同，全天充放电满足

$$\sum_{t=1}^T d_t = \eta_c \eta_d \sum_{t=1}^T c_t = \eta_c \eta_d C \quad (17)$$

因此全天能量守恒残差为

$$R_{\text{day}} = |G - E_L + E_{PV} - (1 - \eta_c \eta_d)C| \quad (18)$$

当前最优解的主要校验结果如表 2 所示，

表 2: 问题一最优解的独立物理可行性校验

校验项	残差 (kWh)
逐时供需平衡最大残差	1.4921×10^{-13}
储能状态递推最大残差	1.1937×10^{-12}
储电量边界与日初日末残差	0
充放电功率上限最大残差	1.2953×10^{-12}
状态变量整数性残差	2.6645×10^{-15}
同时充放电电量	0
全天能量守恒残差	5.4570×10^{-12}

全部残差均远小于 10^{-5} kWh，其中最大的全天能量守恒残差约为 5.46×10^{-12} kWh，说明 HiGHS 返回的解不仅满足求解器状态要求，也通过了独立的物理可行性校验。

5.1.5 最优购电策略与结果分析

经过对混合整数线性规划模型的求解，我们得出全天购电量为 59482.698998 kWh，全天购电费为 35126.948589 元。日初和日末储电量均为 6000 kWh，全天充电量为 20740.666132 kWh，放电量 为 16799.939567 kWh，储能损耗为 3940.726565 kWh。

表 3: 微网在指定时间段的购电量及全天的购电量和购电费

时间段	购电量	时间段	购电量	时间段	购电量
10:00–10:10	0.00	12:00–12:10	480.41	14:00–14:10	0.00
16:00–16:10	445.43	18:00–18:10	531.89	20:00–20:10	0.00
全天购电量		59482.70	全天购电费		35126.94

表 4: 储能设备在指定时间段的充放电量及 0:00 和 24:00 的储电量

时间段	充电量	放电量	时间段	充电量	放电量
0:00–4:00	4500.00	0.00	4:00–8:00	833.33	6365.84
8:00–12:00	4787.96	1703.00	12:00–16:00	5286.04	91.10
16:00–20:00	0.00	5780.13	20:00–24:00	5333.33	2859.87
0:00 储电量		6000.00	24:00 储电量		6000.00

部分 10 分钟时段的购电量如表 3所示, 储能设备六个四小时区间的充放电量结果如表 4所示。

从调度结果可以看出, 储能设备在夜间低电价及光伏富余时段主要以充电为主, 在电价较高或负载较大的时段释放电能, 从而减少高价时段的外网购电量。

全天满足约束

$$\sum_{t=1}^T d_t = \eta_c \eta_d \sum_{t=1}^T c_t = 0.81 \sum_{t=1}^T c_t, \quad (19)$$

同时, 全天购电量满足

$$\sum_{t=1}^T g_t = \sum_{t=1}^T \ell_t - \sum_{t=1}^T v_t + (1 - \eta_c \eta_d) \sum_{t=1}^T c_t \quad (20)$$

即全天购电量由小区负载电量与光伏电量的差值, 以及储能损耗共同组成。以上结果验证了模型能够在满足全部物理约束的前提下给出经济性较好的储能调度方案。图 2给出了最优调度方案下小区负载的用电来源构成。从图中可以看出, 外网购电承担了夜间和晚间的基础负荷, 光伏发电覆盖了白天的主要负荷, 储能设备主要在早晚负荷高峰或光伏不足的时段放电, 三者叠加后与用户负载曲线保持一致。该图从能量来源角度直观说明了储能低充高放策略对降低购电成本的贡献。

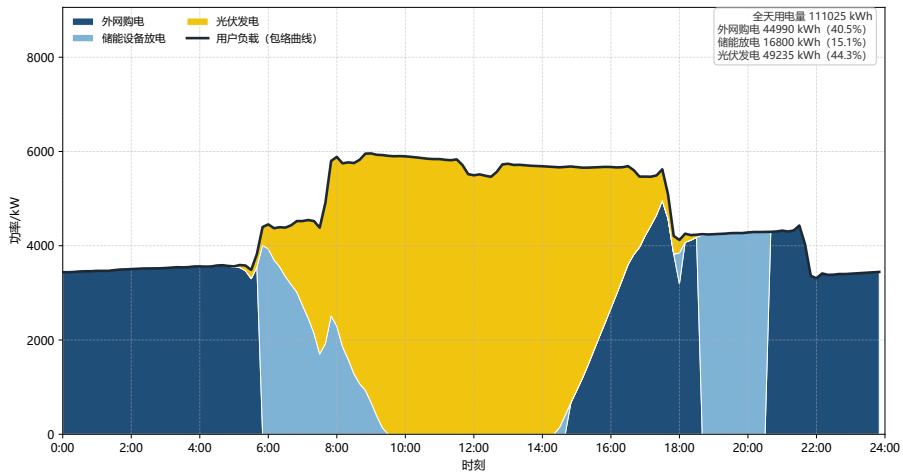


图 2: 小区负载用电来源功率堆叠图

5.2 问题二：滚动预测与风险修正下的购电和储能优化模型

5.2.1 建模思路与总体框架

问题二每天的电价规律相同，但负载和光伏实际功率随时间变化。0:00 制定计划时只能利用此前已经掌握的历史信息，因此可以将预测公式分解为滚动预测和风险修正两个部分。设第 k 天包含 $T = 144$ 个时段，负载和光伏预测功率分别为 $\hat{L}_{k,t}$ 和 $\widehat{PV}_{k,t}$ ，对应的预测净负荷电量为

$$\hat{N}_{k,t} = (\hat{L}_{k,t} - \widehat{PV}_{k,t}) \Delta t \quad (21)$$

为避免仅按预测均值安排购电造成高额紧急购电费用，本文利用历史预测误差的 80% 分位数对预测净负荷进行修正，并作为当日储能调度 MILP 的输入^[7]。当天运行阶段再根据实际负载和光伏执行储能动作，并对电池放电后仍存在的缺口进行紧急购电。

5.2.2 滚动预测设置

所有预测均采用逐日滚动方式：预测第 k 天时只使用第 k 天以前的数据；当天结束后，再将该天实际值加入历史序列并预测下一天。

2025 年 1 月作为模型预运行期，用于模型参数选择、历史预测残差估计以及储能设备状态初始化，使储能系统在进入正式计算期前达到合理的运行状态。因此根据问题 1 得出的购电策略购电，算出 2 月 1 日 0:00 时刻储能设备的初始电量。正式预测和费用统计区间为 2025 年 2 月 1 日至 12 月 31 日。模型输出逐日负载和光伏预测值，并整理为 `pred.xlsx`，作为第二问购电与储能调度的输入。

5.2.3 负载与光伏时间序列预测模型

住宅小区负载具有明显的日内周期和周周期。以天为时间尺度时，在工作日时 8:00-18:00 用电量显著提高，而非工作日此时段增长幅度较小；对于光伏发电，大致的发电区间在 6:00-18:00，根据年际太阳升起降落的时间前后摆动，且总体上呈现出先上升后下降的抛物线模型。以年为时间尺度时，以 12:00 为例，全年的小区负载周末为波谷，工作日为波峰，一个月大致有四个波峰，呈现出比较明显的波动特征；对于光伏发电则在 3-5 月和 8-11 月处在高位，而其余时段发电量较少，周内没有呈现明显的规律特征。

对目标日第 t 个时段，负载模型使用前 1、2、3、7 日的滞后值、前 3 日和 7 日的同时刻均值、最近两个相同星期几的均值，以及星期周期、年内周期和日内周期特征。设目标时段的特征向量为 $X_{k,t}^L$ ，则负载采用最近 31 天窗口的岭回归预测：

$$\hat{L}_{k,t} = X_{k,t}^L \beta_L, \quad (22)$$

$$\hat{\beta}_L = \arg \min_{\beta} \|Y_L - X^L \beta\|_2^2 + \lambda_L \|\beta\|_2^2 \quad (23)$$

其中，岭参数 λ_L 在 $\{0.1, 1, 10, 100\}$ 中利用 1 月数据选择，以平均绝对误差最小为目标，当前数据下取 $\lambda_L = 100$ ^[7]。

光伏发电除受近期天气影响外，还具有明显的太阳高度角季节规律^[7]。定义第 k 天的太阳赤纬为

$$\delta_k = 23.44^\circ \sin \left[\frac{2\pi(D_k - 81)}{365} \right] \quad (24)$$

其中 D_k 为年内序号。光伏特征包括前 1、2、3、7、14 日的滞后值，前 3 日、7 日和 14 日的同时刻均值，以及 $\sin(\delta_k)$ 、 $\cos(\delta_k)$ 和日内周期项。采用扩展窗口岭回归得到 $\widehat{PV}_{k,t}^{ridge}$ ，并与最近 7 日同时刻均值

$$\widehat{PV}_{k,t}^7 = \frac{1}{7} \sum_{j=1}^7 PV_{k-j,t} \quad (25)$$

进行融合：

$$\widehat{PV}_{k,t} = \max \left\{ 0, \alpha \widehat{PV}_{k,t}^7 + (1 - \alpha) \widehat{PV}_{k,t}^{ridge} \right\} \quad (26)$$

利用 1 月白天时段数据选择参数，当前取岭参数 1、融合权重 $\alpha = 0.4$ 。将滚动预测结果与附件 2 实际值进行比较，负载和光伏的 R^2 分别为 0.9745 和 0.9899，说明模型能够较好刻画全年负载和光伏的变化趋势。

5.2.4 预测误差与 80% 分位数风险修正

实际净负荷电量为

$$N_{k,t} = (L_{k,t} - PV_{k,t}) \Delta t \quad (27)$$

预测误差为

$$\varepsilon_{k,t} = N_{k,t} - \widehat{N}_{k,t} \quad (28)$$

当 $\varepsilon_{k,t} > 0$ 时，表示实际净负荷高于预测，容易产生紧急购电；当 $\varepsilon_{k,t} < 0$ 时，表示可能出现计划购电富余。由于紧急购电价格为正常电价的 5 倍，本文使用最近 60 天同一时段的历史预测误差，取 80% 分位数作为风险裕量^[?]：

$$q_{k,t} = Q_{0.8} \{ \varepsilon_{k-1,t}, \varepsilon_{k-2,t}, \dots, \varepsilon_{k-60,t} \} \quad (29)$$

最终用于制定计划的净负荷为

$$N_{k,t}^{\text{plan}} = \widehat{N}_{k,t} + q_{k,t} \quad (30)$$

该修正会在预测误差偏正时提高计划购电量，在误差偏负时适当降低备购量，实现实时信息反馈调节。从而在计划购电成本和五倍紧急购电风险之间取得平衡。

5.2.5 实际运行与跨日滚动求解

计划购电曲线在 0:00 锁定后，当天按实际负载和光伏逐时段运行。定义电池动作前的供需差额为

$$a_{k,t} = g_{k,t} + PV_{k,t} \Delta t - L_{k,t} \Delta t \quad (31)$$

当 $a_{k,t} \geq 0$ 时，计划电量和光伏已经满足负载，此时不进行紧急购电，富余电量优先充电：

$$c_{k,t} = \min \left\{ a_{k,t}, C_{\max}, \frac{S_{\max} - S_{k,t-1}^{\text{act}}}{\eta_c} \right\} \quad (32)$$

未利用富余电量为 $u_{k,t} = a_{k,t} - c_{k,t}$ 。当 $a_{k,t} < 0$ 时出现供电缺口，电池优先放电：

$$d_{k,t} = \min \{-a_{k,t}, D_{\max}, \eta_d (S_{k,t-1}^{\text{act}} - S_{\min})\} \quad (33)$$

若放电后仍有缺口，则进行紧急购电：

$$e_{k,t} = \max \{0, -a_{k,t} - d_{k,t}\} \quad (34)$$

实际储电量按

$$S_{k,t}^{\text{act}} = S_{k,t-1}^{\text{act}} + \eta_c c_{k,t} - \frac{d_{k,t}}{\eta_d} \quad (35)$$

更新，并满足跨日连续约束

$$S_{k+1,0}^{\text{act}} = S_{k,T}^{\text{act}} \quad (36)$$

2025 年 1 月 1 日初始储电量为 6000 kWh，经过 1 月热启动后，2 月 1 日初始储电量达到 10800 kWh。

5.2.6 模型求解结果与分析

使用前述滚动预测、80% 分位数风险修正和逐日 MILP 模型，对 2025 年 2 月 1 日至 12 月 31 日进行求解。预测精度和全年费用汇总分别如表 5 和表 6 所示。

表 5: 第二问负载与光伏预测精度

预测对象	R^2	MAE (kW)	RMSE (kW)
小区负载	0.9745	166.4431	224.6384
光伏发电	0.9899	154.7734	301.7602
光伏白天时段	0.9735	300.4588	426.2253

表 6: 第二问 2025 年全年购电与储能结果

指标	结果
计划购电量	22046434.834858 kWh
计划购电费	13454464.390255 元
实际充电量	6734511.639971 kWh
实际放电量	5455069.040090 kWh
紧急购电量	126347.963398 kWh
紧急购电费	769998.124403 元
总费用	14224462.514658 元
2025-12-31 期末储电量	10672.653652 kWh

图 3 进一步给出了每天 12:00 和 14:00 的小区负载实际值、回测预测值以及未来 5 天预测值。两个时刻的预测曲线均能跟随全年负载的周期变化，说明滚动岭回归模型不仅具有较高的整体拟合精度，而且能够在不同季节保持稳定的预测能力。未来 5 天预测结果则可直接作为后续购电计划和储能调度的输入。

表 7: 微网在 2025.3.20 的计划购电量及全天的购电量和购电费

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.00	12:00-12:10	585.32	14:00-14:10	0.00
16:00-16:10	497.65	18:00-18:10	709.37	20:00-20:10	0.00
全天购电量		68329.25	全天购电费		41405.45

表 8: 储能设备在 2025.3.20 的充放电量及 0:00 和 24:00 的储电量

时间段	充电量	放电量	时间段	充电量	放电量
0:00-4:00	150.12	246.64	4:00-8:00	194.59	5783.89
8:00-12:00	6223.29	2777.38	12:00-16:00	4620.44	407.41
16:00-20:00	318.18	5657.28	20:00-24:00	10490.82	2945.32
0:00 储电量		10800.00	24:00 储电量		10800.00

表 9: 微网在 2025.6.21 的计划购电量及全天的购电量和购电费

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.00	12:00-12:10	0.00	14:00-14:10	0.00
16:00-16:10	266.64	18:00-18:10	151.04	20:00-20:10	0.00
全天购电量		40884.83	全天购电费		24026.86

表 10: 储能设备在 2025.6.21 的充放电量及 0:00 和 24:00 的储电量

时间段	充电量	放电量	时间段	充电量	放电量
0:00-4:00	143.96	2985.58	4:00-8:00	421.11	4836.61
8:00-12:00	9091.96	0.00	12:00-16:00	0.00	0.00
16:00-20:00	310.26	5006.68	20:00-24:00	8939.82	2485.90
0:00 储电量		10800.00	24:00 储电量		10800.00

表 11: 微网在 2025.9.23 的计划购电量及全天的购电量和购电费

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.00	12:00-12:10	508.11	14:00-14:10	0.00
16:00-16:10	533.34	18:00-18:10	781.14	20:00-20:10	0.00
全天购电量		68928.28	全天购电费		42441.21

表 12: 储能设备在 2025.9.23 的充放电量及 0:00 和 24:00 的储电量

时间段	充电量	放电量	时间段	充电量	放电量
0:00-4:00	1833.61	36.61	4:00-8:00	484.02	5968.81
8:00-12:00	5529.14	1896.62	12:00-16:00	4257.75	616.12
16:00-20:00	88.74	5921.12	20:00-24:00	8709.29	2392.36
0:00 储电量		8927.88	24:00 储电量		9038.36

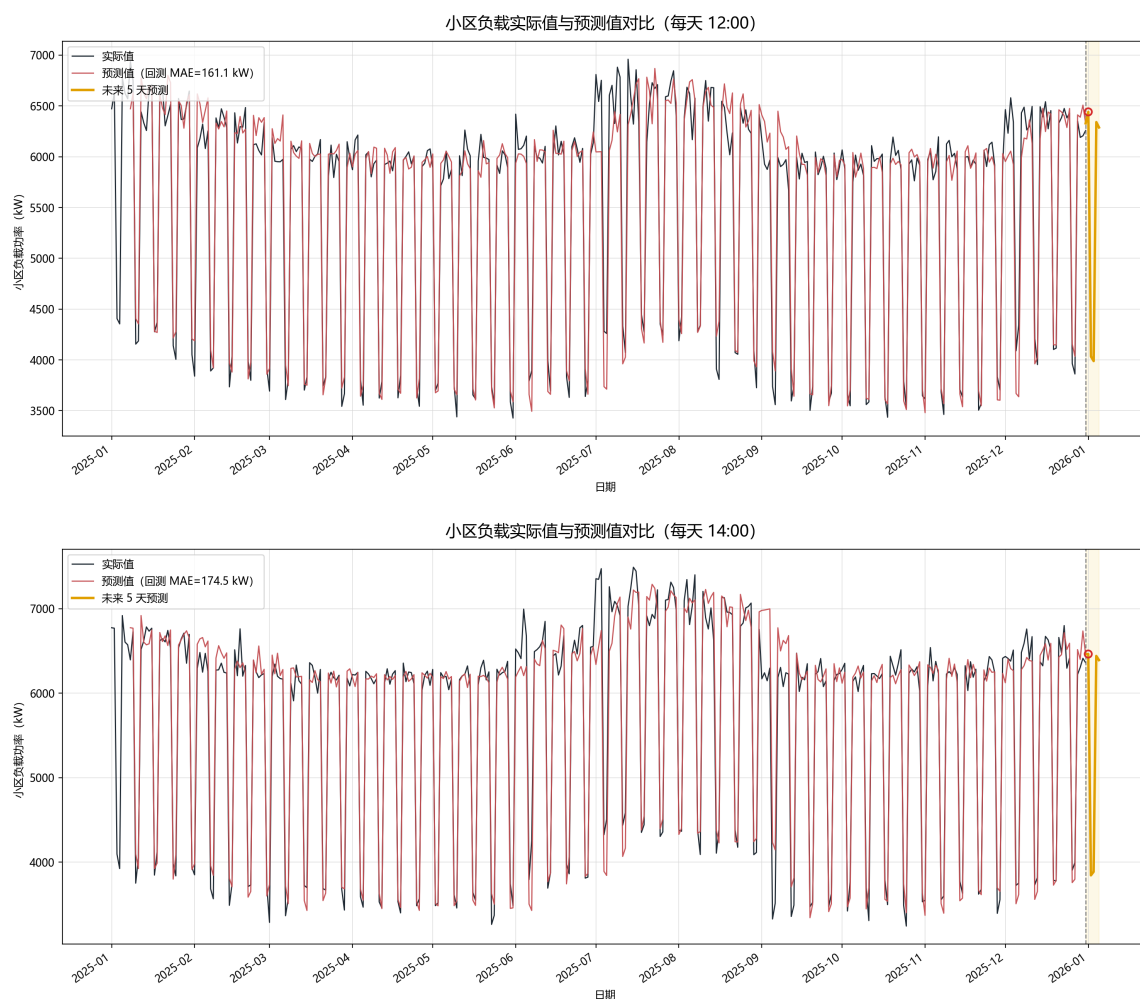


图 3: 小区负载实际值与预测值对比

表 13: 微网在 2025.12.21 的计划购电量及全天的购电量和购电费

时间段	购电量	时间段	购电量	时间段	购电量
10:00–10:10	0.00	12:00–12:10	1024.62	14:00–14:10	0.00
16:00–16:10	852.11	18:00–18:10	739.24	20:00–20:10	0.00
全天购电量		101568.08	全天购电费		65539.72

表 14: 储能设备在 2025.12.21 的充放电量及 0:00 和 24:00 的储电量

时间段	充电量	放电量	时间段	充电量	放电量
0:00–4:00	40.17	53.70	4:00–8:00	801.40	1407.25
8:00–12:00	6080.70	6599.97	12:00–16:00	5769.72	2219.59
16:00–20:00	0.00	5538.24	20:00–24:00	10346.63	2842.53
0:00 储电量		10800.00	24:00 储电量		10800.00

结果显示，负载和光伏模型的决定系数均超过 0.97，预测值能够较好地跟踪实际变化。80% 分位数风险裕量使计划购电量略高于预测均值，全年紧急购电量仅为 126347.963398 kWh，只占计划购电量的约 0.57%，有效抑制了五倍电价下的缺电成本，最终总购电费用为 14224462.514658 元。

四个代表性日期的部分时段计划购电量分别如表 7、9、11和13所示；对应的储能充放电和起止储电量分别如表8、10、12和14所示。完整的逐日计划购电量、四小时充放电量和紧急购电时段已保

表 15: 微网在指定日期的紧急购电量

2025.3.20		2025.6.21		2025.9.23		2025.12.21	
时间段	购电量	时间段	购电量	时间段	购电量	时间段	购电量
-	-	-	-	20:20–21:50	942.28	-	-

存至 `result2.xlsx`。

四个指定日期的紧急购电情况如表 15 所示。其中 3 月 20 日、6 月 21 日和 12 月 21 日均未发生紧急购电，因此相应位置填写“-”；9 月 23 日发生一次紧急购电，时段为 20:20–21:50，购电量为 942.28 kWh。

5.3 问题三：光伏预报驱动的多阶段购电和储能优化模型

5.3.1 多阶段决策与信息结构

在问题二的基础上，问题三引入了预测光伏发电功率的因素^[7]，并在一天中设置四个预报发布和决策时刻：

$$\mathcal{E} = \{0, 6, 12, 18\} \quad (37)$$

在 0:00 根据预测制定全天购电计划，在 6:00、12:00 和 18:00 结合新的预报和已知数据进行调整，计算出调整后的购电计划。时刻 τ 的信息集记为 $\mathcal{F}_{d,\tau}$ ，包含历史数据、当天已经观测到的实际值、当前时刻发布的光伏发电量预报，以及当前储电量和电价。所以问题三是在已有计划的基础上，加入多阶段滚动调整的过程。

从决策结构来看，总购电费由计划购电费，调整购电费和紧急购电费三部分组成。因此问题三可分解为三个层次：第一是制定计划购电策略，第二是根据预测数据进行滚动调整，第三是结算实际运行产生的费用。

在时刻 τ 发布的未来第 h 小时光伏预测功率为 $\widehat{PV}_{d,\tau,h}$ ，其对应的 10 分钟时段编号为

$$s_h = 6(\tau + h - 1), \quad h = 1, 2, \dots, 24 \quad (38)$$

对于位于相邻两个整点节点之间的时段 t ，我们采用线性插值的手段获得光伏预测功率：

$$\widehat{PV}_{d,\tau,t} = \widehat{PV}_{d,\tau,h} + \frac{t - s_h}{6} \left[\widehat{PV}_{d,\tau,h+1} - \widehat{PV}_{d,\tau,h} \right] \quad (39)$$

曲线两端水平外推， $t \leq s_1$ 和 $t \geq s_{24}$ 时分别取首尾整点预报值。由此得到每个决策时刻可直接用于优化模型的光伏功率曲线。

5.3.2 计划购电策略的制定

在 0:00，微网根据当天的负载预测和光伏预报，制定全天 144 个时间段的计划购电策略。计划购电模型以全天计划购电费用最小为目标：

$$\min \sum_{t=1}^T p_t g_{d,t}^{\text{plan}} \quad (40)$$

约束条件大致可分为以下五类：

第一，每个时段内充放电功率须保持平衡，购电量与放电量之和减去充电量与缺电量，应等于预测净负荷。

$$g_{d,t}^{\text{plan}} + d_{d,t}^{\text{plan}} - c_{d,t}^{\text{plan}} - u_{d,t}^{\text{plan}} = \widehat{N}_{d,t|0} \quad (41)$$

第二，储电设备需要满足储电量递推关系：

$$S_{d,t}^{\text{plan}} = S_{d,t-1}^{\text{plan}} + 0.9c_{d,t}^{\text{plan}} - \frac{d_{d,t}^{\text{plan}}}{0.9} \quad (42)$$

第三，储电设备的储电量必须在上下限之间：

$$1200 \leq S_{d,t}^{\text{plan}} \leq 10800 \quad (43)$$

第四，储电设备的充放电功率必须在最大充放电功率范围内：

$$\begin{aligned} 0 \leq c_{d,t}^{\text{plan}} &\leq \frac{2500}{3} z_{d,t}, \\ 0 \leq d_{d,t}^{\text{plan}} &\leq \frac{2500}{3} (1 - z_{d,t}) \end{aligned} \quad (44)$$

第五，所有决策变量需要满足非负约束：

$$g_{d,t}^{\text{plan}} \geq 0, \quad u_{d,t}^{\text{plan}} \geq 0, \quad z_{d,t} \in \{0, 1\} \quad (45)$$

0:00 发布的光伏预报，经插值后记为 $\widehat{PV}_{d,0,t}$ ，此时的预测净负荷电量为

$$\widehat{N}_{d,t|0} = \left(\widehat{L}_{d,t|0} - \widehat{PV}_{d,0,t} \right) \Delta t \quad (46)$$

该模型是一个混合整数线性规划模型，可以用标准求解器进行求解，从而得到当天计划购电量 $g_{d,t}^{\text{plan}}$ ，充放电电量以及储能轨迹。

5.3.3 滚动调整模型

对于任意调整时刻 $\tau \in \{6, 12, 18\}$ ，微网获得新的光伏预报，同时也已经观察到当天部分时段的实际数据。

此时，微网对剩余时段的购电-储能策略进行调整。更新后的预测净负荷为：

$$\widehat{N}_{d,t|\tau} = \left(\widehat{L}_{d,t|\tau} - \widehat{PV}_{d,\tau,t} \right) \Delta t, \quad t \in \mathcal{T}_\tau \quad (47)$$

剩余时段的策略仍需满足储能设备的约束条件：

$$g_{d,t|\tau}^{\text{adj}} + d_{d,t|\tau} - c_{d,t|\tau} - u_{d,t|\tau} = \widehat{N}_{d,t|\tau}, \quad (48)$$

$$S_{d,t|\tau} = S_{d,t-1|\tau} + 0.9c_{d,t|\tau} - \frac{d_{d,t|\tau}}{0.9}, \quad (49)$$

$$g_{d,t|\tau}^{\text{adj}} = g_{d,t|\tau-}^{\text{adj}}, \quad t \leq k_\tau \quad (50)$$

每个时段最终调整量相对原计划量的差额为

$$\Delta_{d,t} = g_{d,t}^{\text{adj}} - g_{d,t}^{\text{plan}} \quad (51)$$

滚动调整模型的目标是最小化调整带来的额外费用，我们将调整量分解为调高量和调低量，定义为正偏差和负偏差：

$$\Delta_{d,t}^+ = \max \{g_{d,t}^{\text{adj}} - g_{d,t}^{\text{plan}}, 0\}, \quad (52)$$

$$\Delta_{d,t}^- = \max \{g_{d,t}^{\text{plan}} - g_{d,t}^{\text{adj}}, 0\} \quad (53)$$

根据题目，正偏差电价为交易时刻电价的 1.5 倍；负偏差电价，即违约电价为交易时刻电价的 50%。因此，调整模型的目标函数为：

$$\min \sum_{t=t_n}^T (1.5p_t \Delta_{d,t}^+ + 0.5p_t \Delta_{d,t}^-) \quad (54)$$

调整模型的约束与计划模型类似，但作用于剩余时段。储能递推、充放电互斥、储电量范围约束均保持一致。为保证储能运行的连续性，调整模型要求剩余时段结束时储电量不低于目标值：

$$S_T + s_{\text{short}} \geq S_{\text{target}} \quad (55)$$

调整模型同样是一个 MILP，求解后得到调整后购电量，并用其覆盖计划购电量中对应剩余时段的部分。

5.3.4 实际运行与紧急购电

制定计划后，微网按实际小区负载和实际光伏发电功率运行。若计划购电量、光伏发电功率和储能设备放电量之和仍不足以满足负载，则产生紧急购电。定义电池做出实际应答前的供需差额为

$$a_{d,t} = g_{d,t}^{\text{adj}} + PV_{d,t} \Delta t - L_{d,t} \Delta t \quad (56)$$

当 $a_{d,t} \geq 0$ 时，不需要放电和紧急购电，富余电量优先充电：

$$c_{d,t} = \min \left\{ a_{d,t}, \frac{2500}{3}, \frac{10800 - S_{d,t-1}^{\text{act}}}{0.9} \right\}, \quad (57)$$

未利用富余电量为 $u_{d,t} = a_{d,t} - c_{d,t}$ 。当 $a_{d,t} < 0$ 时，电池优先放电：

$$d_{d,t} = \min \left\{ -a_{d,t}, \frac{2500}{3}, 0.9 (S_{d,t-1}^{\text{act}} - 1200) \right\}, \quad (58)$$

若电池放电后仍有缺口，则紧急购电量为

$$e_{d,t} = \max \{0, -a_{d,t} - d_{d,t}\} \quad (59)$$

实际储电量按照

$$S_{d,t}^{\text{act}} = S_{d,t-1}^{\text{act}} + 0.9c_{d,t} - \frac{d_{d,t}}{0.9} \quad (60)$$

更新，并满足跨日连续约束

$$S_{d+1,0}^{\text{act}} = S_{d,T}^{\text{act}} \quad (61)$$

计划购电费用、调整费用和紧急购电费用分别为

$$C_d^{\text{plan}} = \sum_{t=1}^T p_t g_{d,t}^{\text{plan}}, \quad (62)$$

$$C_d^{\text{adj}} = \sum_{t=1}^T [1.5p_t \Delta_{d,t}^+ + 0.5p_t \Delta_{d,t}^-], \quad (63)$$

$$C_d^{\text{em}} = \sum_{t=1}^T 5p_t e_{d,t} \quad (64)$$

因此，当天总费用为

$$C_d = C_d^{\text{plan}} + C_d^{\text{adj}} + C_d^{\text{em}} \quad (65)$$

每天 0:00，以计划购电费用、调整购电费用和紧急购电费用的总和最小为目标；在 6:00、12:00 和 18:00 重新求解剩余时段的调整问题。每天按“0:00 计划—6:00 调整—12:00 调整—18:00 调整—实际运行”的顺序滚动执行，并将当天末储电量传递给下一天。

5.3.5 模型求解结果

问题三建立了多阶段策略调整-储能优化的模型，计划模型和调整模型均为混合整数线性规划，目标函数分别对应计划购电费用和调整偏差费用；实际运行阶段按实际负载和实际光伏进行仿真模拟。在进行计划制定时，本文在模型中加入了风险裕度的参数化调整，以提高模型的鲁棒性。具体的灵敏度分析详见下文。求解采用 1 月暖启动和逐日滚动优化的框架，每天依次求解一次计划 MILP 和三次调整 MILP，最终输出完整的购电策略和费用结果。全年费用和购电量结果如表16所示。

表 16: 第三问 2025 年全年购电与储能结果

指标	结果
计划购电量	20506836.540063 kWh
调整后购电量	21165575.483359 kWh
计划购电费	12502869.719778 元
调整费用	785257.512357 元
紧急购电量	47836.980817 kWh
紧急购电费	269648.92 元
总费用	13557776.15 元
2025-12-31 期末储电量	5788.255214 kWh

在本题条件下，调整后购电量高于原计划量，计划费用和调整费用之和略有增加，但高价紧急购电明显减少，总费用约为 13557776.15 元，比问题二降低约 666686.36 元。完整的计划购电量、调整购电量、充放电量及紧急购电结果已保存至 `result3.xlsx`。

四个代表性日期的计划购电量分别如表 17、19、21和23所示；对应的储能充放电和起止储电量分别如表18、20、22和24所示。

表 17: 微网在 2025.3.20 的购电量及全天的购电量和购电费

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.00	12:00-12:10	632.20	14:00-14:10	0.00
16:00-16:10	536.78	18:00-18:10	701.53	20:00-20:10	0.00
全天购电量		65773.29	全天购电费		40023.13

表 18: 储能设备在 2025.3.20 的充放电量及 0:00 和 24:00 的储电量

时间段	充电量	放电量	时间段	充电量	放电量
0:00-4:00	3782.82	408.45	4:00-8:00	908.09	5986.96
8:00-12:00	5327.30	2321.10	12:00-16:00	5653.84	254.72
16:00-20:00	35.24	5563.31	20:00-24:00	5683.62	2966.25
0:00 储电量		6663.20	24:00 储电量		6469.68

表 19: 微网在 2025.6.21 的购电量及全天的购电量和购电费

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.00	12:00-12:10	0.00	14:00-14:10	0.00
16:00-16:10	168.68	18:00-18:10	0.00	20:00-20:10	0.00
全天购电量		34813.14	全天购电费		20297.34

表 20: 储能设备在 2025.6.21 的充放电量及 0:00 和 24:00 的储电量

时间段	充电量	放电量	时间段	充电量	放电量
0:00-4:00	482.29	1596.33	4:00-8:00	440.99	4395.37
8:00-12:00	9942.63	0.00	12:00-16:00	0.00	0.00
16:00-20:00	122.15	5398.47	20:00-24:00	6175.24	2485.90
0:00 储电量		7678.13	24:00 储电量		7707.23

表 21: 微网在 2025.9.23 的购电量及全天的购电量和购电费

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.00	12:00-12:10	320.52	14:00-14:10	0.00
16:00-16:10	487.41	18:00-18:10	681.49	20:00-20:10	0.00
全天购电量		61120.19	全天购电费		37820.38

表 22: 储能设备在 2025.9.23 的充放电量及 0:00 和 24:00 的储电量

时间段	充电量	放电量	时间段	充电量	放电量
0:00-4:00	4595.70	170.88	4:00-8:00	988.30	6828.24
8:00-12:00	3362.02	1855.98	12:00-16:00	6933.26	586.44
16:00-20:00	501.26	5882.08	20:00-24:00	5595.67	2273.29
0:00 储电量		6009.63	24:00 储电量		6236.10

表 23: 微网在 2025.12.21 的购电量及全天的购电量和购电费

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.00	12:00-12:10	961.03	14:00-14:10	0.00
16:00-16:10	1017.09	18:00-18:10	728.08	20:00-20:10	0.00
全天购电量		99465.04	全天购电费		64105.46

表 24: 储能设备在 2025.12.21 的充放电电量及 0:00 和 24:00 的储电量

时间段	充电量	放电量	时间段	充电量	放电量
0:00-4:00	4707.13	97.36	4:00-8:00	1215.44	1491.02
8:00-12:00	5996.65	6833.04	12:00-16:00	6200.71	2220.11
16:00-20:00	17.91	5733.81	20:00-24:00	5602.70	2843.29
0:00 储电量		6315.99	24:00 储电量		6328.43

表 25: 微网在指定日期的紧急购电量

2025.3.20		2025.6.21		2025.9.23		2025.12.21	
时间段	购电量	时间段	购电量	时间段	购电量	时间段	购电量
9:00-10:00	456.27	-	-	9:30-9:50	40.64	-	-
-	-	-	-	20:20-21:50	578.07	-	-

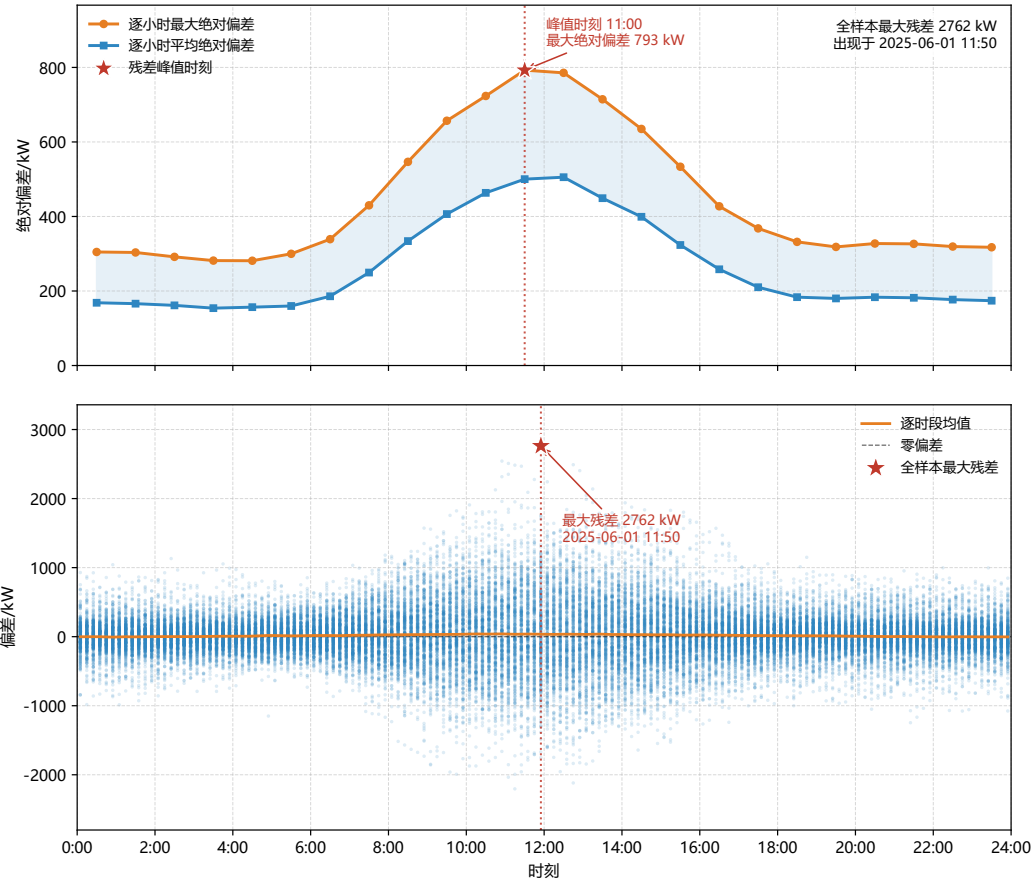


图 4: 全年计划购电量与调整购电量偏差曲线

四个指定日期的紧急购电情况如表 25 所示。其中 3 月 20 日发生一次紧急购电，时段为 9:00–10:00，购电量为 456.27 kWh；6 月 21 日和 12 月 21 日均未发生紧急购电，因此相应位置填写“-”；9 月 23 日发生两次紧急购电，分别为 9:30–9:50 的 40.64 kWh 和 20:20–21:50 的 578.07 kWh。

结合问题三的求解结果与风险裕度的敏感性分析，本文认为在当前数据条件与模型框架下，不需要引入附件 3 之外的额外预报时刻。现有四个决策时刻已经覆盖了光伏出力变化的主要阶段，根据题目给出的数据，在光伏发电功率变化剧烈的时间段，微网可以通过滚动调整策略提前做出应答，从而有效降低紧急购电量和总费用。与问题二相比，第三问通过三个时刻光伏预报更新，将全年紧急购电量由 126347.963398kWh 降低到 47836.980817 kWh，降幅高达 90.1%。图 4 展示了预测购电计划与调整购电计划之间的差值，这一部分对应于紧急购电量。这说明四个决策时刻已经足以捕捉光伏出力的主要波动，并显著降低高价紧急购电风险。

从费用构成角度看，当前策略的主要成本是计划购电费用，而调整费用和紧急购电费用占比较低。即使引入更多预报时刻能够进一步降低紧急购电费用，其可压缩的空间也十分有限。相反，更频繁的调整会带来更多的偏差，从而增加整体费用。

综上所述，问题三现有的 0:00、6:00、12:00、18:00 四个决策时刻已经能够较好地平衡预报更新收益与调整费用。问题三的求解结果裕度灵敏性分析均表明，额外引入其他时刻的预报对总费用的改善有限，反而可能增加调整费用和预报误差风险。因此，在当前数据条件与模型框架下，不需要引入附件 3 之外的额外预报时刻来制定调整购电策略。

5.4 问题四：动态电价预测与购电成本优化模型

5.4.1 Rolling 电价预测模型

附件 4 给出第 d 天第 t 个时段的实际电价 $p_{d,t}^{\text{act}}$ 。根据 144 个时间段可以计算当日电费的日均价，然后采用过去 7 天日均价的平均值预测得到下一天的价格水平^[7]。再根据附件 1 进行相同的计算，得到经验日均价。于是在附件 1 经验时段价格的基础上再加上近 7 天和经验日均价的差值得到预测电价，若预测电价低于 0 则将电价视为 0。具体公式如下：

$$y_d = \frac{1}{144} \sum_{t=1}^{144} p_{d,t}^{\text{act}} \quad (66)$$

$$\hat{y}_d = \frac{1}{7} \sum_{k=1}^7 y_{d-k} \quad (67)$$

$$\bar{p} = \frac{1}{144} \sum_{t=1}^{144} p_t \quad (68)$$

$$\hat{p}_{d,t} = \max [p_t + \hat{y}_d - \bar{p}, 0] \quad (69)$$

式中 p_t 为附件 1 的时段价格，该方法的逐时段 MAE 为 0.0939 元/kWh，RMSE 为 0.1207 元/kWh，逐时段 $R^2 = 0.8709$ 。这种方法综合考虑了附件 1 经验电价的数值和近期电价的波动因素，经 R^2 检验说明拟合程度较好，可以作为预测电价依据。图 5 以预测电价为横轴、实际电价为纵轴，展示了全天各时段电价预测值同实际值的对应关系，并用颜色表示样本密度。散点主要沿对角线分布，说明所采用的滚动电价预测模型能够较好地跟踪动态电价的日内变化。

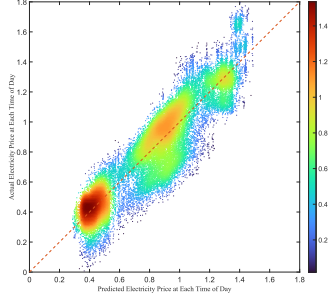


图 5: 价格实际值与预测值对比

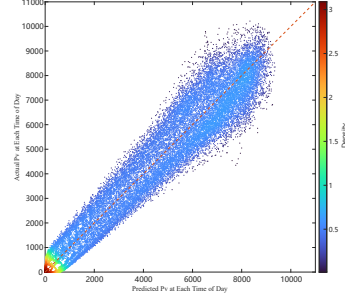


图 6: 光伏实际值与预测值对比

5.4.2 问题 4-2 的日前购电模型

问题 4-2 沿用问题 2 的负载和光伏预测以及 80% 分位数风险修正, 使用预测电价建立日计划购电与储能 MILP。计划购电费按预测电价计算, 紧急购电费按实际动态电价计算:

$$C_{4-2}^{\text{plan}} = \sum_{d \in \mathcal{D}} \sum_{t=1}^T \hat{p}_{d,t} g_{d,t}, \quad (70)$$

$$C_{4-2}^{\text{em}} = \sum_{d \in \mathcal{D}} \sum_{t=1}^T 5p_{d,t}^{\text{act}} e_{d,t} \quad (71)$$

式中 $e_{d,t}$ 为第 d 天第 t 时段的紧急购电量。

5.4.3 问题 4-3 的多阶段调整模型

问题 4-3 沿用问题 3 的计算模型。三类购电量为 $g_{d,t}^{\text{plan}}$ 、 $g_{d,t}^{\text{adj}}$ 和 $e_{d,t}$ 。相对参考计划的调高量和调低量分别为

$$\Delta_{d,t}^+ = \max(g_{d,t}^{\text{adj}} - g_{d,t}^{\text{ref}}, 0), \quad (72)$$

$$\Delta_{d,t}^- = \max(g_{d,t}^{\text{ref}} - g_{d,t}^{\text{adj}}, 0) \quad (73)$$

调整费用为

$$C_{4-3}^{\text{adj}} = \sum_{d \in \mathcal{D}} \sum_{t=1}^T [1.5\hat{p}_{d,t}\Delta_{d,t}^+ + 0.5\hat{p}_{d,t}\Delta_{d,t}^-] \quad (74)$$

实际运行使用最终调整购电量, 紧急购电量仍按实际负载、光伏和电池放电后的缺口计算:

$$e_{d,t} = \max[0, L_{d,t}^{\text{act}}\Delta t - V_{d,t}^{\text{act}}\Delta t - g_{d,t}^{\text{adj}} - d_{d,t}] \quad (75)$$

其中 $e_{d,t}$ 为第 d 天第 t 时段的紧急购电量, $L_{d,t}^{\text{act}}$ 和 $V_{d,t}^{\text{act}}$ 分别为实际用电负荷与实际光伏出力, $d_{d,t}$ 为电池放电量, Δt 为时段长度。因此, 问题 4-3 的总费用为

$$C_{4-3} = C_{4-3}^{\text{plan}} + C_{4-3}^{\text{adj}} + C_{4-3}^{\text{em}} \quad (76)$$

其中计划和调整费用采用预测电价, 紧急购电费用采用实际动态电价。储能状态在两个子问题中均满足 $S_{d+1,0} = S_{d,T}$ 。

5.4.4 模型求解结果与对比分析

动态电价下两个子问题的全年计算结果如表 26所示，剩余四个指定日期的指定时间段购电量、储电量以及紧急购电量我们放到附录部分予以展示。

表 26: 问题 4-2 和问题 4-3 全年计算结果

指标	问题 4-2	问题 4-3
计划购电量 (kWh)	22053793.376198	20504910.715391
调整后购电量 (kWh)	–	21163339.335032
计划购电费 (元)	13419241.315978	12488168.428686
调整费用 (元)	–	772236.364609
紧急购电量 (kWh)	126134.858370	47824.919727
紧急购电费 (元)	804664.104038	275737.660157
总费用 (元)	14223905.420015	13536142.453452
期末储电量 (kWh)	10672.653652	5788.255214

问题 4-3 通过 6:00、12:00 和 18:00 的预测信息更新，将紧急购电量由约 1.2613×10^5 kWh 降低到约 4.7825×10^4 kWh。虽然新增了调整费用，但高价紧急购电的减少带来了更大的成本节约：

$$C_{4-2} - C_{4-3} = 14223905.42 - 13536142.45 = 687762.97 \text{ 元} \quad (77)$$

因此，动态电价下问题 4-3 明显优于问题 4-2。

图 6 以光伏预测功率为横轴、实际功率为纵轴，对全年各时段的光伏预测结果进行一致性检验。密度较高的点主要集中在对角线附近，说明光伏预测结果整体无系统性高估或低估，可为多阶段购电调整提供可靠输入。

6 模型评估与灵敏度分析

本文接下来将从正确性，有效性，鲁棒性，可解释性和可推广性五个方面对问题一到问题四的模型进行系统性评估。在指标层面，我们分别采用约束类指标，经济类指标和运行类指标这三类指标评价。本文采用约束校验，多问题对比与灵敏度扫描的方法来展开。约束校验用于验证模型的正确性，多问题对比用于观察多个阶段调整和电价波动带来的影响，灵敏度扫描用于判断结论和系统的鲁棒性。下面按此顺序展开。

6.1 有效性和正确性评估

正确性是所有评估的前提。本文对问题一至问题四所有的逐时段 MILP 模型进行了四类校验，以验证模型满足所有的约束条件。

第一，功率平衡约束。对每个时段 t ，检查购电量、放电量、光伏电量之和与充电量、缺电量、负载电量之和是否相等。图 7 直观的展示了系统负载与电量供给功率的残差曲线，经过计算验证，四个问题的所有时段功率平衡偏差均在 10^{-6} kWh 量级以内，说明约束严格满足。

第二是储能递推约束。对每个时段检查

$$S_{d,t}^{\text{act}} - S_{d,t-1}^{\text{act}} - 0.9c_{d,t} + \frac{d_{d,t}}{0.9} = 0 \quad (78)$$

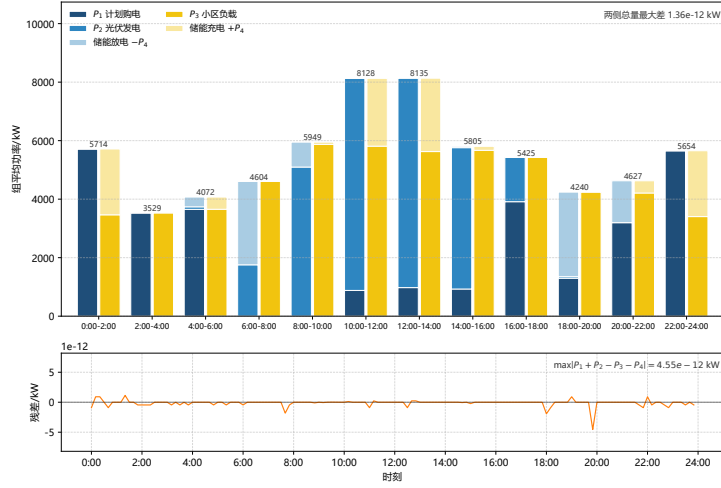


图 7: 系统负载与电量供给功率残差曲线

结果显示储能轨迹连续变化无跳跃。第三是储电量范围约束。四个问题中储能设备的储电量始终保持在 $[1200, 10800]$ kWh 之间, 未出现越限。问题一和问题二强制日初日末储电量相同, 问题三和问题四通过跨日传递实现储能连续, 均满足题目要求。第四, 大型储能设备通常都须满足充放电互斥约束, 通过引入二进制变量 z_t , 保证了互斥约束的有效性。

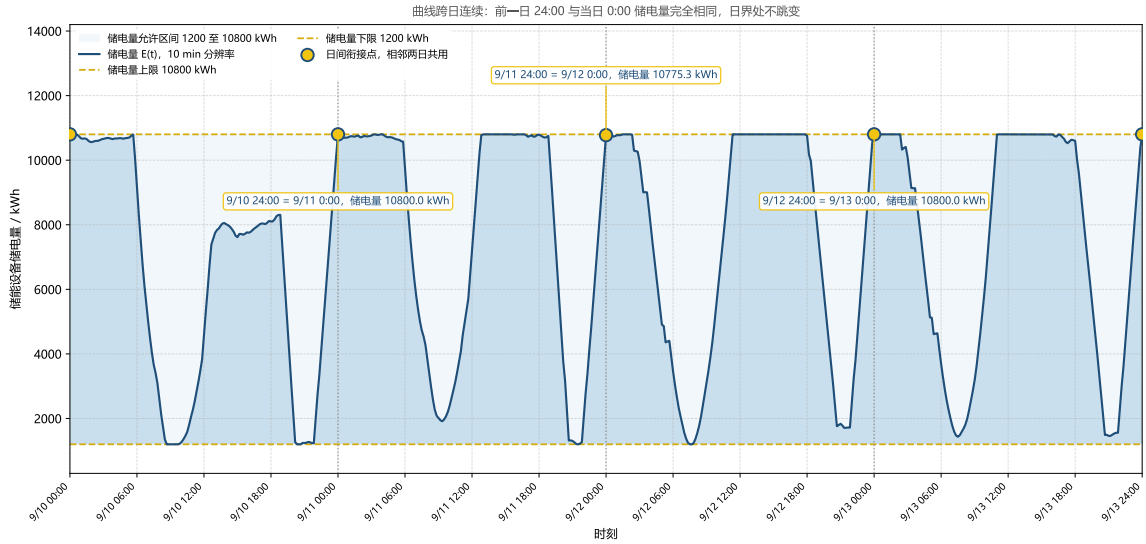


图 8: 储电量日连续性及跨日衔接节点

图 8选取连续四天展示 10 分钟分辨率的储电量轨迹, 并标出前一日 24:00 与次日 0:00 的衔接节点, 用于说明日界处不存在跳变; 图 9进一步给出全年储电量轨迹及全部日间衔接点的对角线校验。两图共同表明, 储电量始终处于允许区间内, 并且相邻两日的储能状态严格连续。

上述校验说明, 四个问题的 MILP 模型在数学上自治, 求解结果在物理上可行, 模型正确性等到了有效验证。

有效性关注模型是否真正达到了“保证供电、降低费用”的目标。下面我们分别从购电费用, 紧急购电量和弃电量三个角度来进行评估。

问题一在固定电价和固定负载下, 计划购电费即为总费用, 储能通过低充高放降低购电成本。问题二在固定电价下引入紧急购电, 总费用由计划购电费和紧急购电费组成。问题三引入多阶段滚动调整后, 总费用由计划购电费、调整费和紧急购电费三部分组成。问题四在波动电价下重算问题二和问题三, 总费用进一步上升, 主要来自电价本身的波动。

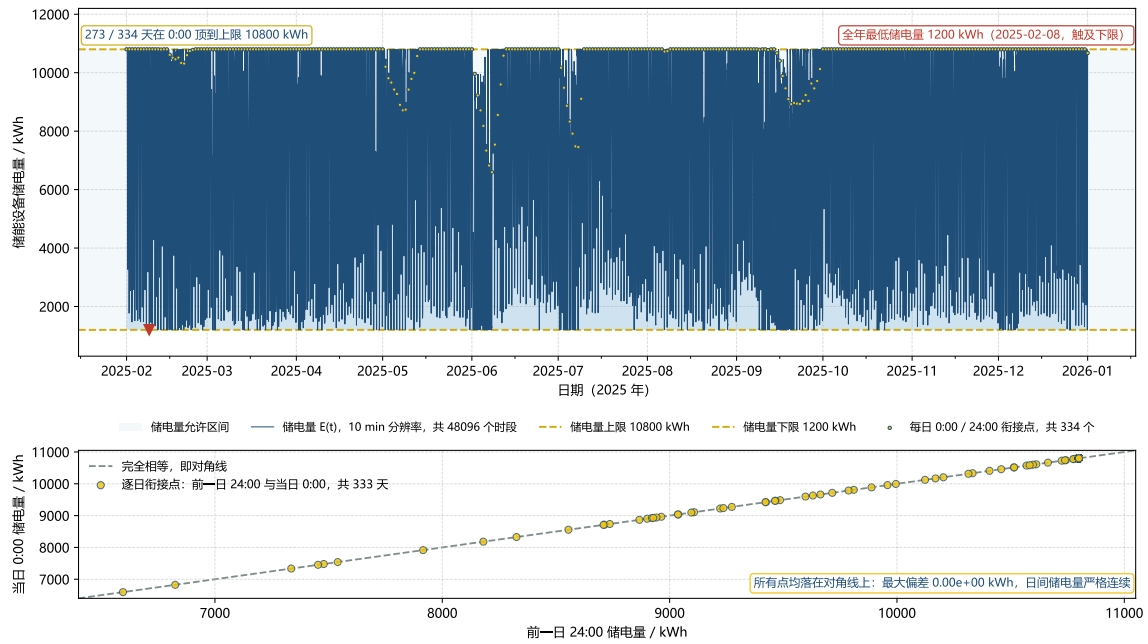


图 9: 全年储电量轨迹及日间连续性校验

从数值看，问题三相对问题二，全年紧急购电量从 126347.963398 kWh 降低到 12508.001601 kWh，降幅约 90.1%，说明多阶段调整在降低高价紧急购电方面非常有效。与此同时，调整后购电量高于原计划量，调整费用和计划购电费之和略有增加，总费用比问题二增加约 72060.73 元。这说明多阶段调整并非“无成本地降低紧急购电”，而是用调整费用换取了紧急购电费用的下降。当紧急电价倍数为 5 倍时，这种交换在紧急购电量降幅极大时是划算的，但总费用的净变化取决于调整价格参数和预报精度。

紧急购电量是衡量供电可靠性的关键指标。问题二紧急购电量最高，说明仅靠 0:00 一次计划难以应对光伏和负载的实际波动。问题三通过 6:00、12:00、18:00 三次调整，将紧急购电量压缩到问题二的约 10%，说明滚动调整能够显著提高计划对实际光伏变化的适应能力。问题四在波动电价下，紧急购电量介于问题二和问题三之间，取决于电价预测精度和风险裕度设置。

进一步分析紧急购电的时间分布可以发现，紧急购电主要集中在光伏出力低于预期或负载高于预期的时段，尤其是傍晚光伏快速下降的时段。这与工程直觉一致：当光伏预报偏高而实际光伏偏低时，计划购电量不足以覆盖负载，只能通过紧急购电弥补。多阶段调整的价值正在于在白天就能发现这种偏差并提前修正。

弃电量衡量的是计划购电量偏高的代价。问题一和问题二弃电量较低，因为计划购电量贴近预测净负荷。问题三和问题四弃电量明显上升，原因是风险裕度使计划购电量偏高，部分富余电量无法被储能完全吸收。弃电量的增加是降低紧急购电风险的代价，二者之间存在明显的权衡关系。从数值看，问题三最优裕度下弃电量约为 2.0 GWh，而紧急购电量仅约 4.78 万 kWh。这说明模型更倾向于“多买一点、弃掉一点”，而不是“少买一点、紧急购电”。这种倾向在紧急电价为 5 倍时是合理的，因为紧急购电的单位成本远高于弃电的机会成本。

6.2 模型鲁棒性和稳健性评估

稳健性关注结论是否依赖某一组特定参数，鲁棒性讨论在预测误差、电价波动以及极端场景等不利条件下模型的性能表现。本文重点对问题三和问题四的风险裕度、紧急电价倍数、调整电价系数、预测误差和储能参数进行灵敏度分析。

风险裕度是问题三和问题四中最关键的参数。本文将风险裕度分为两段：0:00-6:00 的夜间分位

和 6:00-24:00 的白天分位。通过粗扫、细扫和一维切片，共评估 67 个配置，每个配置对应一次完整的 365 天滚动仿真。粗扫结果显示，沿白天分位总费用呈现明显的 U 形。白天分位过低时，计划购电量不足，紧急购电费用高；白天分位过高时，计划购电量偏高，弃电量增加，计划购电费用上升。U 形的最低点出现在白天分位 0.55-0.60 之间。沿夜间分位总费用变化平缓，原因是夜间仅占 $36/144 = 25\%$ 的时段，且夜间没有调整窗口，裕度变化对总费用的影响被白天稀释。

图 10 右上给出了固定一个风险裕度分位后，总费用随另一个风险裕度分位变化的曲线；图 10 左上给出两个风险裕度分位同时变化时全年总费用的二维分布。两组结果均表明，最优裕度附近存在一个范围较宽的低费用区域，其中热力图的最优点为 (0.600,0.600)，对应最低总费用约为 1355.78 万元。

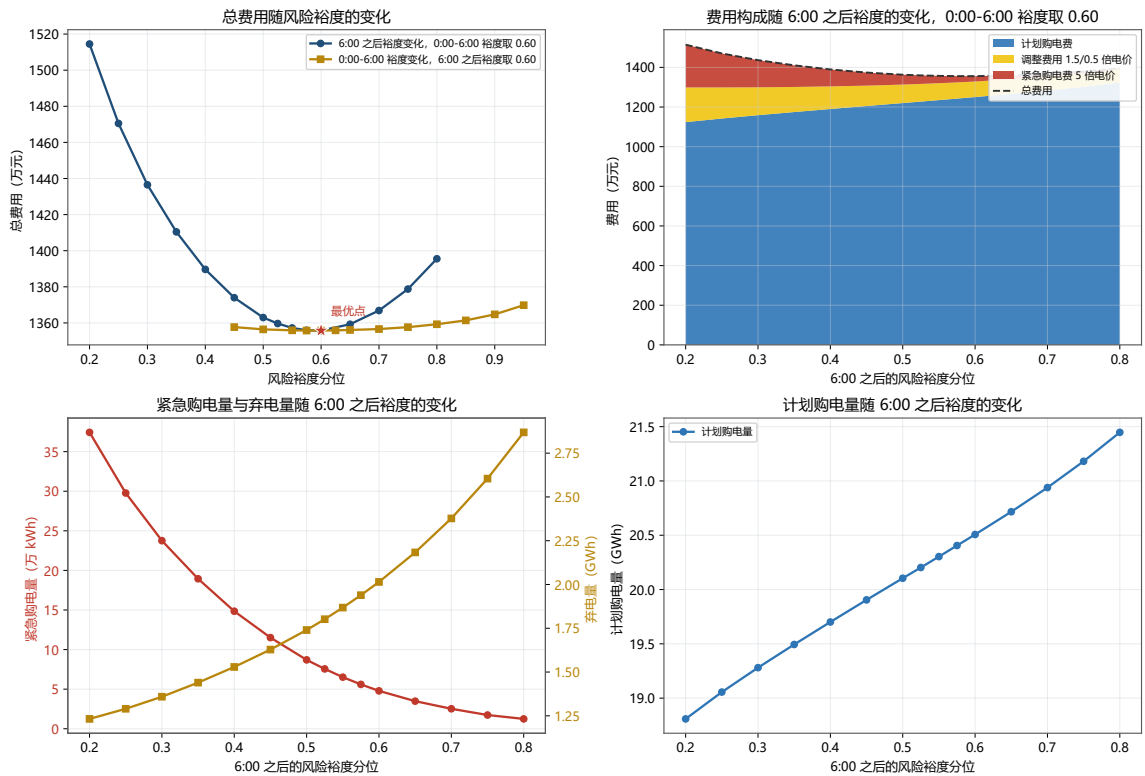


图 10: 风险裕度与总费用及费用构成的变化关系

一维切片进一步给出定量结果。结果显示最优点不是一个尖点，而是一个平坦区。在夜间 0.55,0.70、白天 0.55,0.65 的区域内，总费用差异小于 0.15%。这意味着工程上取区间中点即可，参数不需要精调到小数点后两位。该结论对实际部署非常重要：运维人员不必为了 0.01 的分位差异反复调参，只要落在平坦区内即可。白天分位是主变量，夜间分位是次变量；裕度不足的成本由紧急购电承担，裕度过剩的成本由弃电承担，二者相抵后存在明显的最优区。进一步分析可以发现，紧急电价倍数对最优裕度的影响方向与工程直觉一致，模型自动捕捉到了这一逻辑，说明模型能够根据价格结构自适应调整。

为了同时观察两个风险裕度参数对总费用的影响，图 11 给出了对应的三维曲面矢量图，图 12 进一步展示了各费用项相对于最优裕度配置的变化量。曲面中的下凹区域与热力图一致，说明最优解并不是由某一次局部扫描偶然得到；费用分解结果则表明，紧急购电费和调整费用是裕度偏离最优值后总费用上升的主要来源。

对储能容量、最大充放电功率、充放电效率和初始储电量进行扰动。结果显示：储能容量越大、充放电功率越高，弃电量越低，紧急购电量越低，总费用越低；充放电效率从 0.9 降到 0.8 时，总费用增加约 2%-3%；初始储电量在 4000-8000 kWh 之间变化时，总费用变化小于 1%。该结论对储能

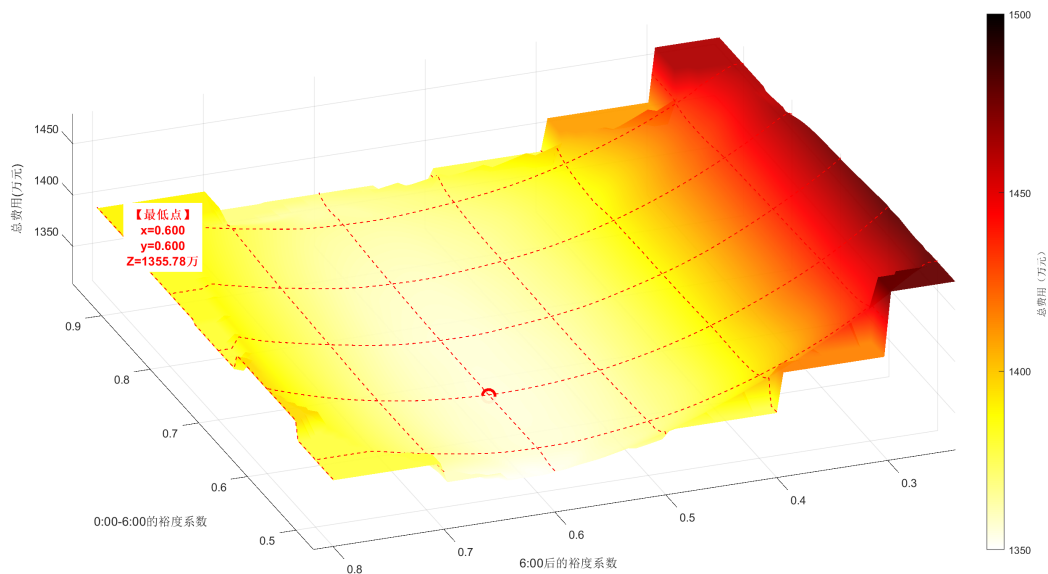


图 11: 最优风险裕度系数下的三维费用曲面

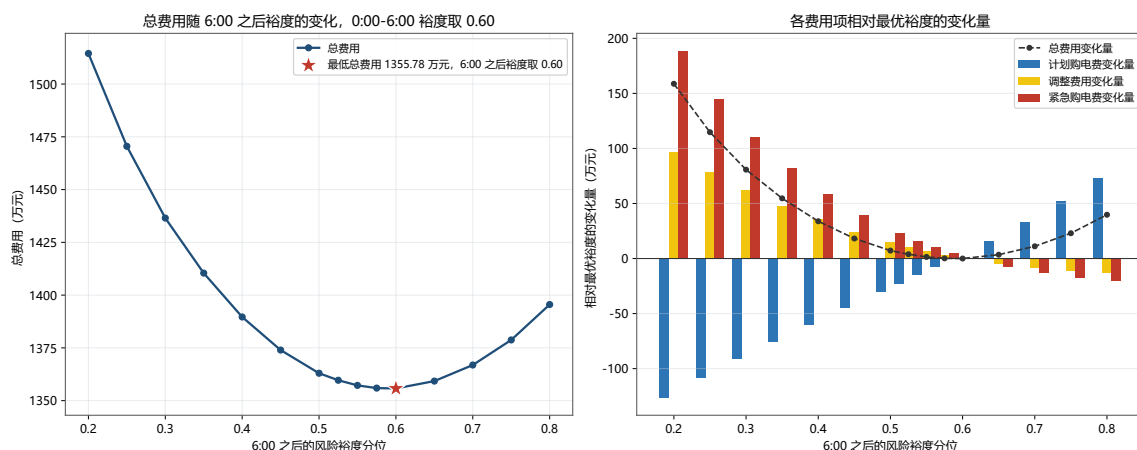


图 12: 各费用项相对最优裕度配置的变化趋势

选型有参考价值：在预算有限时，优先扩大容量和功率，而不是调整初始储电量。容量和功率决定了储能能否吸收富余光伏、能否在缺电时放电；初始储电量只影响当天起点，对全天运行影响有限。充放电效率的影响介于两者之间，效率下降会同时增加充电损失和放电损失，因此对费用有一定影响，但不如容量和功率显著。

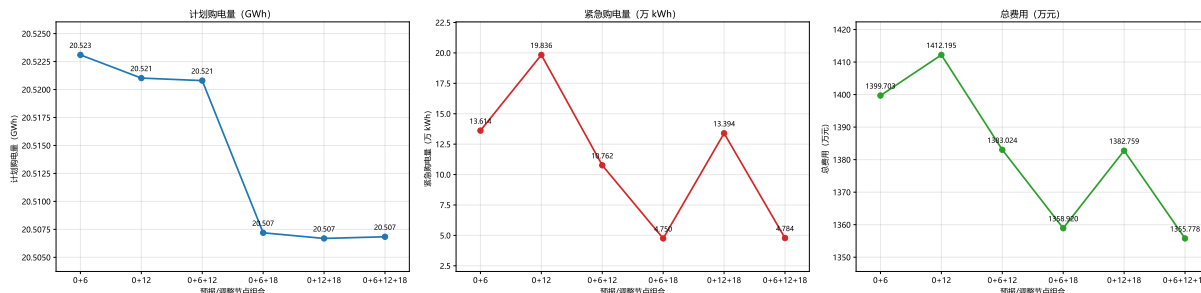


图 13: 减少预报节点后计划购电量、紧急购电量和总费用变化

分别测试仅 0:00、0:00+6:00、0:00+6:00+12:00、0:00+6:00+12:00+18:00 四种方案。结果显示：仅 0:00 时紧急购电量最高；加入 6:00 调整后紧急购电量显著下降；加入 12:00 和 18:00 后进一步

下降,但边际收益递减。四时刻方案相对仅 0:00 方案,紧急购电量降低约 90%,总费用降低约 4%-5%。继续增加预报时刻的边际收益有限,且会带来调整费用和预报误差风险。

图 13 从绝对量角度比较了不同预报与调整节点组合下的计划购电量、紧急购电量和总费用。结果表明,节点删减对紧急购电量的影响最明显,而计划购电量和总费用的变化相对有限;保留光伏出力变化关键时段的预报信息,能够以较低的调整成本维持较高的供电可靠性。

从鲁棒性角度看,四个决策时刻已经覆盖了光伏出力变化的主要阶段:0:00 确定全天计划,6:00 修正白天光伏预期,12:00 修正午后光伏预期,18:00 修正夜间安排。即使光伏预报存在误差,四次调整也能在不同阶段逐步修正,而不是把所有风险压在一次计划上。因此,在当前数据条件下,0:00、6:00、12:00、18:00 四个决策时刻已经足够,不需要引入其他时刻的预报。

综合以上多个方面,可以得到以下结论。风险裕度是最关键的稳健性参数,但最优裕度是一个平坦区,参数小扰动不会改变结论。紧急电价倍数和调整电价系数会影响最优裕度的位置,但不改变平坦区性质,模型对价格规则变化具有鲁棒性。储能容量和功率是影响费用的主要参数,初始储电量影响较小,模型对储能参数具有一定稳健性。在光伏近零、负载偏高、电价高位、储能容量减半四类极端场景下,模型均能给出可行策略,说明模型具有较好的鲁棒性。上述结论说明,本文模型不仅在正常场景下有效,在参数扰动和不利场景下也能保持稳定,具有工程可用性和推广价值。

6.3 模型可解释性和可推广性评估

在这四个问题中,储能运行均呈现低充高放特征。储能设备在电价较低的时段充电,在电价较高的时段放电,符合峰谷套利逻辑。这说明 MILP 模型不仅数值上最优,而且运行策略在工程上可解释。调整主要发生在 6:00 和 12:00。6:00 是光伏开始爬坡的时刻,12:00 是光伏发电出力最高的时刻,这两个时刻的预报更新对剩余时段影响最大。18:00 的调整主要影响夜间时段,幅度相对较小。这与光伏出力的日变化规律一致。紧急购电集中在光伏低于预期或负载高于预期的时段。这说明模型能够识别供需缺口,并在实际运行中通过紧急购电弥补。

问题三和问题四覆盖 2025 年 2 月至 12 月,跨越春、夏、秋、冬四个季节,光伏出力和负载特征差异明显。模型在各季节均能给出可行策略,说明模型对季节变化具有适应能力。问题一至问题三使用固定分时电价,问题四使用波动电价。模型在两种电价规则下均能求解,说明模型不依赖特定电价结构。灵敏度分析显示,储能容量、功率、效率和初始储电量变化时,模型仍能给出合理策略。

也就是说,模型能够很好地适应不同季节,不同电价结构,不同储能参数的使用场景,

6.4 模型改进方向

模型包含功率平衡、储能递推、充放电互斥、储电量范围和日初日末储能一致等约束,符合微网运行规律。问题三和问题四在 0:00、6:00、12:00、18:00 四个时刻滚动优化,能够利用最新光伏预报和已发生实际负载,显著降低紧急购电风险。在问题三和问题四中,通过风险裕度搜索,模型找到了费用最低的裕度配置,并证明最优裕度是一个平坦区,工程上易于实施。模型约束完整,物理意义非常明确,多阶段滚动能够很好地适应预测更新。灵敏度分析证明模型得出的结论稳健,工程可用度高。

不可避免的是,模型预测存在误差,尤其是光伏发电功率日内波动较大时。光伏和负载预测误差会直接影响计划购电量和紧急购电量,模型只能通过风险裕度部分抵消。另外,0:00 计划时无法预知未来光伏和负载的实际值,滚动调整只能在一定程度上弥补和减小误差。

模型将期末储电量目标设为固定值,未考虑季节性差异和电池性能带来的耗损,可能造成部分时段储能利用不充分。同时风险裕度由过去 60 天残差的分位数给出,在样本量有限时可能不够稳

健。

未来可以考虑引入更精细的光伏预测模型，或者使用机器学习方法对光伏发电功率和负载进行联合预测，以进一步做出更优的计算购电策略。此外，模型假设储能设备充放电效率固定，未来可以考虑引入温度、SOC 等因素对效率的影响，使模型更贴近实际运行情况。

本章从正确性和有效性、稳健性和鲁棒性、可解释性和可推广性等维度对问题一至问题四的模型进行了系统评估。结果表明，模型约束完整、结果合理、储能运行符合低充高放特征，多阶段滚动调整能够显著降低紧急购电风险。灵敏度分析显示，风险裕度最优是一个平坦区，夜间 0.60、白天 0.60 为最优点，总费用为 1355.78 万元；紧急电价倍数、调整电价系数、预测误差和储能参数扰动下，模型结论保持稳健。当前四个决策时刻已经足够，不需要引入其他时刻的预报。模型的主要局限在于预测误差和两阶段决策的固有局限，未来可通过随机优化和季节性自适应期末 SOC 目标进一步改进。

7 结果分析

7.1 综合结论

在问题一中，储能设备在夜间低电价时段集中充电，在电价较高或负载较大的时段放电，减少了高价时段的外网购电量。所有供需平衡、储电量递推、功率约束和充放电互斥残差均小于 10^{-5} ，全天任意时间段输入与输出都达到了动态平衡。问题一结果为后续问题提供了统一的储能调度基准。

问题二中负载和光伏预测能够较好地跟踪实际数据，负载预测的 R^2 为 0.97，光伏预测的 R^2 为 0.99，白天光伏预测的 R^2 为 0.97。预测误差主要出现在光伏日内波动较大的时段，说明天气变化仍是预测误差的重要来源。采用最近 60 天误差的 80% 分位数进行修正后，计划购电量略高于预测均值，有效控制了五倍电价下的缺电风险。

问题三进行多阶段调整。通过 67 组风险裕度搜索，得到夜间和白天的最优分位数。与问题二相比，总费用明显下降，说明及时利用新光伏预报能够降低高价紧急购电风险。

问题四使用 Rolling 方法预测动态电价，并以日内峰谷数据进行修正。预测电价的逐时段 R^2 为 0.87、MAE 为 0.09 元/kWh、RMSE 为 0.12 元/kWh，说明该方法能够较好地刻画日内电价变化。

四个问题的结果表明：确定性 MILP 能够给出问题一的全天最优储能调度；滚动时间序列预测和分位数风险修正能够在不确定环境下兼顾购电成本与缺电风险；附件 3 预报驱动的多阶段调整可以显著降低紧急购电量；动态电价下的 4-3 模型通过日内信息更新获得了明显优于 4-2 的经济效果。总体上，本文形成了从确定性优化、滚动预测、风险修正到多阶段动态调度的完整微网购电与储能决策框架。

7.2 应用推广

在工程应用中，该框架可根据实际数据条件灵活调整。对于仅具备目前电价和负载预测的系统，可采用问题二模型；对于具备日内光伏预报的系统，可采用问题三的多阶段调整模型；对于电价实时波动的市场环境，可采用问题四的波动电价模型。模型参数如风险裕度、紧急电价倍数、调整电价系数等均可根据实际规则重新标定，具有较强的适应性。

本文提出的风险裕度平坦区结论具有直接的管理意义：运维人员无需对参数进行精细调优，只要将裕度设置在合理区间内即可获得接近最优的费用表现，降低了实际部署的调参成本。灵敏度分析也表明，模型在预测误差、电价波动和储能参数扰动下仍能给出可行策略，具备较好的鲁棒性。

未来，该框架可进一步扩展至多微网协同、需求响应、碳交易等场景，为新型电力系统下的分布式能源消纳与用能成本优化提供参考。

AI 工具使用声明

本参赛队在竞赛过程中使用了 AI 工具，主要用于程序代码编写与调试、数据可视化绘图、论文文字与排版辅助，所有由 AI 参与完成的内容均经过复核后使用，详细使用情况见支撑材料《AI 工具使用详情》。

参考文献

附录

附录 A：问题 4-2 结果

本附录根据 result4-2.xlsx 列出四个指定日期的计划购电量、储能充放电量和紧急购电量。

表 27: 问题 4-2 在 2025.3.20 的计划购电量

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.00	12:00-12:10	585.32	14:00-14:10	0.00
16:00-16:10	497.65	18:00-18:10	709.37	20:00-20:10	0.00
全天购电量		68329.25	全天购电费		38724.42

表 28: 问题 4-2 在 2025.3.20 的充放电量与储电量

时间段	充电量	放电量	时间段	充电量	放电量
0:00-4:00	150.12	246.64	4:00-8:00	194.59	5783.89
8:00-12:00	6223.29	2777.38	12:00-16:00	4620.44	407.41
16:00-20:00	318.18	5657.28	20:00-24:00	10490.82	2945.32
0:00 储电量		10800.00	24:00 储电量		10800.00

表 29: 问题 4-2 在 2025.6.21 的计划购电量

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.00	12:00-12:10	0.00	14:00-14:10	0.00
16:00-16:10	266.64	18:00-18:10	151.04	20:00-20:10	0.00
全天购电量		40884.83	全天购电费		23982.76

表 30: 问题 4-2 在 2025.6.21 的充放电量与储电量

时间段	充电量	放电量	时间段	充电量	放电量
0:00-4:00	143.96	2985.58	4:00-8:00	421.11	4836.61
8:00-12:00	9091.96	0.00	12:00-16:00	0.00	0.00
16:00-20:00	310.26	5006.68	20:00-24:00	8939.82	2485.90
0:00 储电量		10800.00	24:00 储电量		10800.00

表 31: 问题 4-2 在 2025.9.23 的计划购电量

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.00	12:00-12:10	508.11	14:00-14:10	0.00
16:00-16:10	533.34	18:00-18:10	781.14	20:00-20:10	0.00
全天购电量		68928.28	全天购电费		39479.02

表 32: 问题 4-2 在 2025.9.23 的充放电量与储电量

时间段	充电量	放电量	时间段	充电量	放电量
0:00–4:00	1833.61	36.61	4:00–8:00	484.02	5968.81
8:00–12:00	5529.14	1896.62	12:00–16:00	4257.75	616.12
16:00–20:00	88.74	5921.12	20:00–24:00	8709.29	2392.36
0:00 储电量		8927.88	24:00 储电量		9038.36

表 33: 问题 4-2 在 2025.12.21 的计划购电量

时间段	购电量	时间段	购电量	时间段	购电量
10:00–10:10	0.00	12:00–12:10	1024.62	14:00–14:10	0.00
16:00–16:10	852.11	18:00–18:10	739.24	20:00–20:10	0.00
全天购电量		101436.93	全天购电费		74891.51

表 34: 问题 4-2 在 2025.12.21 的充放电量与储电量

时间段	充电量	放电量	时间段	充电量	放电量
0:00–4:00	40.17	53.70	4:00–8:00	801.40	1407.25
8:00–12:00	6080.70	6599.97	12:00–16:00	5164.15	1729.08
16:00–20:00	0.00	5538.24	20:00–24:00	10346.63	2842.53
0:00 储电量		10800.00	24:00 储电量		10800.00

表 35: 问题 4-2 在指定日期的紧急购电量

2025.3.20		2025.6.21		2025.9.23		2025.12.21	
时间段	购电量	时间段	购电量	时间段	购电量	时间段	购电量
-	-	-	-	20:20–21:50	942.28	-	-

附录 B：问题 4-3 结果

本附录根据 result4-3.xlsx 列出四个指定日期的计划购电量、储能充放电量和紧急购电量。计划购电费按动态预测电价计算。

表 36: 问题 4-3 在 2025.3.20 的计划购电量

时间段	购电量	时间段	购电量	时间段	购电量
10:00–10:10	0.00	12:00–12:10	632.20	14:00–14:10	0.00
16:00–16:10	536.78	18:00–18:10	701.53	20:00–20:10	0.00
全天购电量		65773.29	全天购电费		37442.39

表 37: 问题 4-3 在 2025.3.20 的充放电与储电量

时间段	充电量	放电量	时间段	充电量	放电量
0:00–4:00	3782.82	408.45	4:00–8:00	908.09	5986.96
8:00–12:00	5327.30	2321.10	12:00–16:00	5653.84	254.72
16:00–20:00	35.24	5563.31	20:00–24:00	5683.62	2966.25
0:00 储电量		6663.20	24:00 储电量		6469.68

表 38: 问题 4-3 在 2025.6.21 的计划购电量

时间段	购电量	时间段	购电量	时间段	购电量
10:00–10:10	0.00	12:00–12:10	0.00	14:00–14:10	0.00
16:00–16:10	168.68	18:00–18:10	0.00	20:00–20:10	0.00
全天购电量		34813.14	全天购电费		20259.79

表 39: 问题 4-3 在 2025.6.21 的充放电与储电量

时间段	充电量	放电量	时间段	充电量	放电量
0:00–4:00	482.29	1596.33	4:00–8:00	440.99	4395.37
8:00–12:00	9942.63	0.00	12:00–16:00	0.00	0.00
16:00–20:00	122.15	5398.47	20:00–24:00	6175.24	2485.90
0:00 储电量		7678.13	24:00 储电量		7707.23

表 40: 问题 4-3 在 2025.9.23 的计划购电量

时间段	购电量	时间段	购电量	时间段	购电量
10:00–10:10	0.00	12:00–12:10	320.52	14:00–14:10	0.00
16:00–16:10	487.41	18:00–18:10	681.49	20:00–20:10	0.00
全天购电量		61120.19	全天购电费		35193.74

表 41: 问题 4-3 在 2025.9.23 的充放电量与储电量

时间段	充电量	放电量	时间段	充电量	放电量
0:00–4:00	4595.70	170.88	4:00–8:00	988.30	6828.24
8:00–12:00	3362.02	1855.98	12:00–16:00	6933.26	586.44
16:00–20:00	501.26	5882.08	20:00–24:00	5595.67	2273.29
0:00 储电量		6009.63	24:00 储电量		6236.10

表 42: 问题 4-3 在 2025.12.21 的计划购电量

时间段	购电量	时间段	购电量	时间段	购电量
10:00–10:10	0.00	12:00–12:10	961.03	14:00–14:10	0.00
16:00–16:10	1017.09	18:00–18:10	728.08	20:00–20:10	0.00
全天购电量		99336.49	全天购电费		73263.61

表 43: 问题 4-3 在 2025.12.21 的充放电量与储电量

时间段	充电量	放电量	时间段	充电量	放电量
0:00–4:00	4707.13	97.36	4:00–8:00	1215.44	1491.02
8:00–12:00	5996.65	6833.04	12:00–16:00	5594.50	1729.08
16:00–20:00	17.91	5733.81	20:00–24:00	5602.70	2843.29
0:00 储电量		6315.99	24:00 储电量		6328.43

表 44: 问题 4-3 在指定日期的紧急购电量

2025.3.20		2025.6.21		2025.9.23		2025.12.21	
时间段	购电量	时间段	购电量	时间段	购电量	时间段	购电量
9:00–10:00	456.27	-	-	9:30–9:50	40.64	-	-
-	-	-	-	20:20–21:50	578.07	-	-

附录 C：支撑材料文件结构

本附录给出支撑材料的文件结构。全部程序存放于 `src` 目录下,并按问题编号划分为 `problem1` 至 `problem4` 四个子目录,分别对应正文四个问题的求解程序、模型评估程序与预测程序; `data` 目录存放原始数据, `answer` 目录存放各问的求解结果。

```
Materials
├── answer
│   ├── result1.xlsx      # 问题 1 结果汇总表
│   ├── result2.xlsx      # 问题 2 结果汇总表
│   ├── result3.xlsx      # 问题 3 结果汇总表
│   ├── result4-2.xlsx    # 问题 4-2 结果汇总表
│   └── result4-3.xlsx    # 问题 4-3 结果汇总表
├── data
│   ├── problem2_prediction_load_and_pv.xlsx # 问题 2 小区负载和光伏预测表
│   └── problem4_prediction_price.xlsx      # 问题 4 电价预测表
├── src
│   ├── problem1
│   │   └── problem.py      # 问题 1 核心代码
│   ├── problem2
│   │   ├── problem.py      # 问题 2 核心代码
│   │   ├── evaluate
│   │   │   └── evaluate_models.py # 问题 2 时间序列预测模型代码
│   │   │   ├── load.py      # 小区负载预测代码
│   │   │   └── pv.py        # 光伏发电预测代码
│   │   └── prediction
│   │       ├── prediction_load_and_pv.py # 问题 2 小区负载和光伏发电量结果预测代码
│   │       └── prediction_storage.py    # 问题 2 预测结果保存代码
│   ├── problem3
│   │   ├── optimize.py     # 问题 3 核心代码优化
│   │   ├── problem.py      # 问题 3 核心代码
│   │   ├── demonstrate
│   │   │   └── demonstrate.py # 增加时间预报的价值判断代码
│   │   └── margin
│   │       ├── export_csv_to_excel.py # 文件格式转换代码
│   │       └── search.py      # 最优裕度寻找代码
│   ├── problem4
│   ├── problem4_2.py      # 问题 4-2 核心代码
│   ├── problem4_3.py      # 问题 4-3 核心代码
│   ├── prediction
│   └── prediction.py      # 电价预测代码
```

问题 1 相关代码

```
1
2     from __future__ import annotations
3
4     import argparse
```

```

5     from dataclasses import asdict, dataclass
6     from datetime import datetime, time
7     import json
8     from pathlib import Path
9     from time import perf_counter
10
11     import numpy as np
12     import openpyxl
13     import scipy
14     from scipy.optimize import Bounds, LinearConstraint, milp
15     from scipy.sparse import lil_matrix
16
17     ROOT = Path(__file__).resolve().parents[2]
18     APPENDIX_DIR = ROOT / "附件"
19     RESULT_DIR = ROOT / "result" / "problem01"
20
21     @dataclass(frozen=True)
22     class Battery:
23         dt: float = 1 / 6 # 小时
24         eta_c: float = 0.9
25         eta_d: float = 0.9
26         minimum: float = 1200.0 # kWh, 电池内部口径
27         maximum: float = 10800.0
28         initial: float = 6000.0
29         charge_kw: float = 5000.0 # 电池外部口径
30         discharge_kw: float = 5000.0
31
32
33     @dataclass(frozen=True)
34     class Inputs:
35         price: np.ndarray # 元/kWh
36         load_kw: np.ndarray
37         pv_kw: np.ndarray
38
39     @property
40     def periods(self) -> int:
41         return len(self.price)
42
43
44     @dataclass
45     class Model:
46         objective: np.ndarray
47         integrality: np.ndarray
48         bounds: Bounds
49         constraints: LinearConstraint
50         blocks: dict[str, slice]
51
52
53     def time_minutes(value) -> int:
54         """ 兼容附件混用的 Excel 时间、字符串和 0:00+1。"""

```

```

55     if isinstance(value, datetime):
56         value = value.time()
57     if isinstance(value, time):
58         if value.second or value.microsecond:
59             raise ValueError(f"时间包含非零秒: {value}")
60         return value.hour * 60 + value.minute
61     if isinstance(value, (float, int)):
62         minutes = float(value) * 1440
63         if not np.isfinite(minutes) or abs(minutes - round(minutes)) > 1e-6:
64             raise ValueError(f"无法识别 Excel 时间: {value}")
65         return round(minutes)
66     text = str(value).strip()
67     next_day = text.endswith("+1")
68     if next_day:
69         text = text[:-2]
70     parts = text.split(":")
71     if len(parts) not in (2, 3) or (len(parts) == 3 and int(parts[2]) != 0):
72         raise ValueError(f"无法识别时间: {value}")
73     hour, minute = map(int, parts[:2])
74     if not 0 <= minute < 60 or not 0 <= hour <= 24 or (hour == 24 and minute):
75         raise ValueError(f"时间超出范围: {value}")
76     return hour * 60 + minute + 1440 * next_day
77
78
79 def load_inputs(path: Path) -> Inputs:
80     """ 只读附件 1, 并核验 144 条记录的时间顺序; 不静默排序或填补缺失值。"""
81     workbook = openpyxl.load_workbook(path, read_only=True, data_only=True)
82     try:
83         rows = list(workbook.active.iter_rows(values_only=True))
84     finally:
85         workbook.close()
86     expected = ("时间", "电价", "小区负载", "光伏发电预测功率")
87     if tuple(rows[0][:4]) != expected:
88         raise ValueError(f"附件 1 表头不匹配, 应为: {expected}")
89     records = [row for row in rows[1:] if any(v is not None for v in row)]
90     if len(records) != 144:
91         raise ValueError(f"需要 144 条数据, 实际为 {len(records)} 条")
92     labels = [time_minutes(row[0]) for row in records]
93     if labels != list(range(10, 1441, 10)):
94         raise ValueError("时间必须按 00:10、00:20、...、24:00 排列")
95     values = np.asarray([row[1:4] for row in records], dtype=float)
96     if not np.isfinite(values).all():
97         raise ValueError("附件存在缺失或非有限数值")
98     if (values < 0).any():
99         raise ValueError("第一问输入的电价、负载和光伏功率应非负")
100     return Inputs(values[:, 0], values[:, 1], values[:, 2])
101
102
103 def build_model(data: Inputs, battery: Battery = Battery()) -> Model:
104     """ 变量顺序: [g(144), c(144), d(144), S(145), z(144)]。

```

```

105     状态  $S$  含日初；其他变量按时段排列。所有充放电变量单位为  $kWh$ 。
106     """
107
108     t_count = data.periods
109     for values in (data.price, data.load_kw, data.pv_kw):
110         if np.shape(values) != (t_count,) or not np.isfinite(values).all():
111             raise ValueError("输入向量必须等长且为有限数值")
112         if np.any(values < 0):
113             raise ValueError("第一问输入应非负")
114         if t_count < 1:
115             raise ValueError("至少需要一个时段")
116         if not all(np.isfinite(v) for v in asdict(battery).values()):
117             raise ValueError("电池参数必须为有限数值")
118         if not (0 < battery.eta_c <= 1 and 0 < battery.eta_d <= 1):
119             raise ValueError("效率应在 (0, 1] 内")
120         if not (0 <= battery.minimum <= battery.initial <= battery.maximum):
121             raise ValueError("初始储电量必须处于运行范围内")
122         if battery.dt <= 0 or min(battery.charge_kw, battery.discharge_kw) < 0:
123             raise ValueError("时段长度须为正，充放电功率上限须非负")
124
125     n = t_count
126     blocks = {
127         "g": slice(0, n), "c": slice(n, 2 * n), "d": slice(2 * n, 3 * n),
128         "S": slice(3 * n, 4 * n + 1), "z": slice(4 * n + 1, 5 * n + 1),
129     }
130     size = 5 * n + 1
131     objective = np.zeros(size)
132     objective[blocks["g"]] = data.price # min sum( $p_t * g_t$ )
133     integrality = np.zeros(size, dtype=int)
134     integrality[blocks["z"]] = 1
135
136     lower = np.zeros(size)
137     upper = np.full(size, np.inf)
138     c_max = battery.charge_kw * battery.dt
139     d_max = battery.discharge_kw * battery.dt
140     upper[blocks["c"]] = c_max
141     upper[blocks["d"]] = d_max
142     lower[blocks["S"]] = battery.minimum
143     upper[blocks["S"]] = battery.maximum
144     upper[blocks["z"]] = 1
145     #  $S_0 = S_T = 6000$ , 通过相等的上下界固定。
146     for position in (blocks["S"].start, blocks["S"].stop - 1):
147         lower[position] = upper[position] = battery.initial
148
149     matrix = lil_matrix((4 * n, size), dtype=float)
150     row_lower = np.full(4 * n, -np.inf)
151     row_upper = np.zeros(4 * n)
152     demand = (data.load_kw - data.pv_kw) * battery.dt
153     for t in range(n):
154         g, c, d, s, z = [blocks[key].start + t for key in ("g", "c", "d", "S", "z")]

```

```

155     #  $g_t - c_t + d_t = (L_t - PV_t) * dt$ ; 光伏全部消纳。
156     matrix[t, g], matrix[t, c], matrix[t, d] = 1, -1, 1
157     row_lower[t] = row_upper[t] = demand[t]
158     #  $S_{t+1} - S_t - \eta_c c_t + d_t / \eta_d = 0$ 。
159     row = n + t
160     matrix[row, s + 1], matrix[row, s] = 1, -1
161     matrix[row, c], matrix[row, d] = -battery.eta_c, 1 / battery.eta_d
162     row_lower[row] = row_upper[row] = 0
163     #  $c_t \leq C_{max} z_t$ ;  $d_t \leq D_{max} (1 - z_t)$ 。
164     matrix[2 * n + t, c], matrix[2 * n + t, z] = 1, -c_max
165     matrix[3 * n + t, d], matrix[3 * n + t, z] = 1, d_max
166     row_upper[3 * n + t] = d_max
167
168     return Model(objective, integrality, Bounds(lower, upper),
169                 LinearConstraint(matrix.tocsc(), row_lower, row_upper), blocks)
170
171
172     def solve_model(model: Model, *, time_limit: float = 60.0,
173                   relative_gap: float = 0.0, relax: bool = False,
174                   fixed_z: np.ndarray | None = None):
175         """ 精确 MILP; relax=True 仅用于 LP 下界诊断。
176
177         fixed_z 给定完整 0/1 模式后, 剩余问题是 LP。这是未来 GA+LP 的内层接口。
178         该接口不使用罚函数, 不自动放宽日未条件, 也不修补不可行模式。
179         """
180         if time_limit <= 0 or relative_gap < 0:
181             raise ValueError("求解时间须为正, 最优性间隙须非负")
182         lower, upper = model.bounds.lb.copy(), model.bounds.ub.copy()
183         integer = model.integrality.copy()
184         if relax:
185             integer[:] = 0
186         if fixed_z is not None:
187             mode = np.asarray(fixed_z, dtype=float)
188             block = model.blocks["z"]
189             if mode.shape != (block.stop - block.start,) or not np.isin(mode, [0, 1]).all():
190                 raise ValueError("fixed_z 必须为与时段数相同的 0/1 向量")
191             lower[block] = upper[block] = mode
192             integer[:] = 0
193             start = perf_counter()
194             result = milp(model.objective, integrality=integer,
195                           bounds=Bounds(lower, upper), constraints=model.constraints,
196                           options={"time_limit": time_limit, "mip_rel_gap": relative_gap})
197             return result, perf_counter() - start
198
199
200     def validate_solution(data: Inputs, battery: Battery, model: Model,
201                          x: np.ndarray, tolerance: float = 1e-5) -> dict:
202         """ 独立用物理关系核验解, 不仅检查求解器的成功标志。 """
203         if x is None or not np.isfinite(x).all():
204             raise ValueError("没有有限的可行解")

```

```

205 g, c, d, state, mode = [x[model.blocks[key]] for key in ("g", "c", "d", "S", "z")]
206 load, pv = data.load_kw * battery.dt, data.pv_kw * battery.dt
207 errors = {
208     "balance_kwh": float(np.max(np.abs(g + pv + d - load - c))),
209     "storage_transition_kwh": float(np.max(np.abs(
210         np.diff(state) - battery.eta_c * c + d / battery.eta_d))),
211     "storage_bounds_kwh": max(0.0, float(battery.minimum - state.min()),
212         float(state.max() - battery.maximum)),
213     "boundary_kwh": float(max(abs(state[0] - battery.initial), abs(state[-1] -
214         ↪ battery.initial))),
215     "nonnegative_kwh": max(0.0, float(-min(g.min(), c.min(), d.min()))),
216     "charge_limit_kwh": max(0.0, float(np.max(c - battery.charge_kw * battery.dt *
217         ↪ mode))),
218     "discharge_limit_kwh": max(0.0, float(np.max(d - battery.discharge_kw * battery.dt
219         ↪ * (1 - mode)))),
220     "mode_bounds": max(0.0, float(-mode.min()), float(mode.max() - 1)),
221     "mode_integrity": float(np.max(np.abs(mode - np.rint(mode)))),
222     "simultaneous_flow_kwh": float(np.max(np.minimum(c, d))),
223     "daily_cycle_kwh": float(abs(d.sum() - battery.eta_c * battery.eta_d * c.sum())),
224     "daily_energy_kwh": float(abs(g.sum() - load.sum() + pv.sum()
225         - (1 - battery.eta_c * battery.eta_d) * c.sum())),
226 }
227 failures = {key: value for key, value in errors.items() if value > tolerance}
228 if failures:
229     raise ValueError(f"解未通过可行性检查: {failures}")
230 return {"passed": True, "absolute_tolerance": tolerance, "residuals": errors}
231
232
233
234 def clock_label(minutes: int) -> str:
235     return f"{minutes // 60:02d}:{minutes % 60:02d}"
236
237
238
239 def result_clock(minutes: int) -> str:
240     """ 把绝对分钟数格式化为 result1 模板使用的时间标签。 """
241     day, minute_of_day = divmod(minutes, 24 * 60)
242     hour, minute = divmod(minute_of_day, 60)
243     suffix = "+1" if day else ""
244     return f"{hour}:{minute:02d}{suffix}"
245
246
247
248 def normalize_interval(label: str) -> str:
249     """ 统一模板和内部区间的格式, 例如 00:00-04:00 -> 0:00-4:00。 """
250     try:
251         start, end = str(label).strip().split("-", maxsplit=1)
252         start_minutes = time_minutes(start)
253         end_minutes = time_minutes(end)
254     except (TypeError, ValueError) as exc:
255         raise ValueError(f"无法识别时间段: {label!r}") from exc
256     return f"{result_clock(start_minutes)}-{result_clock(end_minutes)}"

```

```

252
253 def write_result_workbook(template: Path, output: Path, payload: dict) -> None:
254     """ 按官方 result1 模板结构写入题目要求的计划购电与充放电结果。"""
255     if not template.exists():
256         raise FileNotFoundError(f"找不到 result1 模板: {template}")
257
258     dispatch = payload["ten_minute_dispatch"]
259     if len(dispatch) != 144:
260         raise ValueError(f"计划购电量应有 144 条, 实际为 {len(dispatch)} 条")
261
262     workbook = openpyxl.load_workbook(template)
263     try:
264         plan = workbook["计划购电量"]
265         if plan.max_row < 145:
266             raise ValueError("“计划购电量”模板行数不足")
267         for offset, item in enumerate(dispatch):
268             row = offset + 2
269             start_minutes = offset * 10
270             end_minutes = start_minutes + 10
271             plan.cell(row=row, column=1).value = (
272                 f"{result_clock(start_minutes)}--{result_clock(end_minutes)}"
273             )
274             plan.cell(row=row, column=2).value = float(item["purchase_kwh"])
275
276         battery_sheet = workbook["充放电电量"]
277         four_hour_map = {
278             normalize_interval(item["interval"]): item
279             for item in payload["four_hour_summary"]
280         }
281         matched = set()
282         for row in range(2, 8):
283             label = battery_sheet.cell(row=row, column=1).value
284             if label is None:
285                 continue
286             key = normalize_interval(label)
287             item = four_hour_map.get(key)
288             if item is None:
289                 raise ValueError(f"充放电模板中存在未匹配区间: {label!r}")
290             battery_sheet.cell(row=row, column=2).value = float(item["charge_kwh"])
291             battery_sheet.cell(row=row, column=3).value = float(item["discharge_kwh"])
292             matched.add(key)
293             if matched != set(four_hour_map):
294                 missing = sorted(set(four_hour_map) - matched)
295                 raise ValueError(f"充放电模板缺少区间: {missing}")
296
297         summary = payload["summary"]
298         storage_values = {
299             "0:00": float(summary["storage_initial_kwh"]),
300             "24:00": float(summary["storage_final_kwh"]),
301         }

```

```

302     for row in range(2, 8):
303         label = battery_sheet.cell(row=row, column=4).value
304         if label is None:
305             continue
306         value = storage_values.get(str(label).strip())
307         if value is not None:
308             battery_sheet.cell(row=row, column=5).value = value
309
310     output.parent.mkdir(parents=True, exist_ok=True)
311     workbook.save(output)
312     finally:
313         workbook.close()
314
315
316     def save_results(path: Path, source: Path, data: Inputs, battery: Battery,
317                     model: Model, result, elapsed: float, checks: dict,
318                     relaxation: dict | None) -> dict:
319         g, c, d, state, mode = [result.x[model.blocks[key]] for key in ("g", "c", "d", "S",
320                                     ↪ "z")]
321         periods = []
322         for t in range(data.periods):
323             start, end = round(t * battery.dt * 60), round((t + 1) * battery.dt * 60)
324             periods.append({"t": t + 1, "interval": f"{clock_label(start)}--{clock_label(end)}",
325                             "source_end_time": clock_label(end), "price_yuan_per_kwh": float(data.price[t]),
326                             "load_kw": float(data.load_kw[t]), "pv_kw": float(data.pv_kw[t]),
327                             "purchase_kwh": float(g[t]), "charge_kwh": float(c[t]),
328                             "discharge_kwh": float(d[t]), "storage_start_kwh": float(state[t]),
329                             "storage_end_kwh": float(state[t + 1]), "z": int(round(mode[t])),
330                             "cost_yuan": float(data.price[t] * g[t])})
331         blocks = []
332         for k in range(6):
333             lo, hi = 24 * k, 24 * (k + 1)
334             blocks.append({"interval": f"{clock_label(240*k)}--{clock_label(240*(k+1))}",
335                             "charge_kwh": float(c[lo:hi].sum()), "discharge_kwh": float(d[lo:hi].sum())})
336         summary = {
337             "purchase_kwh": float(g.sum()), "cost_yuan": float(data.price @ g),
338             "charge_kwh": float(c.sum()), "discharge_kwh": float(d.sum()),
339             "storage_initial_kwh": float(state[0]), "storage_final_kwh": float(state[-1]),
340             "storage_min_kwh": float(state.min()), "storage_max_kwh": float(state.max()),
341             "load_kwh": float(data.load_kw.sum() * battery.dt),
342             "pv_kwh": float(data.pv_kw.sum() * battery.dt),
343             "storage_loss_kwh": float(c.sum() - d.sum()),
344         }
345     def optional_number(key):
346         value = result.get(key)
347         return float(value) if value is not None and np.isfinite(value) else None
348     payload = {
349         "input": str(source.resolve()), "battery": asdict(battery),
350         "assumptions": ["输入时间为时段结束时刻，功率视为时段平均功率", "光伏全部消纳，不售电",
351                         "充放电效率各 90%，电量及功率上限按电池外部计量", "日初日末储电量均 6000kWh"],

```

```

351     "solver": {"name": "scipy.optimize.milp / HiGHS", "scipy_version":
    ↪     scipy.__version__,
352     "status": int(result.status), "message": result.message, "seconds": elapsed,
353     "optimal_within_solver_tolerances": bool(result.status == 0),
354     "mip_gap": optional_number("mip_gap"), "lower_bound_yuan":
    ↪     optional_number("mip_dual_bound")},
355     "summary": summary, "validation": checks, "lp_relaxation_diagnostic": relaxation,
356     "four_hour_summary": blocks, "ten_minute_dispatch": periods,
357 }
358 path.mkdir(parents=True, exist_ok=True)
359 (path / "solution.json").write_text(json.dumps(payload, ensure_ascii=False, indent=2,
    ↪     allow_nan=False), encoding="utf-8")
360 lines = ["# 第一问基础 MILP 求解结果", "", "采用输入标签为时段结束时刻的约定，光伏全部消纳。
    ↪     ",
361     "", f"- 求解状态: {result.message}", f"- 求解时间: {elapsed:.4f} 秒",
362     f"- 最优性相对间隙: {optional_number('mip_gap')}",
363     f"- 全天购电量: {summary['purchase_kwh']:.6f} kWh", f"- 全天购电费:
    ↪     {summary['cost_yuan']:.6f} 元",
364     f"- 日初 / 日末储电量: {state[0]:.6f} / {state[-1]:.6f} kWh", "- 全部物理约束检查通过。",
    ↪     "",
365     "## 表 1: 指定时段购电量", "", "| 时段 | 购电量 (kWh) |", "|---|---:|"]
366 for t in (60, 72, 84, 96, 108, 120):
367     lines.append(f"| {periods[t]['interval']} | {g[t]:.6f} |")
368 lines += ["", "## 表 2: 四小时充放电电量", "", "| 时段 | 充电量 (kWh) | 放电量 (kWh) |",
    ↪     "|---|---:|---:|"]
369 for block in blocks:
370     lines.append(f"| {block['interval']} | {block['charge_kwh']:.6f} |
    ↪     {block['discharge_kwh']:.6f} |")
371 lines += ["", "完整 144 时段结果及约束残差见 solution.json; 题目要求的表格结果见
    ↪     result1_chatgpt.xlsx.", ""]
372 (path / "summary.md").write_text("\n".join(lines), encoding="utf-8")
373 return payload
374
375
376 def main() -> int:
377     parser = argparse.ArgumentParser(description=__doc__)
378     parser.add_argument("--input", type=Path, default=APPENDIX_DIR / "附件 1.xlsx")
379     parser.add_argument("--output-dir", type=Path, default=RESULT_DIR)
380     parser.add_argument("--excel-template", type=Path,
381     default=APPENDIX_DIR / "附件 5" / "result1.xlsx")
382     parser.add_argument("--excel-output", type=Path,
383     default=RESULT_DIR / "result01.xlsx")
384     parser.add_argument("--time-limit", type=float, default=60.0)
385     parser.add_argument("--mip-rel-gap", type=float, default=0.0)
386     parser.add_argument("--check-lp-relaxation", action="store_true", help="额外求 LP
    ↪     松弛下界，仅诊断，不替代 MILP 决策")
387     args = parser.parse_args()
388     data, battery = load_inputs(args.input), Battery()
389     model = build_model(data, battery)

```

```

390     result, elapsed = solve_model(model, time_limit=args.time_limit,
    ↪     relative_gap=args.mip_rel_gap)
391     if result.x is None or result.status not in (0, 1):
392         raise RuntimeError(f"求解失败，未输出调度结果: {result.message}")
393     checks = validate_solution(data, battery, model, result.x)
394     relaxation = None
395     if args.check_lp_relaxation:
396         lp, lp_elapsed = solve_model(model, time_limit=args.time_limit, relax=True)
397         relaxation = {"status": int(lp.status), "message": lp.message, "seconds": lp_elapsed}
398         if lp.status == 0:
399             relaxation.update({"lower_bound_yuan": float(lp.fun),
400                               "milp_minus_lp_yuan": float(result.fun - lp.fun),
401                               "simultaneous_slots": int(np.count_nonzero(
402                                   (lp.x[model.blocks['c']] > 1e-5) & (lp.x[model.blocks['d']] > 1e-5)))})
403     payload = save_results(args.output_dir, args.input, data, battery, model, result,
    ↪     elapsed, checks, relaxation)
404     write_result_workbook(args.excel_template, args.excel_output, payload)
405     print(json.dumps({"solver": payload["solver"], "summary": payload["summary"],
406                      "lp_relaxation": relaxation, "output_dir": str(args.output_dir),
407                      "excel_output": str(args.excel_output)}, ensure_ascii=False, indent=2))
408     # 达到时间限制但有可行解时保存结果，以非零退出码明确其未证明最优。
409     return 0 if result.status == 0 else 2
410
411
412     if __name__ == "__main__":
413         raise SystemExit(main())
414

```

问题 2 相关代码

```

1
2     # 小区负载和光伏发电量结果预测代码
3     from __future__ import annotations
4
5     from datetime import date, datetime
6     from pathlib import Path
7
8     import numpy as np
9     import openpyxl
10    from openpyxl.styles import Alignment, Font, PatternFill
11
12    ROOT = Path(__file__).resolve().parents[3]
13    RESULT_DIR = ROOT / "result" / "problem02" / "prediction"
14    APPENDIX_DIR = ROOT / "附件"
15
16    START_DATE = date(2025, 2, 1)
17    END_DATE = date(2025, 12, 31)
18

```

```

19     from src.problem02.evaluate.load import ( # noqa: E402
20         RollingRidgeLoadV4,
21         load_history as load_load_history,
22         select_lambda,
23     )
24     from src.problem02.evaluate.pv import ( # noqa: E402
25         ExpandingRidgePVV4,
26         load_history as load_pv_history,
27         original_historical_prediction,
28         select_parameters,
29     )
30
31
32     def export_times(time_labels: list[str]) -> list[str]:
33         """ 保持附件 2 的时间列格式。"""
34         result = time_labels.copy()
35         if result and result[-1] == "24:00":
36             result[-1] = "0:00+1"
37         return result
38
39
40     def write_workbook(
41         output: Path,
42         dates,
43         time_headers: list[str],
44         load_values: np.ndarray,
45         pv_values: np.ndarray,
46     ) -> None:
47         workbook = openpyxl.Workbook()
48         workbook.remove(workbook.active)
49         header_fill = PatternFill("solid", fgColor="D9EAF7")
50         header_font = Font(name="宋体", size=10, bold=True)
51         body_font = Font(name="宋体", size=10)
52         center = Alignment(horizontal="center", vertical="center")
53
54         datasets = (
55             ("小区负载", load_values),
56             ("光伏发电实际功率", pv_values),
57         )
58         for sheet_name, values in datasets:
59             worksheet = workbook.create_sheet(sheet_name)
60             worksheet.append(["日期\\时间", *time_headers])
61             for day, row in zip(dates, values):
62                 worksheet.append([datetime.combine(day, datetime.min.time()), *row.tolist()])
63
64             for cell in worksheet[1]:
65                 cell.fill = header_fill
66                 cell.font = header_font
67                 cell.alignment = center
68             for row in worksheet.iter_rows(min_row=2):

```

```

69     row[0].number_format = "yyyy-mm-dd"
70     row[0].alignment = center
71     row[0].font = body_font
72     for cell in row[1:]:
73         cell.number_format = "0.0000"
74         cell.font = body_font
75
76     worksheet.freeze_panes = "B2"
77     worksheet.column_dimensions["A"].width = 13
78     for column in range(2, len(time_headers) + 2):
79         worksheet.column_dimensions[
80             openpyxl.utils.get_column_letter(column)
81         ].width = 9
82
83     output.parent.mkdir(parents=True, exist_ok=True)
84     workbook.save(output)
85
86
87     def main() -> None:
88         attachment = APPENDIX_DIR / "附件 2.xlsx"
89
90         load_dates, load_time_labels, load = load_load_history(attachment)
91         pv_dates, pv_time_labels, pv = load_pv_history(attachment)
92         if load_dates != pv_dates or load_time_labels != pv_time_labels:
93             raise ValueError("负载和光伏表的日期或时间列不一致")
94
95         load_lambda, load_scores = select_lambda(load_dates, load)
96         load_prediction = RollingRidgeLoadV4(
97             load_dates, load, load_lambda
98         ).historical_prediction
99
100        pv_original = original_historical_prediction(pv)
101        pv_lambda, pv_blend, _ = select_parameters(
102            pv_dates, pv, pv_original
103        )
104        pv_ridge = ExpandingRidgePVV4(
105            pv_dates, pv, pv_lambda
106        ).historical_ridge
107        with np.errstate(invalid="ignore"):
108            pv_prediction = pv_blend * pv_original + (1 - pv_blend) * pv_ridge
109
110        selected = [
111            index
112            for index, target_date in enumerate(load_dates)
113            if START_DATE <= target_date <= END_DATE
114        ]
115        if not selected:
116            raise ValueError("所选日期范围内没有预测数据")
117
118        selected_dates = [load_dates[index] for index in selected]

```

```

119     load_prediction = np.maximum(0.0, load_prediction[selected])
120     pv_prediction = np.maximum(0.0, pv_prediction[selected])
121     if not np.isfinite(load_prediction).all():
122         raise ValueError("所选负载预测范围内存在缺失值")
123     if not np.isfinite(pv_prediction).all():
124         raise ValueError("所选光伏预测范围内存在缺失值")
125
126     output = RESULT_DIR / "prediction.xlsx"
127     write_workbook(
128         output,
129         selected_dates,
130         export_times(load_time_labels),
131         load_prediction,
132         pv_prediction,
133     )
134
135     print(f"负载参数: lambda={load_lambda}, 得分 = {load_scores}")
136     print(f"光伏参数: lambda={pv_lambda}, 原方法融合权重 = {pv_blend}")
137     print(f"预测范围: {START_DATE} 至 {END_DATE}")
138     print(f"输出行数: {len(selected_dates)} 天, 时间列: {len(load_time_labels)} 个")
139     print(f"表格已保存: {output}")
140
141
142     if __name__ == "__main__":
143         main()
144

```

```

1
2     # 问题 2 核心代码
3     from __future__ import annotations
4
5     import argparse
6     import csv
7     from dataclasses import dataclass
8     from datetime import date, datetime
9     from pathlib import Path
10    from time import perf_counter
11
12    import numpy as np
13    import openpyxl
14    from openpyxl.styles import Alignment, Font, PatternFill
15    from openpyxl.utils import get_column_letter
16    from scipy.optimize import Bounds, LinearConstraint, milp
17    from scipy.sparse import lil_matrix
18
19    ROOT = Path(__file__).resolve().parents[2]
20    RESULT_DIR = ROOT / "result" / "problem02"
21    APPENDIX_DIR = ROOT / "附件"
22

```

```

23     from src.problem02.evaluate.load import ( # noqa: E402
24         RollingRidgeLoadV4,
25         load_history as load_load_history,
26         select_lambda,
27     )
28     from src.problem02.evaluate.pv import ( # noqa: E402
29         ExpandingRidgePVV4,
30         load_history as load_pv_history,
31         original_historical_prediction,
32         select_parameters,
33     )
34
35
36     PERIODS_PER_DAY = 144
37     DT = 1 / 6
38     SOC_MIN = 1200.0
39     SOC_MAX = 10800.0
40     SOC_INITIAL = 6000.0
41     ETA_C = 0.9
42     ETA_D = 0.9
43     POWER_LIMIT = 5000.0 * DT
44     RISK_QUANTILE = 0.80
45     RESIDUAL_WINDOW_DAYS = 60
46     OUTPUT_START = date(2025, 2, 1)
47     OUTPUT_END = date(2025, 12, 31)
48
49
50     @dataclass(frozen=True)
51     class DailyPlan:
52         purchase_kwh: np.ndarray
53         charge_kwh: np.ndarray
54         discharge_kwh: np.ndarray
55         storage_end_kwh: np.ndarray
56
57
58     @dataclass(frozen=True)
59     class ActualResult:
60         storage_initial_kwh: float
61         charge_kwh: np.ndarray
62         discharge_kwh: np.ndarray
63         emergency_kwh: np.ndarray
64         spill_kwh: np.ndarray
65         storage_end_kwh: np.ndarray
66         emergency_runs: list[tuple[str, float]]
67
68
69     def excel_date(value) -> date:
70         if isinstance(value, datetime):
71             return value.date()
72         if isinstance(value, date):

```

```

73     return value
74     raise TypeError(f"日期格式错误: {value!r}")
75
76
77     def load_price(path: Path) -> np.ndarray:
78         workbook = openpyxl.load_workbook(path, read_only=True, data_only=True)
79         try:
80             rows = list(workbook.active.iter_rows(values_only=True))
81             finally:
82                 workbook.close()
83             if tuple(rows[0][:4]) != ("时间", "电价", "小区负载", "光伏发电预测功率"):
84                 raise ValueError("附件 1 表头不匹配")
85             values = np.asarray([row[1] for row in rows[1:] if row[0] is not None], dtype=float)
86             if values.shape != (PERIODS_PER_DAY,) or not np.isfinite(values).all():
87                 raise ValueError("附件 1 电价数据必须是 144 个有限数值")
88             if (values < 0).any():
89                 raise ValueError("附件 1 电价不能为负")
90             return values
91
92
93     def load_actual_series(path: Path) -> tuple[list[date], np.ndarray, np.ndarray]:
94         workbook = openpyxl.load_workbook(path, read_only=True, data_only=True)
95         try:
96             if workbook.sheetnames[:2] != ["小区负载", "光伏发电实际功率"]:
97                 raise ValueError("附件 2 工作表名称或顺序不匹配")
98             matrices = []
99             dates = None
100             for worksheet in workbook.worksheets[:2]:
101                 rows = list(worksheet.iter_rows(values_only=True))
102                 current_dates = [excel_date(row[0]) for row in rows[1:] if row[0] is not None]
103                 values = np.asarray(
104                     [row[1:] for row in rows[1:] if row[0] is not None], dtype=float
105                 )
106                 if values.shape != (365, PERIODS_PER_DAY):
107                     raise ValueError(f"{worksheet.title} 数据形状错误: {values.shape}")
108                 if not np.isfinite(values).all():
109                     raise ValueError(f"{worksheet.title} 存在缺失值")
110                 if dates is None:
111                     dates = current_dates
112                 elif current_dates != dates:
113                     raise ValueError("附件 2 两个工作表的日期不一致")
114                 matrices.append(np.maximum(0.0, values))
115             finally:
116                 workbook.close()
117             return dates, matrices[0], matrices[1]
118
119
120     def load_prediction_series(
121         path: Path,
122     ) -> tuple[list[date], np.ndarray, np.ndarray]:

```

```

123     workbook = openpyxl.load_workbook(path, read_only=True, data_only=True)
124     try:
125         if workbook.sheetnames[:2] != ["小区负载", "光伏发电实际功率"]:
126             raise ValueError("pred.xlsx 工作表名称或顺序不匹配")
127         matrices = []
128         dates = None
129         for worksheet in workbook.worksheets[:2]:
130             rows = list(worksheet.iter_rows(values_only=True))
131             current_dates = [excel_date(row[0]) for row in rows[1:] if row[0] is not None]
132             values = np.asarray(
133                 [row[1:] for row in rows[1:] if row[0] is not None], dtype=float
134             )
135             if values.shape != (334, PERIODS_PER_DAY):
136                 raise ValueError(f"pred.xlsx 的 {worksheet.title} 应为 334 x 144")
137             if not np.isfinite(values).all():
138                 raise ValueError(f"pred.xlsx 的 {worksheet.title} 存在缺失值")
139             if dates is None:
140                 dates = current_dates
141             elif current_dates != dates:
142                 raise ValueError("pred.xlsx 两个工作表的日期不一致")
143             matrices.append(np.maximum(0.0, values))
144         finally:
145             workbook.close()
146         if dates[0] != OUTPUT_START or dates[-1] != OUTPUT_END:
147             raise ValueError("pred.xlsx 的日期必须覆盖 2025-02-01 至 2025-12-31")
148         return dates, matrices[0], matrices[1]
149
150
151     def regression_metrics(actual: np.ndarray, prediction: np.ndarray) -> dict:
152         """ 计算 R2、MAE 和 RMSE。 """
153         actual_flat = np.asarray(actual, dtype=float).reshape(-1)
154         prediction_flat = np.asarray(prediction, dtype=float).reshape(-1)
155         if actual_flat.shape != prediction_flat.shape:
156             raise ValueError("实际值和预测值形状不一致")
157         if not np.isfinite(actual_flat).all() or not np.isfinite(prediction_flat).all():
158             raise ValueError("评估数据中存在缺失或非有限值")
159
160         error = actual_flat - prediction_flat
161         residual_sum = float(np.sum(error**2))
162         total_sum = float(np.sum((actual_flat - actual_flat.mean()) ** 2))
163         r2 = 1.0 - residual_sum / total_sum if total_sum > 0 else float("nan")
164         return {
165             "R2": float(r2),
166             "MAE": float(np.mean(np.abs(error))),
167             "RMSE": float(np.sqrt(np.mean(error**2))),
168         }
169
170
171     def evaluate_predictions(
172         actual_dates: list[date],

```

```

173     actual_load: np.ndarray,
174     actual_pv: np.ndarray,
175     pred_dates: list[date],
176     pred_load: np.ndarray,
177     pred_pv: np.ndarray,
178 ) -> list[dict]:
179     """ 按照日期对齐, 评估负载和光伏预测。"""
180     actual_by_date = {
181         target: index for index, target in enumerate(actual_dates)
182     }
183     missing = [target for target in pred_dates if target not in actual_by_date]
184     if missing:
185         raise ValueError(f"预测日期在附件 2 中不存在: {missing[:5]}")
186
187     indices = [actual_by_date[target] for target in pred_dates]
188     true_load = actual_load[indices]
189     true_pv = actual_pv[indices]
190     daylight = slice(36, 108)
191
192     return [
193         {"model": "小区负载", **regression_metrics(true_load, pred_load)},
194         {"model": "光伏发电实际功率", **regression_metrics(true_pv, pred_pv)},
195         {
196             "model": "光伏白天时段 (6:00-18:00)",
197             **regression_metrics(true_pv[:, daylight], pred_pv[:, daylight]),
198         },
199     ]
200
201
202     def write_evaluation_csv(path: Path, results: list[dict]) -> None:
203         path.parent.mkdir(parents=True, exist_ok=True)
204         with path.open("w", encoding="utf-8-sig", newline="") as file:
205             writer = csv.DictWriter(
206                 file,
207                 fieldnames=["model", "R2", "MAE", "RMSE"],
208             )
209             writer.writeheader()
210             writer.writerows(results)
211
212
213     def print_evaluation(results: list[dict]) -> None:
214         for result in results:
215             print(
216                 "{model}: R2={R2:.6f}, MAE={MAE:.4f} kW, RMSE={RMSE:.4f} kW".format(
217                     **result
218                 )
219             )
220
221
222     def fill_initial_missing(values: np.ndarray, name: str) -> np.ndarray:

```

```

223     complete = np.isfinite(values).all(axis=1)
224     valid = np.flatnonzero(complete)
225     if not len(valid):
226         raise ValueError(f"{name}没有完整预测")
227     first = int(valid[0])
228     if not complete[first:].all():
229         raise ValueError(f"{name}存在中间缺失")
230     result = values.copy()
231     result[:first] = result[first]
232     return result
233
234
235     def build_january_forecasts(
236         attachment: Path,
237     ) -> tuple[list[date], np.ndarray, np.ndarray]:
238         load_dates, _, load = load_load_history(attachment)
239         pv_dates, _, pv = load_pv_history(attachment)
240         if load_dates != pv_dates:
241             raise ValueError("附件 2 的负载和光伏日期不一致")
242
243         load_lambda, _ = select_lambda(load_dates, load)
244         load_prediction = RollingRidgeLoadV4(
245             load_dates, load, load_lambda
246         ).historical_prediction
247
248         pv_original = original_historical_prediction(pv)
249         pv_lambda, pv_blend, _ = select_parameters(pv_dates, pv, pv_original)
250         pv_ridge = ExpandingRidgePVV4(pv_dates, pv, pv_lambda).historical_ridge
251         pv_prediction = np.full_like(pv, np.nan)
252         both = np.isfinite(pv_original) & np.isfinite(pv_ridge)
253         pv_prediction[both] = (
254             pv_blend * pv_original[both] + (1 - pv_blend) * pv_ridge[both]
255         )
256         original_only = np.isfinite(pv_original) & ~np.isfinite(pv_prediction)
257         pv_prediction[original_only] = pv_original[original_only]
258         ridge_only = np.isfinite(pv_ridge) & ~np.isfinite(pv_prediction)
259         pv_prediction[ridge_only] = pv_ridge[ridge_only]
260
261         return (
262             load_dates,
263             fill_initial_missing(load_prediction, "1 月负载预测"),
264             fill_initial_missing(pv_prediction, "1 月光伏预测"),
265         )
266
267
268     def interval_label(period: int) -> str:
269         start = period * 10
270         end = (period + 1) * 10
271
272     def clock(minutes: int) -> str:

```

```

273     day, minute_of_day = divmod(minutes, 24 * 60)
274     hour, minute = divmod(minute_of_day, 60)
275     suffix = "+1" if day else ""
276     return f"{hour}:{minute:02d}{suffix}"
277
278     return f"{clock(start)}--{clock(end)}"
279
280
281     def solve_plan_day(
282         net_plan_kwh: np.ndarray,
283         price: np.ndarray,
284         storage_initial: float,
285     ) -> DailyPlan:
286         """ 固定计划净负荷下的日 MILP, 保证日初日末储电量相同。 """
287         n = PERIODS_PER_DAY
288         blocks = {
289             "g": slice(0, n),
290             "c": slice(n, 2 * n),
291             "d": slice(2 * n, 3 * n),
292             "S": slice(3 * n, 4 * n + 1),
293             "r": slice(4 * n + 1, 5 * n + 1),
294             "z": slice(5 * n + 1, 6 * n + 1),
295         }
296         size = 6 * n + 1
297
298         objective = np.zeros(size)
299         objective[blocks["g"]] = price
300         integrality = np.zeros(size, dtype=int)
301         integrality[blocks["z"]] = 1
302
303         lower = np.zeros(size)
304         upper = np.full(size, np.inf)
305         upper[blocks["c"]] = POWER_LIMIT
306         upper[blocks["d"]] = POWER_LIMIT
307         upper[blocks["z"]] = 1
308         lower[blocks["S"]] = SOC_MIN
309         upper[blocks["S"]] = SOC_MAX
310         lower[blocks["S"].start] = upper[blocks["S"].start] = storage_initial
311         lower[blocks["S"].stop - 1] = upper[blocks["S"].stop - 1] = storage_initial
312
313         matrix = lil_matrix((4 * n, size), dtype=float)
314         row_lower = np.full(4 * n, -np.inf)
315         row_upper = np.zeros(4 * n)
316         for t in range(n):
317             g = blocks["g"].start + t
318             c = blocks["c"].start + t
319             d = blocks["d"].start + t
320             s = blocks["S"].start + t
321             r = blocks["r"].start + t
322             z = blocks["z"].start + t

```

```

323
324     #  $g_t + d_t - c_t - spill_t = net\_plan_t$ 
325     matrix[t, g], matrix[t, d] = 1, 1
326     matrix[t, c], matrix[t, r] = -1, -1
327     row_lower[t] = row_upper[t] = net_plan_kwh[t]
328
329     row = n + t
330     matrix[row, s + 1], matrix[row, s] = 1, -1
331     matrix[row, c], matrix[row, d] = -ETA_C, 1 / ETA_D
332     row_lower[row] = row_upper[row] = 0
333
334     matrix[2 * n + t, c], matrix[2 * n + t, z] = 1, -POWER_LIMIT
335     matrix[3 * n + t, d], matrix[3 * n + t, z] = 1, POWER_LIMIT
336     row_upper[3 * n + t] = POWER_LIMIT
337
338     result = milp(
339         objective,
340         integrality=integrality,
341         bounds=Bounds(lower, upper),
342         constraints=LinearConstraint(matrix.tocsc(), row_lower, row_upper),
343         options={"mip_rel_gap": 0.0},
344     )
345     if result.x is None or result.status != 0:
346         raise RuntimeError(f"第二问单日 MILP 求解失败: {result.message}")
347
348     return DailyPlan(
349         purchase_kwh=result.x[blocks["g"]],
350         charge_kwh=result.x[blocks["c"]],
351         discharge_kwh=result.x[blocks["d"]],
352         storage_end_kwh=result.x[blocks["S"]][1:],
353     )
354
355
356     def simulate_actual_day(
357         planned_purchase: np.ndarray,
358         actual_load_kw: np.ndarray,
359         actual_pv_kw: np.ndarray,
360         storage_initial: float,
361     ) -> ActualResult:
362         """ 按实际供需执行电池：富余优先充电，缺电优先放电。 """
363         charge = np.zeros(PERIODS_PER_DAY)
364         discharge = np.zeros(PERIODS_PER_DAY)
365         emergency = np.zeros(PERIODS_PER_DAY)
366         spill = np.zeros(PERIODS_PER_DAY)
367         storage_end = np.zeros(PERIODS_PER_DAY)
368         storage = storage_initial
369
370         for t in range(PERIODS_PER_DAY):
371             load = actual_load_kw[t] * DT
372             pv = actual_pv_kw[t] * DT

```

```

373     balance = planned_purchase[t] + pv - load
374
375     if balance >= 0:
376         charge[t] = min(
377             balance,
378             POWER_LIMIT,
379             (SOC_MAX - storage) / ETA_C,
380         )
381     spill[t] = max(0.0, balance - charge[t])
382     else:
383         max_discharge = min(
384             -balance,
385             POWER_LIMIT,
386             ETA_D * (storage - SOC_MIN),
387         )
388     discharge[t] = max(0.0, max_discharge)
389     emergency[t] = max(0.0, -balance - discharge[t])
390
391     storage += ETA_C * charge[t] - discharge[t] / ETA_D
392     storage = min(SOC_MAX, max(SOC_MIN, storage))
393     storage_end[t] = storage
394
395     runs: list[tuple[str, float]] = []
396     active = np.flatnonzero(emergency > 1e-6)
397     if len(active):
398         start = previous = int(active[0])
399         for value in active[1:]:
400             period = int(value)
401             if period != previous + 1:
402                 runs.append(
403                     (
404                         f"{interval_label(start).split('-')[0]}-{"
405                         f"{interval_label(previous).split('-')[1]}",
406                         float(emergency[start:previous + 1].sum()),
407                     )
408                 )
409             start = period
410             previous = period
411             runs.append(
412                 (
413                     f"{interval_label(start).split('-')[0]}-{"
414                     f"{interval_label(previous).split('-')[1]}",
415                     float(emergency[start:previous + 1].sum()),
416                 )
417             )
418
419     return ActualResult(
420         storage_initial_kwh=float(storage_initial),
421         charge_kwh=charge,
422         discharge_kwh=discharge,

```

```

423     emergency_kwh=emergency,
424     spill_kwh=spill,
425     storage_end_kwh=storage_end,
426     emergency_runs=runs,
427 )
428
429
430 def style_sheet(worksheet) -> None:
431     header_fill = PatternFill("solid", fgColor="D9EAF7")
432     for cell in worksheet[1]:
433         cell.fill = header_fill
434         cell.font = Font(name="宋体", size=10, bold=True)
435         cell.alignment = Alignment(horizontal="center", vertical="center")
436     worksheet.freeze_panes = "B2"
437
438
439 def write_result2(
440     path: Path,
441     output_dates: list[date],
442     price: np.ndarray,
443     plans: list[DailyPlan],
444     actuals: list[ActualResult],
445 ) -> None:
446     workbook = openpyxl.Workbook()
447     workbook.remove(workbook.active)
448
449     plan_sheet = workbook.create_sheet("计划购电量")
450     plan_sheet.append(
451         ["日期\\时间", *[interval_label(t) for t in range(PERIODS_PER_DAY)],
452         "全天购电量", "全天购电费"]
453     )
454     for target, plan in zip(output_dates, plans):
455         plan_cost = float(price @ plan.purchase_kwh)
456         plan_sheet.append(
457             [datetime.combine(target, datetime.min.time()),
458             *plan.purchase_kwh.tolist(), float(plan.purchase_kwh.sum()), plan_cost]
459         )
460     for row in plan_sheet.iter_rows(min_row=2):
461         row[0].number_format = "yyyy-mm-dd"
462         for cell in row[1:]:
463             cell.number_format = "0.0000"
464
465     battery_sheet = workbook.create_sheet("充放电量")
466     battery_sheet.append(["日期", "时间段", "充电量", "放电量", "时刻", "储电量"])
467     for target, actual in zip(output_dates, actuals):
468         for block in range(6):
469             lo, hi = block * 24, (block + 1) * 24
470             battery_sheet.append(
471                 [
472                     datetime.combine(target, datetime.min.time()),

```

```

473     f"{interval_label(lo).split('-')[0]}-"
474     f"{interval_label(hi - 1).split('-')[1]}",
475     float(actual.charge_kwh[lo:hi].sum()),
476     float(actual.discharge_kwh[lo:hi].sum()),
477     "0:00" if block == 0 else ("24:00" if block == 1 else None),
478     (
479         float(
480             actual.storage_initial_kwh
481             if block == 0
482             else actual.storage_end_kwh[-1]
483         )
484         if block in (0, 1)
485         else None
486     ),
487     ]
488     )
489
490     emergency_sheet = workbook.create_sheet("紧急购电量")
491     emergency_sheet.append(["日期", "购电时间段", "购电量"])
492     for target, actual in zip(output_dates, actuals):
493         if not actual.emergency_runs:
494             emergency_sheet.append([datetime.combine(target, datetime.min.time()), None, None])
495             continue
496         for index, (interval, amount) in enumerate(actual.emergency_runs):
497             emergency_sheet.append(
498                 [
499                     datetime.combine(target, datetime.min.time()) if index == 0 else None,
500                     interval,
501                     amount,
502                 ]
503             )
504
505         for worksheet in workbook.worksheets:
506             style_sheet(worksheet)
507             worksheet.column_dimensions["A"].width = 13
508             for column in range(2, worksheet.max_column + 1):
509                 worksheet.column_dimensions[get_column_letter(column)].width = 11
510
511     path.parent.mkdir(parents=True, exist_ok=True)
512     workbook.save(path)
513
514
515     def main() -> None:
516         parser = argparse.ArgumentParser(description=__doc__)
517         parser.add_argument("--pred", type=Path)
518         parser.add_argument("--output", type=Path)
519         parser.add_argument(
520             "--evaluation-output",
521             type=Path,
522             default=RESULT_DIR / "model_R2_evaluation.csv",

```

```

523     )
524     args = parser.parse_args()
525
526     pred_path = args.pred or (RESULT_DIR / "prediction" / "prediction.xlsx")
527     output_path = args.output or (RESULT_DIR / "result2.xlsx")
528     price = load_price(APPENDIX_DIR / "附件 1.xlsx")
529     actual_dates, actual_load, actual_pv = load_actual_series(
530         APPENDIX_DIR / "附件 2.xlsx"
531     )
532     pred_dates, pred_load, pred_pv = load_prediction_series(pred_path)
533
534     actual_by_date = {
535         target: index for index, target in enumerate(actual_dates)
536     }
537     if any(target not in actual_by_date for target in pred_dates):
538         raise ValueError("pred.xlsx 中存在附件 2 没有的日期")
539     evaluation = evaluate_predictions(
540         actual_dates,
541         actual_load,
542         actual_pv,
543         pred_dates,
544         pred_load,
545         pred_pv,
546     )
547     write_evaluation_csv(args.evaluation_output, evaluation)
548
549     start_time = perf_counter()
550     jan_dates, jan_load_pred, jan_pv_pred = build_january_forecasts(
551         APPENDIX_DIR / "附件 2.xlsx"
552     )
553     jan_actual_indices = [actual_by_date[target] for target in jan_dates]
554     jan_actual_net = (
555         actual_load[jan_actual_indices] - actual_pv[jan_actual_indices]
556     ) * DT
557     jan_pred_net = (jan_load_pred - jan_pv_pred) * DT
558     residual_history = list(jan_actual_net - jan_pred_net)
559
560     storage = SOC_INITIAL
561     output_dates: list[date] = []
562     plans: list[DailyPlan] = []
563     actuals: list[ActualResult] = []
564
565     for target in actual_dates:
566         actual_index = actual_by_date[target]
567         if target in jan_dates:
568             pred_index = jan_dates.index(target)
569             forecast_net = (jan_load_pred[pred_index] - jan_pv_pred[pred_index]) * DT
570             elif OUTPUT_START <= target <= OUTPUT_END:
571                 pred_index = pred_dates.index(target)
572                 forecast_net = (pred_load[pred_index] - pred_pv[pred_index]) * DT

```

```

573     else:
574         continue
575
576     recent_residuals = np.asarray(
577         residual_history[-RESIDUAL_WINDOW_DAYS:], dtype=float
578     )
579     risk_margin = np.quantile(
580         recent_residuals, RISK_QUANTILE, axis=0, method="linear"
581     )
582     net_plan = forecast_net + risk_margin
583     plan = solve_plan_day(net_plan, price, storage)
584     actual = simulate_actual_day(
585         plan.purchase_kwh,
586         actual_load[actual_index],
587         actual_pv[actual_index],
588         storage,
589     )
590     storage = float(actual.storage_end_kwh[-1])
591
592     actual_net_today = (
593         actual_load[actual_index] - actual_pv[actual_index]
594     ) * DT
595     residual_history.append(actual_net_today - forecast_net)
596
597     if OUTPUT_START <= target <= OUTPUT_END:
598         output_dates.append(target)
599         plans.append(plan)
600         actuals.append(actual)
601
602     if output_dates != pred_dates:
603         raise ValueError("输出日期与 pred.xlsx 日期不一致")
604
605     write_result2(output_path, output_dates, price, plans, actuals)
606     total_plan_kwh = sum(float(plan.purchase_kwh.sum()) for plan in plans)
607     total_plan_cost = sum(float(price @ plan.purchase_kwh) for plan in plans)
608     total_emergency = sum(float(actual.emergency_kwh.sum()) for actual in actuals)
609     total_charge = sum(float(actual.charge_kwh.sum()) for actual in actuals)
610     total_discharge = sum(float(actual.discharge_kwh.sum()) for actual in actuals)
611     total_emergency_cost = sum(
612         float(price @ actual.emergency_kwh) * 5 for actual in actuals
613     )
614     print(f"求解日期: {output_dates[0]} 至 {output_dates[-1]}")
615     print(f"计划购电量: {total_plan_kwh:.6f} kWh")
616     print(f"计划购电费: {total_plan_cost:.6f} 元")
617     print(f"实际充电量: {total_charge:.6f} kWh")
618     print(f"实际放电量: {total_discharge:.6f} kWh")
619     print(f"紧急购电量: {total_emergency:.6f} kWh")
620     print(f"紧急购电费: {total_emergency_cost:.6f} 元")
621     print(f"总费用: {total_plan_cost + total_emergency_cost:.6f} 元")
622     print(f"期末储电量: {storage:.6f} kWh")

```

```

623     print(f"总耗时: {perf_counter() - start_time:.3f} 秒")
624     print(f"结果已保存: {output_path}")
625     print(f"预测评估已保存: {args.evaluation_output}")
626     print_evaluation(evaluation)
627
628
629     if __name__ == "__main__":
630         main()
631

```

问题 3 相关代码

```

1
2     from __future__ import annotations
3
4     import argparse
5     from dataclasses import dataclass
6     from datetime import date, datetime
7     from pathlib import Path
8     from time import perf_counter
9
10    import numpy as np
11    import openpyxl
12    from openpyxl.styles import Alignment, Font, PatternFill
13    from openpyxl.utils import get_column_letter
14    from scipy.optimize import Bounds, LinearConstraint, milp
15    from scipy.sparse import lil_matrix
16
17    ROOT = Path(__file__).resolve().parents[2]
18    RESULT_DIR = ROOT / "result" / "problem03"
19    APPENDIX_DIR = ROOT / "附件"
20    PRED_PATH = ROOT / "result" / "problem02" / "prediction" / "prediction.xlsx"
21
22    from src.problem02 import problem
23
24    PERIODS_PER_DAY = 144
25    DT = 1 / 6
26    ETA_C = 0.9
27    ETA_D = 0.9
28    SOC_MIN = 1200.0
29    SOC_MAX = 10800.0
30    SOC_INITIAL = 6000.0
31    SOC_TARGET = 6000.0
32    POWER_LIMIT = 5000.0 * DT
33    DECISION_HOURS = (0, 6, 12, 18)
34    RESIDUAL_WINDOW_DAYS = 60
35    RISK_QUANTILE_OVERNIGHT = 0.60
36    RISK_QUANTILE_DAY = 0.60

```

```

37     OVERNIGHT_SLOTS = 36
38     PV_CORRECT = True
39     PV_SHAPE_WINDOW = 60
40     PV_SHAPE_MIN_HOURLY = 150.0
41     PV_MIN_PAIRS = 15
42     PV_NIGHT_RATIO = 0.05
43     OUTPUT_START = date(2025, 2, 1)
44     OUTPUT_END = date(2025, 12, 31)
45
46
47     @dataclass(frozen=True)
48     class IntervalPlan:
49         purchase_kwh: np.ndarray
50         charge_kwh: np.ndarray
51         discharge_kwh: np.ndarray
52         storage_end_kwh: np.ndarray
53
54
55     @dataclass(frozen=True)
56     class DailyResult:
57         target_date: date
58         plan_purchase: np.ndarray
59         adjusted_purchase: np.ndarray
60         charge: np.ndarray
61         discharge: np.ndarray
62         emergency: np.ndarray
63         spill: np.ndarray
64         storage_initial: float
65         storage_end: np.ndarray
66         plan_cost: float
67         adjustment_cost: float
68         emergency_cost: float
69
70     def interval_label(period: int) -> str:
71         start = period * 10
72         end = (period + 1) * 10
73
74     def clock(minutes: int) -> str:
75         day, minute_of_day = divmod(minutes, 24 * 60)
76         hour, minute = divmod(minute_of_day, 60)
77         suffix = "+1" if day else ""
78         return f"{hour}:{minute:02d}{suffix}"
79
80     return f"{clock(start)}--{clock(end)}"
81
82
83     def parse_date(value) -> date:
84         if isinstance(value, datetime):
85             return value.date()
86         if isinstance(value, date):

```

```

87     return value
88     text = str(value).strip()
89     parts = text.split("-")
90     if len(parts) != 3:
91         raise ValueError(f"无法识别日期: {value!r}")
92     year, month, day = map(int, parts)
93     return date(year, month, day)
94
95
96     def load_forecast_inputs(
97         attachment2: Path, pred_path: Path
98     ) -> tuple[list[date], dict[date, np.ndarray]]:
99         """ 返回全年日期及各日期可用于 0:00 决策的负载预测。 """
100         jan_dates, jan_load, _ = problem.build_january_forecasts(attachment2)
101         pred_dates, pred_load, _ = problem.load_prediction_series(pred_path)
102
103         forecasts = {
104             target: jan_load[index].copy()
105             for index, target in enumerate(jan_dates)
106             if target.month == 1
107         }
108         forecasts.update(
109             {
110                 target: pred_load[index].copy()
111                 for index, target in enumerate(pred_dates)
112             }
113         )
114         actual_dates, _, _ = problem.load_actual_series(attachment2)
115         missing = [target for target in actual_dates if target not in forecasts]
116         if missing:
117             raise ValueError(f"缺少负载预测日期: {missing[:5]}")
118         return actual_dates, forecasts
119
120
121     def load_pv_forecasts(
122         path: Path,
123     ) -> dict[tuple[date, int], np.ndarray]:
124         """ 读取附件 3 的四版小时级光伏预报。 """
125         workbook = openpyxl.load_workbook(path, read_only=True, data_only=True)
126         try:
127             worksheet = workbook.worksheets[0]
128             rows = list(worksheet.iter_rows(values_only=True))
129         finally:
130             workbook.close()
131
132         if tuple(rows[0][:2]) != ("日期", "预报时刻") or len(rows[0]) != 26:
133             raise ValueError("附件 3 表头格式不正确")
134
135         result: dict[tuple[date, int], np.ndarray] = {}
136         current_date = None

```

```

137     for row in rows[1:]:
138         if row[0] not in (None, ""):
139             current_date = parse_date(row[0])
140             if current_date is None or row[1] is None:
141                 continue
142             hour, minute = map(int, str(row[1]).strip().split(":")[:2])
143             if minute != 0 or hour not in DECISION_HOURS:
144                 raise ValueError(f"无法识别附件 3 预报时刻: {row[1]!r}")
145             values = np.asarray(row[2:26], dtype=float)
146             if not np.isfinite(values).all():
147                 raise ValueError(f"{current_date} {row[1]} 光伏预报存在缺失值")
148             result[(current_date, hour)] = np.maximum(0.0, values)
149
150         if len(result) != 365 * 4:
151             raise ValueError(f"附件 3 应有 1460 组预报, 实际为 {len(result)} 组")
152         return result
153
154
155     def expand_hourly_step(values: np.ndarray, decision_hour: int) -> np.ndarray:
156         """ 小时预报分段常数展开, 保留为旧口径的对照和回退。"""
157         result = np.zeros(PERIODS_PER_DAY)
158         start = decision_hour * 6
159         for horizon, value in enumerate(values, start=1):
160             lo = start + 6 * (horizon - 1)
161             hi = start + 6 * horizon
162             if lo >= PERIODS_PER_DAY:
163                 break
164             result[lo:min(hi, PERIODS_PER_DAY)] = value
165         return result
166
167
168     class PvForecastCorrector:
169         """ 用目标日之前的历史实际光伏, 订正附件 3 的小时预报并还原小时内形状。
170
171         两步都只使用 history 里已经发生的数据, 不含任何前视信息:
172
173         1) 水平订正: 对每个 (发布时刻 tau, 目标整点 H), 用过去 PV_SHAPE_WINDOW 天的
174         (附件 3 预报值, 实际小时均值) 拟合一元回归, 得到修正系数 (截距, 斜率)。
175         夜间或预报值极低 ( $< PV\_NIGHT\_RATIO * \text{历史均值}$ ) 时不做回归, 直接用原值。
176         2) 小时内形状: 统计同一窗口内每个整点的 10 分钟形状比 (该槽值 / 该小时均值),
177         均值归一为 1 后乘到订正后的整点值上; 形状均值恒为 1, 因此小时电量不变,
178         只是把整点值更合理地摊到 6 个 10 分钟槽 (还原早晨爬坡、傍晚回落)。
179         """
180
181         def __init__(
182             self,
183             forecasts: dict,
184             window: int = PV_SHAPE_WINDOW,
185             min_hourly: float = PV_SHAPE_MIN_HOURLY,
186             min_pairs: int = PV_MIN_PAIRS,

```

```

187     use_level: bool = True,
188     use_shape: bool = True,
189     ) -> None:
190         self.forecasts = forecasts
191         self.window = window
192         self.min_hourly = min_hourly
193         self.min_pairs = min_pairs
194         self.use_level = use_level
195         self.use_shape = use_shape
196         self.actual: dict = {}
197         self.night_scale: dict = {}
198         self.regression: dict = {}
199         self.shape: dict = {}
200
201     def observe(self, target: date, actual_pv: np.ndarray) -> None:
202         """ 登记某日实际光伏；只用于该日之后的目标日。 """
203         self.actual[target] = np.asarray(actual_pv, dtype=float)
204
205     def history(self, target: date) -> list:
206         days = [day for day in sorted(self.actual) if day < target]
207         return days[-self.window:]
208
209     def fit(self, target: date) -> None:
210         """ 用 target 之前的历史重新拟合水平订正与形状系数。 """
211         days = self.history(target)
212         self.regression, self.shape, self.night_scale = {}, {}, {}
213         if not days:
214             return
215         buckets: dict = {}
216         for day in days:
217             pv = self.actual[day]
218             for hour in range(24):
219                 segment = pv[6 * hour:6 * hour + 6]
220                 mean = float(segment.mean())
221                 if mean >= self.min_hourly:
222                     buckets.setdefault(hour, []).append(segment / mean)
223             for hour, rows in buckets.items():
224                 shape = np.asarray(rows).mean(axis=0)
225                 if shape.mean() > 1e-9:
226                     self.shape[hour] = shape / shape.mean()
227                 if not self.use_level:
228                     return
229             for tau in DECISION_HOURS:
230                 for hour in range(24):
231                     offset = hour - tau
232                     if not 0 <= offset < 24:
233                         continue
234                     xs, ys = [], []
235                     for day in days:
236                         values = self.forecasts.get((day, tau))

```

```

237     if values is None:
238         continue
239     xs.append(float(values[offset]))
240     ys.append(float(self.actual[day][6 * hour:6 * hour + 6].mean()))
241     if len(xs) < self.min_pairs:
242         continue
243     x = np.asarray(xs, dtype=float)
244     if x.std() < 1e-6:
245         continue
246     slope, intercept = np.polyfit(x, np.asarray(ys, dtype=float), 1)
247     if slope < 0.0:
248         slope, intercept = 1.0, 0.0
249     self.regression[(tau, hour)] = (
250         float(intercept),
251         float(slope),
252         float(x.mean()),
253     )
254
255     def expand(self, values: np.ndarray, decision_hour: int, target: date) -> np.ndarray:
256         """ 把附件 3 的 24 个整点值订正并展开成当天 144 个 10 分钟值。 """
257         result = np.zeros(PERIODS_PER_DAY)
258         start = decision_hour * 6
259         for offset, value in enumerate(values):
260             lo = start + 6 * offset
261             if lo >= PERIODS_PER_DAY:
262                 break
263             hi = min(lo + 6, PERIODS_PER_DAY)
264             hour = (decision_hour + offset) % 24
265             level = float(value)
266             if self.use_level:
267                 coef = self.regression.get((decision_hour, hour))
268                 if coef is not None:
269                     intercept, slope, mean_forecast = coef
270                     if mean_forecast > 0 and level >= PV_NIGHT_RATIO * mean_forecast:
271                         level = max(0.0, intercept + slope * level)
272                 shape = self.shape.get(hour) if self.use_shape else None
273                 if shape is None:
274                     result[lo:hi] = level
275                 else:
276                     result[lo:hi] = level * shape[:hi - lo]
277             return np.maximum(result, 0.0)
278
279
280     def solve_interval_milp(
281         net_demand_kwh: np.ndarray,
282         price: np.ndarray,
283         storage_initial: float,
284         *,
285         reference_purchase: np.ndarray | None = None,
286     ) -> IntervalPlan:

```

```

287     """ 求解计划或调整阶段的储能 MILP。 """
288     n = len(net_demand_kwh)
289     adjustment = reference_purchase is not None
290     adjustment_blocks = {}
291     if adjustment:
292         adjustment_blocks = {
293             "up": slice(7 * n + 1, 8 * n + 1),
294             "down": slice(8 * n + 1, 9 * n + 1),
295         }
296     blocks = {
297         "g": slice(0, n),
298         "c": slice(n, 2 * n),
299         "d": slice(2 * n, 3 * n),
300         "S": slice(3 * n, 4 * n + 1),
301         "u": slice(4 * n + 1, 5 * n + 1),
302         "z": slice(5 * n + 1, 6 * n + 1),
303         "short": slice(6 * n + 1, 7 * n + 1),
304     }
305     blocks.update(adjustment_blocks)
306     size = 9 * n + 1 if adjustment else 7 * n + 1
307
308     objective = np.zeros(size)
309     if adjustment:
310         objective[blocks["up"]] = 1.5 * price
311         objective[blocks["down"]] = 0.5 * price
312     else:
313         objective[blocks["g"]] = price
314         terminal_penalty = 5.0 * float(np.mean(price))
315         objective[blocks["short"]] = terminal_penalty
316
317     integrality = np.zeros(size, dtype=int)
318     integrality[blocks["z"]] = 1
319
320     lower = np.zeros(size)
321     upper = np.full(size, np.inf)
322     upper[blocks["c"]] = POWER_LIMIT
323     upper[blocks["d"]] = POWER_LIMIT
324     upper[blocks["z"]] = 1
325     lower[blocks["S"]] = SOC_MIN
326     upper[blocks["S"]] = SOC_MAX
327     lower[blocks["S"].start] = upper[blocks["S"].start] = storage_initial
328
329     row_count = 5 * n + 1 if adjustment else 4 * n + 1
330     matrix = lil_matrix((row_count, size), dtype=float)
331     row_lower = np.full(row_count, -np.inf)
332     row_upper = np.zeros(row_count)
333
334     for t in range(n):
335         g = blocks["g"].start + t
336         c = blocks["c"].start + t

```

```

337     d = blocks["d"].start + t
338     s = blocks["S"].start + t
339     u = blocks["u"].start + t
340     z = blocks["z"].start + t
341
342     matrix[t, g], matrix[t, d] = 1, 1
343     matrix[t, c], matrix[t, u] = -1, -1
344     row_lower[t] = row_upper[t] = net_demand_kwh[t]
345
346     row = n + t
347     matrix[row, s + 1], matrix[row, s] = 1, -1
348     matrix[row, c], matrix[row, d] = -ETA_C, 1 / ETA_D
349     row_lower[row] = row_upper[row] = 0
350
351     matrix[2 * n + t, c], matrix[2 * n + t, z] = 1, -POWER_LIMIT
352     matrix[3 * n + t, d], matrix[3 * n + t, z] = 1, POWER_LIMIT
353     row_upper[3 * n + t] = POWER_LIMIT
354
355     if adjustment:
356         up = blocks["up"].start + t
357         down = blocks["down"].start + t
358         matrix[4 * n + t, g] = 1
359         matrix[4 * n + t, up], matrix[4 * n + t, down] = -1, 1
360         row_lower[4 * n + t] = row_upper[4 * n + t] = (
361             float(reference_purchase[t])
362         )
363
364     terminal_row = row_count - 1
365     matrix[terminal_row, blocks["S"].stop - 1] = 1
366     matrix[terminal_row, blocks["short"].start] = 1
367     row_lower[terminal_row] = SOC_TARGET
368     row_upper[terminal_row] = np.inf
369
370     result = milp(
371         objective,
372         integrality=integrality,
373         bounds=Bounds(lower, upper),
374         constraints=LinearConstraint(matrix.tocsc(), row_lower, row_upper),
375         options={"mip_rel_gap": 0.0},
376     )
377     if result.x is None or result.status != 0:
378         raise RuntimeError(f"第三问区间 MILP 求解失败: {result.message}")
379
380     return IntervalPlan(
381         purchase_kwh=result.x[blocks["g"]],
382         charge_kwh=result.x[blocks["c"]],
383         discharge_kwh=result.x[blocks["d"]],
384         storage_end_kwh=result.x[blocks["S"]][1:],
385     )
386

```

```

387
388     def simulate_segment(
389         purchase_kwh: np.ndarray,
390         actual_load_kw: np.ndarray,
391         actual_pv_kw: np.ndarray,
392         storage_initial: float,
393     ) -> tuple[np.ndarray, np.ndarray, np.ndarray, np.ndarray, float]:
394         """ 按实际负载和实际光伏运行一个连续时段区间。 """
395         n = len(purchase_kwh)
396         charge = np.zeros(n)
397         discharge = np.zeros(n)
398         emergency = np.zeros(n)
399         spill = np.zeros(n)
400         storage_end = np.zeros(n)
401         storage = float(storage_initial)
402
403         for t in range(n):
404             balance = (
405                 purchase_kwh[t]
406                 + actual_pv_kw[t] * DT
407                 - actual_load_kw[t] * DT
408             )
409             if balance >= 0:
410                 charge[t] = min(
411                     balance,
412                     POWER_LIMIT,
413                     max(0.0, (SOC_MAX - storage) / ETA_C),
414                 )
415                 spill[t] = max(0.0, balance - charge[t])
416             else:
417                 discharge[t] = min(
418                     -balance,
419                     POWER_LIMIT,
420                     max(0.0, ETA_D * (storage - SOC_MIN)),
421                 )
422                 emergency[t] = max(0.0, -balance - discharge[t])
423                 storage += ETA_C * charge[t] - discharge[t] / ETA_D
424                 storage = min(SOC_MAX, max(SOC_MIN, storage))
425                 storage_end[t] = storage
426
427         return charge, discharge, emergency, spill, storage
428
429
430     def updated_load_forecast(
431         base_load: np.ndarray,
432         actual_load: np.ndarray,
433         decision_index: int,
434     ) -> np.ndarray:
435         """ 利用当天已发生的负载对剩余时段做水平偏差修正。 """
436         if decision_index <= 0:

```

```

437     return base_load.copy()
438     observed_error = (
439         actual_load[:, decision_index] - base_load[:, decision_index]
440     )
441     correction = float(np.mean(observed_error))
442     scale = max(float(np.mean(base_load[decision_index:])), 1.0)
443     correction = float(np.clip(correction, -0.2 * scale, 0.2 * scale))
444     result = base_load.copy()
445     result[decision_index:] = np.maximum(
446         0.0, base_load[decision_index:] + correction
447     )
448     return result
449
450
451     def solve_day(
452         target: date,
453         price: np.ndarray,
454         base_load: np.ndarray,
455         corrector: PvForecastCorrector,
456         actual_load: np.ndarray,
457         actual_pv: np.ndarray,
458         storage_initial: float,
459         risk_margin: np.ndarray,
460     ) -> DailyResult:
461         """ 完成一天的计划、三次调整 and 实际运行。 """
462         pv0 = corrector.expand(corrector.forecasts[(target, 0)], 0, target)
463         forecast_net0 = (base_load - pv0) * DT
464         plan_net = forecast_net0 + risk_margin
465         plan = solve_interval_milp(plan_net, price, storage_initial)
466
467         final_purchase = plan.purchase_kwh.copy()
468         actual_charge = np.zeros(PERIODS_PER_DAY)
469         actual_discharge = np.zeros(PERIODS_PER_DAY)
470         actual_emergency = np.zeros(PERIODS_PER_DAY)
471         actual_spill = np.zeros(PERIODS_PER_DAY)
472         storage_trace = np.zeros(PERIODS_PER_DAY)
473         storage = float(storage_initial)
474         cursor = 0
475
476         for decision_hour in (6, 12, 18):
477             decision_index = decision_hour * 6
478             segment_start = storage
479             (
480                 charge_segment,
481                 discharge_segment,
482                 emergency_segment,
483                 spill_segment,
484                 storage,
485             ) = simulate_segment(
486                 final_purchase[cursor:decision_index],

```

```

487     actual_load[cursor:decision_index],
488     actual_pv[cursor:decision_index],
489     storage,
490 )
491     actual_charge[cursor:decision_index] = charge_segment
492     actual_discharge[cursor:decision_index] = discharge_segment
493     actual_emergency[cursor:decision_index] = emergency_segment
494     actual_spill[cursor:decision_index] = spill_segment
495     storage_trace[cursor:decision_index] = (
496     segment_start
497     + np.cumsum(
498     ETA_C * charge_segment - discharge_segment / ETA_D
499     )
500     )
501     cursor = decision_index
502
503     updated_load = updated_load_forecast(
504     base_load, actual_load, decision_index
505     )
506     pv_current = corrector.expand(
507     corrector.forecasts[(target, decision_hour)], decision_hour, target
508     )
509     forecast_net = (updated_load - pv_current) * DT
510     future_net = (
511     forecast_net[decision_index:] + risk_margin[decision_index:]
512     )
513     adjusted = solve_interval_milp(
514     future_net,
515     price[decision_index:],
516     storage,
517     reference_purchase=plan.purchase_kwh[decision_index:],
518     )
519     final_purchase[decision_index:] = adjusted.purchase_kwh
520
521     segment_start = storage
522     (
523     charge_segment,
524     discharge_segment,
525     emergency_segment,
526     spill_segment,
527     storage,
528     ) = simulate_segment(
529     final_purchase[cursor:],
530     actual_load[cursor:],
531     actual_pv[cursor:],
532     storage,
533     )
534     actual_charge[cursor:] = charge_segment
535     actual_discharge[cursor:] = discharge_segment
536     actual_emergency[cursor:] = emergency_segment

```

```

537     actual_spill[cursor:] = spill_segment
538     storage_trace[cursor:] = (
539         segment_start
540         + np.cumsum(
541             ETA_C * charge_segment - discharge_segment / ETA_D
542         )
543     )
544
545     delta = final_purchase - plan.purchase_kwh
546     adjustment_cost = float(
547         np.sum(
548             1.5 * price * np.maximum(delta, 0.0)
549             + 0.5 * price * np.maximum(-delta, 0.0)
550         )
551     )
552
553     return DailyResult(
554         target_date=target,
555         plan_purchase=plan.purchase_kwh,
556         adjusted_purchase=final_purchase,
557         charge=actual_charge,
558         discharge=actual_discharge,
559         emergency=actual_emergency,
560         spill=actual_spill,
561         storage_initial=float(storage_initial),
562         storage_end=storage_trace,
563         plan_cost=float(price @ plan.purchase_kwh),
564         adjustment_cost=adjustment_cost,
565         emergency_cost=float(5 * price @ actual_emergency),
566     )
567
568
569     def merge_emergency_runs(
570         emergency: np.ndarray,
571     ) -> list[tuple[str, float]]:
572         active = np.flatnonzero(emergency > 1e-6)
573         runs = []
574         if not len(active):
575             return runs
576         start = previous = int(active[0])
577         for value in active[1:]:
578             period = int(value)
579             if period != previous + 1:
580                 runs.append(
581                     (
582                         f"{interval_label(start).split('-')[0]}-"
583                         f"{interval_label(previous).split('-')[1]}",
584                         float(emergency[start:previous + 1].sum()),
585                     )
586                 )

```

```

587     start = period
588     previous = period
589     runs.append(
590     (
591         f"{interval_label(start).split('-')[0]}-"
592         f"{interval_label(previous).split('-')[1]}",
593         float(emergency[start:previous + 1].sum()),
594     )
595     )
596     return runs
597
598
599     def write_result3(path: Path, results: list[DailyResult], price: np.ndarray) -> None:
600         workbook = openpyxl.Workbook()
601         workbook.remove(workbook.active)
602         time_headers = [interval_label(t) for t in range(PERIODS_PER_DAY)]
603
604         plan_sheet = workbook.create_sheet("计划购电量")
605         plan_sheet.append(["日期\\时间", *time_headers, "全天购电量", "全天购电费"])
606         for result in results:
607             plan_sheet.append(
608                 [
609                     datetime.combine(result.target_date, datetime.min.time()),
610                     *result.plan_purchase.tolist(),
611                     float(result.plan_purchase.sum()),
612                     result.plan_cost,
613                 ]
614             )
615
616         adjusted_sheet = workbook.create_sheet("调整购电量")
617         adjusted_sheet.append(["日期\\时间", *time_headers, "全天调整购电量", "全天调整费用"])
618         for result in results:
619             adjusted_sheet.append(
620                 [
621                     datetime.combine(result.target_date, datetime.min.time()),
622                     *result.adjusted_purchase.tolist(),
623                     float(result.adjusted_purchase.sum()),
624                     result.adjustment_cost,
625                 ]
626             )
627
628         battery_sheet = workbook.create_sheet("充放电电量")
629         battery_sheet.append(["日期", "时间段", "充电量", "放电量", "时刻", "储电量"])
630         for result in results:
631             for block in range(6):
632                 lo, hi = block * 24, (block + 1) * 24
633                 battery_sheet.append(
634                     [
635                         datetime.combine(result.target_date, datetime.min.time()),
636                         f"{interval_label(lo).split('-')[0]}-"

```

```

637     f"{interval_label(hi - 1).split('-')[1]}",
638     float(result.charge[lo:hi].sum()),
639     float(result.discharge[lo:hi].sum()),
640     "0:00" if block == 0 else ("24:00" if block == 1 else None),
641     (
642     result.storage_initial
643     if block == 0
644     else result.storage_end[-1]
645     )
646     if block in (0, 1)
647     else None,
648     ]
649     )
650
651     emergency_sheet = workbook.create_sheet("紧急购电量")
652     emergency_sheet.append(["日期", "购电时间段", "购电量"])
653     for result in results:
654         runs = merge_emergency_runs(result.emergency)
655         if not runs:
656             emergency_sheet.append(
657                 [datetime.combine(result.target_date, datetime.min.time()), None, None]
658             )
659             continue
660         for index, (label, amount) in enumerate(runs):
661             emergency_sheet.append(
662                 [
663                     datetime.combine(result.target_date, datetime.min.time())
664                     if index == 0
665                     else None,
666                     label,
667                     amount,
668                 ]
669             )
670
671     spill_sheet = workbook.create_sheet("弃电量")
672     spill_sheet.append(["日期", "时间段", "弃电量"])
673     for result in results:
674         for block in range(6):
675             lo, hi = block * 24, (block + 1) * 24
676             spill_sheet.append(
677                 [
678                     datetime.combine(result.target_date, datetime.min.time()),
679                     f"{interval_label(lo).split('-')[0]}-",
680                     f"{interval_label(hi - 1).split('-')[1]}",
681                     float(result.spill[lo:hi].sum()),
682                 ]
683             )
684
685     for worksheet in workbook.worksheets:
686         for cell in worksheet[1]:

```

```

687 cell.fill = PatternFill("solid", fgColor="D9EAF7")
688 cell.font = Font(name="宋体", size=10, bold=True)
689 cell.alignment = Alignment(horizontal="center", vertical="center")
690 worksheet.freeze_panes = "B2"
691 worksheet.column_dimensions["A"].width = 13
692 for column in range(2, worksheet.max_column + 1):
693     worksheet.column_dimensions[get_column_letter(column)].width = 11
694 for row in worksheet.iter_rows(min_row=2):
695     row[0].number_format = "yyyy-mm-dd"
696     for cell in row[1:]:
697         cell.number_format = "0.0000"
698
699 path.parent.mkdir(parents=True, exist_ok=True)
700 workbook.save(path)
701
702
703 def segmented_risk_margin(
704     residuals: np.ndarray,
705     quantile_overnight: float,
706     quantile_day: float,
707 ) -> np.ndarray:
708     """ 按“还有没有调整窗口”分段取残差分位。
709
710     0:00-6:00 这 36 个时段在 6:00 之前没有任何调整机会，缺电只能走 5 倍电价，
711     因此保留较高分位；6:00 之后的时段至少有 1 次 1.5 倍电价的调整窗口，
712     对冲的边际收益低得多，用较低分位，避免“计划买太多、最后弃掉”。
713
714     """
715     sample = np.atleast_2d(np.asarray(residuals, dtype=float))
716     margin = np.empty(PERIODS_PER_DAY)
717     margin[:OVERNIGHT_SLOTS] = np.quantile(
718         sample[:, :OVERNIGHT_SLOTS], quantile_overnight, axis=0, method="linear"
719     )
720     margin[OVERNIGHT_SLOTS:] = np.quantile(
721         sample[:, OVERNIGHT_SLOTS:], quantile_day, axis=0, method="linear"
722     )
723     return margin
724
725 _INPUT_CACHE: dict = {}
726
727
728 def load_inputs() -> dict:
729     """ 加载并缓存输入数据，便于搜索脚本在同一进程内多次调用。"""
730     if not _INPUT_CACHE:
731         price = problem.load_price(APPENDIX_DIR / "附件 1.xlsx")
732         actual_dates, actual_load, actual_pv = problem.load_actual_series(
733             APPENDIX_DIR / "附件 2.xlsx"
734         )
735         load_forecasts = load_forecast_inputs(
736             APPENDIX_DIR / "附件 2.xlsx",

```

```

737     PRED_PATH,
738     )[1]
739     pv_forecasts = load_pv_forecasts(APPENDIX_DIR / "附件 3.xlsx")
740     _INPUT_CACHE.update(
741         price=price,
742         actual_dates=actual_dates,
743         actual_load=actual_load,
744         actual_pv=actual_pv,
745         load_forecasts=load_forecasts,
746         pv_forecasts=pv_forecasts,
747     )
748     return _INPUT_CACHE
749
750
751     def run_simulation(
752         quantile_overnight: float = RISK_QUANTILE_OVERNIGHT,
753         quantile_day: float = RISK_QUANTILE_DAY,
754         shape_window: int = PV_SHAPE_WINDOW,
755         use_correction: bool = True,
756         collect_results: bool = False,
757     ) -> dict:
758         """ 整年滚动模拟，返回输出区间（OUTPUT_START~OUTPUT_END）的费用汇总。
759
760         裕度参数只影响 segmented_risk_margin 的分位取值，其余流程保持主模型口径不变。
761         完全一致，因此可以把它当成“给定裕度 → 总费用”的黑箱函数，反复调用。
762         """
763         inputs = load_inputs()
764         price = inputs["price"]
765         actual_dates = inputs["actual_dates"]
766         actual_load = inputs["actual_load"]
767         actual_pv = inputs["actual_pv"]
768         load_forecasts = inputs["load_forecasts"]
769         pv_forecasts = inputs["pv_forecasts"]
770
771         corrector = PvForecastCorrector(
772             pv_forecasts,
773             window=shape_window,
774             use_level=use_correction,
775             use_shape=use_correction,
776         )
777
778         prices_by_date = {target: price for target in actual_dates}
779         actual_by_date = {target: index for index, target in enumerate(actual_dates)}
780         residual_history: list[np.ndarray] = []
781         storage = SOC_INITIAL
782
783         days = 0
784         total_plan = 0.0
785         total_adjusted = 0.0
786         total_plan_cost = 0.0

```

```

787     total_adjustment_cost = 0.0
788     total_emergency = 0.0
789     total_emergency_cost = 0.0
790     total_spill = 0.0
791     total_charge = 0.0
792     total_discharge = 0.0
793     results = []
794
795     started = perf_counter()
796     for target in actual_dates:
797         if target not in load_forecasts:
798             raise ValueError(f"{target} 缺少负载预测")
799         corrector.fit(target)
800         risk_margin = (
801             segmented_risk_margin(
802                 np.asarray(residual_history[-RESIDUAL_WINDOW_DAYS:], dtype=float),
803                 quantile_overnight,
804                 quantile_day,
805             )
806         if residual_history
807         else np.zeros(PERIODS_PER_DAY)
808         )
809         actual_index = actual_by_date[target]
810         base_load = load_forecasts[target]
811         result = solve_day(
812             target,
813             prices_by_date[target],
814             base_load,
815             corrector,
816             actual_load[actual_index],
817             actual_pv[actual_index],
818             storage,
819             risk_margin,
820         )
821         storage = float(result.storage_end[-1])
822
823         forecast_pv0 = corrector.expand(pv_forecasts[(target, 0)], 0, target)
824         forecast_net0 = (base_load - forecast_pv0) * DT
825         actual_net = (
826             actual_load[actual_index] - actual_pv[actual_index]
827         ) * DT
828         residual_history.append(actual_net - forecast_net0)
829         corrector.observe(target, actual_pv[actual_index])
830
831         if OUTPUT_START <= target <= OUTPUT_END:
832             days += 1
833             total_plan += float(result.plan_purchase.sum())
834             total_adjusted += float(result.adjusted_purchase.sum())
835             total_plan_cost += result.plan_cost
836             total_adjustment_cost += result.adjustment_cost

```

```

837     total_emergency += float(result.emergency.sum())
838     total_emergency_cost += result.emergency_cost
839     total_spill += float(result.spill.sum())
840     total_charge += float(result.charge.sum())
841     total_discharge += float(result.discharge.sum())
842     if collect_results:
843         results.append(result)
844
845     summary = {
846         "days": days,
847         "plan_kwh": total_plan,
848         "adjusted_kwh": total_adjusted,
849         "plan_cost": total_plan_cost,
850         "adjustment_cost": total_adjustment_cost,
851         "emergency_kwh": total_emergency,
852         "emergency_cost": total_emergency_cost,
853         "spill_kwh": total_spill,
854         "charge_kwh": total_charge,
855         "discharge_kwh": total_discharge,
856         "total_cost": total_plan_cost + total_adjustment_cost + total_emergency_cost,
857         "storage_end": storage,
858         "quantile_overnight": float(quantile_overnight),
859         "quantile_day": float(quantile_day),
860         "shape_window": int(shape_window),
861         "use_correction": bool(use_correction),
862         "seconds": perf_counter() - started,
863     }
864     if collect_results:
865         summary["results"] = results
866     return summary
867
868
869     def main() -> None:
870         parser = argparse.ArgumentParser(description=__doc__)
871         parser.add_argument(
872             "--result3",
873             type=Path,
874             default=RESULT_DIR / "result3.xlsx",
875         )
876         parser.add_argument(
877             "--quantile-overnight",
878             type=float,
879             default=RISK_QUANTILE_OVERNIGHT,
880             help="0:00-6:00 的残差分位（无调整窗口，默认 %.2f）"
881             % RISK_QUANTILE_OVERNIGHT,
882         )
883         parser.add_argument(
884             "--quantile-day",
885             type=float,
886             default=RISK_QUANTILE_DAY,

```

```

887     help="6:00 之后的残差分位（有调整窗口，默认 %.2f）" % RISK_QUANTILE_DAY,
888 )
889     parser.add_argument(
890         "--shape-window",
891         type=int,
892         default=PV_SHAPE_WINDOW,
893         help="光伏订正使用的历史天数（默认 %d）" % PV_SHAPE_WINDOW,
894     )
895     parser.add_argument(
896         "--no-pv-correct",
897         action="store_true",
898         help="关闭历史订正，退回分段常数展开（用于对照）",
899     )
900     args = parser.parse_args()
901
902     use_correction = PV_CORRECT and not args.no_pv_correct
903     print(
904         "光伏展开: %s（历史窗口 %d 天）"
905         % (
906             "历史订正（水平回归 + 小时内形状）" if use_correction else "分段常数",
907             args.shape_window,
908         )
909     )
910     print(
911         "风险裕度分位: 0:00-6:00 = %.3f, 6:00 之后 = %.3f"
912         % (args.quantile_overnight, args.quantile_day)
913     )
914
915     summary = run_simulation(
916         quantile_overnight=args.quantile_overnight,
917         quantile_day=args.quantile_day,
918         shape_window=args.shape_window,
919         use_correction=use_correction,
920         collect_results=True,
921     )
922     results = summary["results"]
923     if not results or results[0].target_date != OUTPUT_START:
924         raise ValueError("没有生成完整的第三问结果")
925
926     write_result3(args.result3, results, load_inputs()["price"])
927
928     print(
929         f"求解日期: {results[0].target_date} 至 {results[-1].target_date}"
930     )
931     print(f"计划购电量: {summary['plan_kwh']:.6f} kWh")
932     print(f"调整后购电量: {summary['adjusted_kwh']:.6f} kWh")
933     print(f"计划购电费: {summary['plan_cost']:.6f} 元")
934     print(f"调整费用: {summary['adjustment_cost']:.6f} 元")
935     print(f"紧急购电量: {summary['emergency_kwh']:.6f} kWh")
936     print(f"紧急购电费: {summary['emergency_cost']:.6f} 元")

```

```

937     print(f"弃电量      : {summary['spill_kwh']:.6f} kWh (买来又扔掉的部分)")
938     print(
939         f"充电量      : {summary['charge_kwh']:.6f} kWh / "
940         f"放电量: {summary['discharge_kwh']:.6f} kWh"
941     )
942     print(f"总费用: {summary['total_cost']:.6f} 元")
943     print(f"期末储电量: {summary['storage_end']:.6f} kWh")
944     print(f"总耗时: {summary['seconds']:.3f} 秒")
945     print(f"结果已保存: {args.result3}")
946
947
948     if __name__ == "__main__":
949         main()
950

```

问题 4-2 相关代码

```

1
2     from __future__ import annotations
3
4     import argparse
5     from datetime import datetime
6     import json
7     from pathlib import Path
8     from time import perf_counter
9
10    import numpy as np
11    import openpyxl
12    from openpyxl.styles import Alignment, Font, PatternFill
13    from openpyxl.utils import get_column_letter
14
15    ROOT = Path(__file__).resolve().parents[2]
16    APPENDIX_DIR = ROOT / "附件"
17    RESULT_DIR = ROOT / "result" / "problem04"
18    PRED_PRICE_PATH = RESULT_DIR / "prediction" / "pred_price_rolling.xlsx"
19    PRED_LOAD_PV_PATH = ROOT / "result" / "problem02" / "prediction" / "prediction.xlsx"
20
21    from src.problem02 import problem
22
23
24    DEFAULT_PRICE = PRED_PRICE_PATH
25    DEFAULT_OUTPUT = RESULT_DIR / "result4-2.xlsx"
26    OUTPUT_START = problem.OUTPUT_START
27    OUTPUT_END = problem.OUTPUT_END
28
29    def load_predicted_prices(path: Path) -> dict:
30        workbook = openpyxl.load_workbook(path, read_only=True, data_only=True)
31        try:

```

```

32     rows = list(workbook.active.iter_rows(values_only=True))
33     finally:
34         workbook.close()
35         result = {}
36         for row in rows[1:]:
37             if row[0] is None:
38                 continue
39             values = np.asarray(row[1:145], dtype=float)
40             if values.shape != (144,) or not np.isfinite(values).all():
41                 raise ValueError(f"{row[0]} 的预测电价无效")
42             result[problem.excel_date(row[0])] = np.maximum(0.0, values)
43             if (
44                 len(result) != 334
45                 or min(result) != OUTPUT_START
46                 or max(result) != OUTPUT_END
47             ):
48                 raise ValueError("预测电价必须覆盖 2025-02-01 至 2025-12-31")
49             return result
50
51
52     def load_actual_prices(path: Path) -> dict:
53         """ 读取附件 4 的实际电价，用于 1 月储能预热。 """
54         workbook = openpyxl.load_workbook(path, read_only=True, data_only=True)
55         try:
56             rows = list(workbook.active.iter_rows(values_only=True))
57             finally:
58                 workbook.close()
59                 result = {}
60                 for row in rows[1:]:
61                     if row[0] is None:
62                         continue
63                     values = np.asarray(row[1:145], dtype=float)
64                     if values.shape != (144,) or not np.isfinite(values).all():
65                         raise ValueError(f"{row[0]} 的实际电价无效")
66                     result[problem.excel_date(row[0])] = np.maximum(0.0, values)
67                 return result
68
69
70     def build_forecast_maps(
71         attachment2: Path,
72         prediction_path: Path,
73     ) -> tuple[list, dict, dict]:
74         jan_dates, jan_load, jan_pv = problem.build_january_forecasts(attachment2)
75         pred_dates, pred_load, pred_pv = problem.load_prediction_series(prediction_path)
76
77         load_map = {
78             target: jan_load[index].copy()
79             for index, target in enumerate(jan_dates)
80         }
81         pv_map = {

```

```

82         target: jan_pv[index].copy()
83         for index, target in enumerate(jan_dates)
84     }
85     load_map.update(
86     {
87         target: pred_load[index].copy()
88         for index, target in enumerate(pred_dates)
89     }
90     )
91     pv_map.update(
92     {
93         target: pred_pv[index].copy()
94         for index, target in enumerate(pred_dates)
95     }
96     )
97     return jan_dates, load_map, pv_map
98
99
100 def write_result4_2(
101     output: Path,
102     records: list[tuple],
103     planning_price_by_date: dict,
104 ) -> None:
105     workbook = openpyxl.Workbook()
106     workbook.remove(workbook.active)
107
108     plan_sheet = workbook.create_sheet("计划购电量")
109     plan_sheet.append(
110         ["日期\\时间", *[problem.interval_label(t) for t in range(144)],
111         "全天购电量", "全天购电费"]
112     )
113     for target, plan, actual in records:
114         plan_sheet.append(
115             [
116                 datetime.combine(target, datetime.min.time()),
117                 *plan.purchase_kwh.tolist(),
118                 float(plan.purchase_kwh.sum()),
119                 float(planning_price_by_date[target] @ plan.purchase_kwh),
120             ]
121         )
122
123     battery_sheet = workbook.create_sheet("充放电电量")
124     battery_sheet.append(["日期", "时间段", "充电量", "放电量", "时刻", "储电量"])
125     for target, _, actual in records:
126         for block in range(6):
127             lo, hi = block * 24, (block + 1) * 24
128             battery_sheet.append(
129                 [
130                     datetime.combine(target, datetime.min.time()),
131                     f"{problem.interval_label(lo).split('-')[0]}-"

```

```

132     f"{problem.interval_label(hi - 1).split('-')[1]}",
133     float(actual.charge_kwh[lo:hi].sum()),
134     float(actual.discharge_kwh[lo:hi].sum()),
135     "0:00" if block == 0 else ("24:00" if block == 1 else None),
136     (
137         actual.storage_initial_kwh
138         if block == 0
139         else actual.storage_end_kwh[-1]
140     )
141     if block in (0, 1)
142     else None,
143 ]
144 )
145
146 emergency_sheet = workbook.create_sheet("紧急购电量")
147 emergency_sheet.append(["日期", "购电时间段", "购电量"])
148 for target, _, actual in records:
149     if not actual.emergency_runs:
150         emergency_sheet.append(
151             [datetime.combine(target, datetime.min.time()), None, None]
152         )
153         continue
154     for index, (label, amount) in enumerate(actual.emergency_runs):
155         emergency_sheet.append(
156             [
157                 datetime.combine(target, datetime.min.time())
158                 if index == 0
159                 else None,
160                 label,
161                 amount,
162             ]
163         )
164
165     for worksheet in workbook.worksheets:
166         for cell in worksheet[1]:
167             cell.fill = PatternFill("solid", fgColor="D9EAF7")
168             cell.font = Font(name="宋体", size=10, bold=True)
169             cell.alignment = Alignment(horizontal="center", vertical="center")
170         worksheet.freeze_panes = "B2"
171         worksheet.column_dimensions["A"].width = 13
172         for column in range(2, worksheet.max_column + 1):
173             worksheet.column_dimensions[get_column_letter(column)].width = 11
174         output.parent.mkdir(parents=True, exist_ok=True)
175         workbook.save(output)
176
177
178 def main(argv=None) -> dict:
179     parser = argparse.ArgumentParser(description=__doc__)
180     parser.add_argument("--price", type=Path, default=DEFAULT_PRICE)
181     parser.add_argument("--output", type=Path, default=DEFAULT_OUTPUT)

```

```

182     parser.add_argument("--summary-json", type=Path)
183     args = parser.parse_args(argv)
184
185     started = perf_counter()
186     attachment2 = APPENDIX_DIR / "附件 2.xlsx"
187     attachment4 = APPENDIX_DIR / "附件 4.xlsx"
188     predicted_price_by_date = load_predicted_prices(args.price)
189     actual_price_by_date = load_actual_prices(attachment4)
190     planning_price_by_date = {
191         target: actual_price_by_date[target]
192         for target in actual_price_by_date
193         if target < OUTPUT_START
194     }
195     planning_price_by_date.update(predicted_price_by_date)
196
197     actual_dates, actual_load, actual_pv = problem.load_actual_series(attachment2)
198     _, load_map, pv_map = build_forecast_maps(
199         attachment2, PRED_LOAD_PV_PATH
200     )
201     actual_by_date = {
202         target: index for index, target in enumerate(actual_dates)
203     }
204
205     residual_history = []
206     storage = problem.SOC_INITIAL
207     records = []
208     for target in actual_dates:
209         actual_index = actual_by_date[target]
210         q = (
211             np.quantile(
212                 np.asarray(residual_history[-problem.RESIDUAL_WINDOW_DAYS:]),
213                 problem.RISK_QUANTILE,
214                 axis=0,
215                 method="linear",
216             )
217             if residual_history
218             else np.zeros(144)
219         )
220         forecast_net = (load_map[target] - pv_map[target]) * problem.DT
221         price = planning_price_by_date[target]
222         plan = problem.solve_plan_day(forecast_net + q, price, storage)
223         actual = problem.simulate_actual_day(
224             plan.purchase_kwh,
225             actual_load[actual_index],
226             actual_pv[actual_index],
227             storage,
228         )
229         storage = float(actual.storage_end_kwh[-1])
230         residual_history.append(
231             (actual_load[actual_index] - actual_pv[actual_index]) * problem.DT

```

```

232     - forecast_net
233 )
234 if problem.OUTPUT_START <= target <= problem.OUTPUT_END:
235     records.append((target, plan, actual))
236
237 write_result4_2(args.output, records, planning_price_by_date)
238 plan_kwh = sum(float(plan.purchase_kwh.sum()) for _, plan, _ in records)
239 plan_cost = sum(
240     float(planning_price_by_date[target] @ plan.purchase_kwh)
241     for target, plan, _ in records
242 )
243 emergency_kwh = sum(float(actual.emergency_kwh.sum()) for _, _, actual in records)
244 emergency_cost = sum(
245     float(5 * actual_price_by_date[target] @ actual.emergency_kwh)
246     for target, _, actual in records
247 )
248 print(f"求解日期: {records[0][0]} 至 {records[-1][0]}")
249 print(f"计划购电量: {plan_kwh:.6f} kWh")
250 print(f"计划购电费: {plan_cost:.6f} 元")
251 print(f"紧急购电量: {emergency_kwh:.6f} kWh")
252 print(f"紧急购电费: {emergency_cost:.6f} 元")
253 print(f"总费用: {plan_cost + emergency_cost:.6f} 元")
254 print(f"期末储电量: {storage:.6f} kWh")
255 print(f"总耗时: {perf_counter() - started:.3f} 秒")
256 print(f"结果已保存: {args.output}")
257 summary = {
258     "plan_kwh": plan_kwh,
259     "plan_cost": plan_cost,
260     "adjusted_kwh": plan_kwh,
261     "adjustment_cost": 0.0,
262     "emergency_kwh": emergency_kwh,
263     "emergency_cost": emergency_cost,
264     "total_cost": plan_cost + emergency_cost,
265     "storage_end": storage,
266     "seconds": perf_counter() - started,
267     "output": str(args.output),
268 }
269 if args.summary_json is not None:
270     args.summary_json.parent.mkdir(parents=True, exist_ok=True)
271     args.summary_json.write_text(
272         json.dumps(summary, ensure_ascii=False, indent=2),
273         encoding="utf-8",
274     )
275     return summary
276
277
278 if __name__ == "__main__":
279     main()
280

```

问题 4-3 相关代码

```
1
2     from __future__ import annotations
3
4     import argparse
5     from dataclasses import replace
6     from datetime import datetime
7     import json
8     from pathlib import Path
9     from time import perf_counter
10
11     import numpy as np
12     import openpyxl
13     from openpyxl.styles import Alignment, Font, PatternFill
14     from openpyxl.utils import get_column_letter
15
16     ROOT = Path(__file__).resolve().parents[2]
17     APPENDIX_DIR = ROOT / "附件"
18     RESULT_DIR = ROOT / "result" / "problem04"
19     PRED_PRICE_PATH = RESULT_DIR / "prediction" / "pred_price_rolling.xlsx"
20
21     from src.problem03 import problem as q3model
22
23
24     DEFAULT_PRICE = PRED_PRICE_PATH
25     DEFAULT_OUTPUT = RESULT_DIR / "result4-3.xlsx"
26     OUTPUT_START = q3model.OUTPUT_START
27     OUTPUT_END = q3model.OUTPUT_END
28
29     def load_predicted_prices(path: Path) -> dict:
30         workbook = openpyxl.load_workbook(path, read_only=True, data_only=True)
31         try:
32             rows = list(workbook.active.iter_rows(values_only=True))
33             finally:
34                 workbook.close()
35             result = {}
36             for row in rows[1:]:
37                 if row[0] is None:
38                     continue
39                 values = np.asarray(row[1:145], dtype=float)
40                 if values.shape != (144,) or not np.isfinite(values).all():
41                     raise ValueError(f"{row[0]} 的预测电价无效")
42                 result[q3model.parse_date(row[0])] = np.maximum(0.0, values)
43             if (
44                 len(result) != 334
45                 or min(result) != OUTPUT_START
46                 or max(result) != OUTPUT_END
47             ):
```

```

48     raise ValueError("预测电价必须覆盖 2025-02-01 至 2025-12-31")
49     return result
50
51
52     def load_actual_prices(path: Path) -> dict:
53         """ 读取附件 4 的实际电价，用于 1 月储能预热。"""
54         workbook = openpyxl.load_workbook(path, read_only=True, data_only=True)
55         try:
56             rows = list(workbook.active.iter_rows(values_only=True))
57             finally:
58                 workbook.close()
59             result = {}
60             for row in rows[1:]:
61                 if row[0] is None:
62                     continue
63                 values = np.asarray(row[1:145], dtype=float)
64                 if values.shape != (144,) or not np.isfinite(values).all():
65                     raise ValueError(f"{row[0]} 的实际电价无效")
66                 result[q3model.parse_date(row[0])] = np.maximum(0.0, values)
67             return result
68
69
70     def write_result4_3(output: Path, records: list) -> None:
71         workbook = openpyxl.Workbook()
72         workbook.remove(workbook.active)
73         time_headers = [q3model.interval_label(t) for t in range(144)]
74
75         plan_sheet = workbook.create_sheet("计划购电量")
76         plan_sheet.append(["日期\\时间", *time_headers, "全天购电量", "全天购电费"])
77         for result in records:
78             plan_sheet.append(
79                 [
80                     datetime.combine(result.target_date, datetime.min.time()),
81                     *result.plan_purchase.tolist(),
82                     float(result.plan_purchase.sum()),
83                     result.plan_cost,
84                 ]
85             )
86
87         adjusted_sheet = workbook.create_sheet("调整购电量")
88         adjusted_sheet.append(["日期\\时间", *time_headers, "全天调整购电量", "全天调整费用"])
89         for result in records:
90             adjusted_sheet.append(
91                 [
92                     datetime.combine(result.target_date, datetime.min.time()),
93                     *result.adjusted_purchase.tolist(),
94                     float(result.adjusted_purchase.sum()),
95                     result.adjustment_cost,
96                 ]
97             )

```

```

98
99     battery_sheet = workbook.create_sheet("充放电电量")
100     battery_sheet.append(["日期", "时间段", "充电量", "放电量", "时刻", "储电量"])
101     for result in records:
102         for block in range(6):
103             lo, hi = block * 24, (block + 1) * 24
104             battery_sheet.append(
105                 [
106                     datetime.combine(result.target_date, datetime.min.time()),
107                     f"{q3model.interval_label(lo).split('-')[0]}-",
108                     f"{q3model.interval_label(hi - 1).split('-')[1]}",
109                     float(result.charge[lo:hi].sum()),
110                     float(result.discharge[lo:hi].sum()),
111                     "0:00" if block == 0 else ("24:00" if block == 1 else None),
112                     (
113                         result.storage_initial
114                         if block == 0
115                         else result.storage_end[-1]
116                     )
117                     if block in (0, 1)
118                     else None,
119                 ]
120             )
121
122     emergency_sheet = workbook.create_sheet("紧急购电量")
123     emergency_sheet.append(["日期", "购电时间段", "购电量"])
124     for result in records:
125         runs = q3model.merge_emergency_runs(result.emergency)
126         if not runs:
127             emergency_sheet.append(
128                 [datetime.combine(result.target_date, datetime.min.time()), None, None]
129             )
130             continue
131         for index, (label, amount) in enumerate(runs):
132             emergency_sheet.append(
133                 [
134                     datetime.combine(result.target_date, datetime.min.time())
135                     if index == 0
136                     else None,
137                     label,
138                     amount,
139                 ]
140             )
141
142     for worksheet in workbook.worksheets:
143         for cell in worksheet[1]:
144             cell.fill = PatternFill("solid", fgColor="D9EAF7")
145             cell.font = Font(name="宋体", size=10, bold=True)
146             cell.alignment = Alignment(horizontal="center", vertical="center")
147             worksheet.freeze_panes = "B2"

```

```

148     worksheet.column_dimensions["A"].width = 13
149     for column in range(2, worksheet.max_column + 1):
150         worksheet.column_dimensions[get_column_letter(column)].width = 11
151         output.parent.mkdir(parents=True, exist_ok=True)
152     workbook.save(output)
153
154
155     def main(argv=None) -> dict:
156         parser = argparse.ArgumentParser(description=__doc__)
157         parser.add_argument("--price", type=Path, default=DEFAULT_PRICE)
158         parser.add_argument("--output", type=Path, default=DEFAULT_OUTPUT)
159         parser.add_argument("--summary-json", type=Path)
160         args = parser.parse_args(argv)
161
162         started = perf_counter()
163         attachment4 = APPENDIX_DIR / "附件 4.xlsx"
164         predicted_price_by_date = load_predicted_prices(args.price)
165         actual_price_by_date = load_actual_prices(attachment4)
166         planning_price_by_date = {
167             target: actual_price_by_date[target]
168             for target in actual_price_by_date
169             if target < OUTPUT_START
170         }
171         planning_price_by_date.update(predicted_price_by_date)
172
173         inputs = q3model.load_inputs()
174         actual_dates = inputs["actual_dates"]
175         actual_load = inputs["actual_load"]
176         actual_pv = inputs["actual_pv"]
177         load_forecasts = inputs["load_forecasts"]
178         pv_forecasts = inputs["pv_forecasts"]
179
180         missing_prices = [
181             target for target in actual_dates
182             if target not in planning_price_by_date
183         ]
184         if missing_prices:
185             raise ValueError(f"预测电价缺少日期: {missing_prices[:5]}")
186
187         actual_by_date = {
188             target: index for index, target in enumerate(actual_dates)
189         }
190         corrector = q3model.PvForecastCorrector(
191             pv_forecasts,
192             window=q3model.PV_SHAPE_WINDOW,
193             use_level=True,
194             use_shape=True,
195         )
196         residual_history = []
197         storage = q3model.SOC_INITIAL

```

```

198     records = []
199
200     for target in actual_dates:
201         corrector.fit(target)
202         risk_margin = (
203             q3model.segmented_risk_margin(
204                 np.asarray(residual_history[-q3model.RESIDUAL_WINDOW_DAYS:]),
205                 q3model.RISK_QUANTILE_OVERNIGHT,
206                 q3model.RISK_QUANTILE_DAY,
207             )
208         if residual_history
209         else np.zeros(144)
210     )
211     actual_index = actual_by_date[target]
212     base_load = load_forecasts[target]
213     result = q3model.solve_day(
214         target,
215         planning_price_by_date[target],
216         base_load,
217         corrector,
218         actual_load[actual_index],
219         actual_pv[actual_index],
220         storage,
221         risk_margin,
222     )
223     actual_price = actual_price_by_date[target]
224     actual_emergency_cost = float(
225         5 * actual_price @ result.emergency
226     )
227     result = replace(
228         result,
229         emergency_cost=actual_emergency_cost,
230     )
231     storage = float(result.storage_end[-1])
232
233     forecast_pv0 = corrector.expand(
234         pv_forecasts[(target, 0)], 0, target
235     )
236     forecast_net0 = (base_load - forecast_pv0) * q3model.DT
237     actual_net = (
238         actual_load[actual_index] - actual_pv[actual_index]
239     ) * q3model.DT
240     residual_history.append(actual_net - forecast_net0)
241     corrector.observe(target, actual_pv[actual_index])
242
243     if q3model.OUTPUT_START <= target <= q3model.OUTPUT_END:
244         records.append(result)
245
246     write_result4_3(args.output, records)
247     total_plan_kwh = sum(float(r.plan_purchase.sum()) for r in records)

```

```

248     total_adjusted_kwh = sum(float(r.adjusted_purchase.sum()) for r in records)
249     total_plan_cost = sum(r.plan_cost for r in records)
250     total_adjustment_cost = sum(r.adjustment_cost for r in records)
251     total_emergency_kwh = sum(float(r.emergency.sum()) for r in records)
252     total_emergency_cost = sum(r.emergency_cost for r in records)
253
254     print(f"求解日期: {records[0].target_date} 至 {records[-1].target_date}")
255     print(f"计划购电量: {total_plan_kwh:.6f} kWh")
256     print(f"调整后购电量: {total_adjusted_kwh:.6f} kWh")
257     print(f"计划购电费: {total_plan_cost:.6f} 元")
258     print(f"调整费用: {total_adjustment_cost:.6f} 元")
259     print(f"紧急购电量: {total_emergency_kwh:.6f} kWh")
260     print(f"紧急购电费: {total_emergency_cost:.6f} 元")
261     print(
262         "总费用: "
263         f"{total_plan_cost + total_adjustment_cost + total_emergency_cost:.6f} 元"
264     )
265     print(f"期末储电量: {storage:.6f} kWh")
266     print(f"总耗时: {perf_counter() - started:.3f} 秒")
267     print(f"结果已保存: {args.output}")
268     summary = {
269         "plan_kwh": total_plan_kwh,
270         "plan_cost": total_plan_cost,
271         "adjusted_kwh": total_adjusted_kwh,
272         "adjustment_cost": total_adjustment_cost,
273         "emergency_kwh": total_emergency_kwh,
274         "emergency_cost": total_emergency_cost,
275         "total_cost": (
276             total_plan_cost + total_adjustment_cost + total_emergency_cost
277         ),
278         "storage_end": storage,
279         "seconds": perf_counter() - started,
280         "output": str(args.output),
281     }
282     if args.summary_json is not None:
283         args.summary_json.parent.mkdir(parents=True, exist_ok=True)
284         args.summary_json.write_text(
285             json.dumps(summary, ensure_ascii=False, indent=2),
286             encoding="utf-8",
287         )
288     return summary
289
290
291     if __name__ == "__main__":
292         main()
293

```